

Admissible

A fail-closed admissibility kernel for agents that do not repeat.

Roque Briceño. Version 0.8.0, 30 August 2026. CC BY 4.0.

Reference implementation: **fcd/core.py** (identity), **rga/core.py** (scrutiny), **rga/calibration.py** (standing).

What this volume is

The complete 0.8.0 technical-report set in one document: the composition paper, the scrutiny and standing paper, the identity paper, each layer's premises, invariants and proofs, plain-language companions, and the machine-readable contracts and deterministic bench record.

The predicate the whole stack computes is $\text{admissible}(\text{id}) := \text{id} \in S_R \wedge \text{mediated}(\text{id}) \wedge \neg \text{tainted}(\text{id}) \wedge \neg \text{impeached}(\text{id})$: sealed under scrutiny, counter-signed by the calibration authority, no instrument it relied on later caught unreliable, and no escape standing against it. Each conjunct is the top of one theorem family.

Every guarantee names the transition that enforces it. The R and C proofs cite file:line:symbol, bound by a move-detecting test; every R and C guard is a named method with a delete-the-guard proof. Each report states what is not proved as the theorems' complement.

Contents

Part I — The composition. The predicate, threat model, three layers, composition theorems, methodology, evaluation, related work and limitations.

Part II — Scrutiny and standing. Refutation-gated admission and the escape ledger, including the premise, invariants R1–R13 and C1–C7, and proofs.

Part III — Identity. Fail-closed class dispatch, its invariants I1–I17, and proofs.

Part IV — In plain words. The theorem set without notation and the vocabulary exposed by the reference interface.

Part V — Contracts and records. The journal event contract, deterministic kernel bench, and section map.

The standalone PDFs are generated from the same Markdown and figure sources as this volume. Repository READMEs and review logs are intentionally not reprinted as paper content.

Part I — The composition

One machine, three layers, one predicate. This part states what the composition buys over its parts and proves the two composition theorems; the layers' own invariants are inherited by citation, never re-derived.

Admissible: A Fail-Closed Admissibility Kernel for Agents That Do Not Repeat

Roque Briceño

Version 0.8.0. 30 August 2026. Licensed under CC BY 4.0.

Companion reports: *Fail-Closed Class Dispatch* (layer I) and *Refutation-Gated Admission* (layers R and C). Each report stands on its own, with its own invariants, proofs and review rounds; this paper is the composition — one machine, one predicate, and the two theorems the composition earns. It adds framing and two composition results, not new mathematics, and says so.

Abstract

A system that delegates work to large language models cannot certify an output by naming its author: the same worker, prompt and context produce a different artifact on every run. We present an integrity kernel for this setting, built as three fail-closed transition systems composed by non-interference, whose union of guarantees is the soundness of a single query, *admissible(id)*, and the loudness of its complement.

Layer **I** (fail-closed class dispatch) proves *identity*: a stage passes only when the executed model equals the one bound from a class-scoped allow set, with no fallback edge; seventeen invariants over the binding, envelope, receipts and memory promotion. Layer **R** (refutation-gated admission) proves *scrutiny*: an artifact seals only when declared, deterministic, replayable refuters — pinned before generation, measured against named defect models, run over *k* concordant samples — were given a fair chance to kill its claims and failed, with the measured power carried on the seal and never raised afterwards. Layer **C** (the escape ledger) proves *standing*: a defect later found in sealed work, demonstrated as a counterfactual trial of the very instrument the seal trusted, mechanically impeaches the seal, charges every vouching refuter, and can never be silently forgotten by any successor policy.

The composed kernel is a reference monitor for machine work-products, in the classical sense: complete mediation of the stores, journals whose replay re-derives every transition and compares it to the supplied record field by field — refusing inconsistent alteration, forgery and duplication, while coherent root rewrites and deletion require a previously trusted authenticated head — and verifiability by construction — every guard is a named method that a mutation suite deletes one at a time, proving each individually load-bearing, and every proof cites the file and line that enforces it, bound by a test. The guarantees are about the *record*, never about truth: what is proved is that the paperwork cannot lie about who ran, what was survived at what measured strength, and what has been found against it since. This is the admissibility standard of evidence law — provenance, known error rate, controlling procedure, impeachment — enforced by a kernel, and it deliberately certifies exactly what a court's admission does: procedure, never truth.

1. Introduction

1.1 The problem

Delegation to stochastic generators breaks the oldest assumption in software integrity: that knowing *which* component ran tells you *what* it did. A deterministic function's identity determines its output; an agent's does not. Three questions follow, in order, and each is the whole subject of one layer:

1. **Who actually did the work?** Control planes name one model while data planes call another; a dead bind can silently continue on a fallback; a reviewer can be the author. Without identity, nothing downstream is attributable.
2. **What was the work subjected to before acceptance?** A green check from an empty test suite and one from a suite that kills 95% of seeded defects print identically. Survival without measured severity launders weak scrutiny into strong.
3. **What happens when accepted work turns out to be wrong anyway?** In most pipelines: nothing mechanical. The miss is fixed, the instruments that vouched keep vouching, and the next test battery is free to forget the defect class entirely.

1.2 The predicate

The kernel's guarantees compose into one query, exposed by the implementation and defined here:

admissible(id) := $id \in S_R \wedge mediated(id) \wedge \neg tainted(id) \wedge \neg impeached(id)$

where S_R is the sealed store (written only by Seal, which requires the identity layer's Accept), *tainted* means a refuter the seal relied on was later caught nondeterministic, and *impeached* means a valid escape stands against a sealed claim. Each conjunct is the top of one theorem family: membership in S_R rests on identity (I1–I17, inherited by citation) and scrutiny (R1–R13); the two negative conjuncts rest on standing (C1–C7). The project takes its name from this predicate.

The name is also an argument. Evidence law admits testimony not because it is true but because it satisfies *procedure*: established provenance and custody, a known error rate, controlling standards, and exposure to cross-examination — with impeachment available afterwards. The Daubert factors for expert evidence (testability, known or potential error rate, maintenance of standards; *Daubert v. Merrell Dow*, 1993, itself citing Popper) map onto this kernel exactly: refuters are the testability requirement, carried power is the known error rate — in ledger mode exact on the named defect model D , in bounded mode a computed function of declared (ϵ, N) and no more trustworthy than ϵ ; either way a claim about the instrument, never the generator's real-world error rate, pinned versioned policy is the controlling standard, and the escape ledger is impeachment. The analogy is honest precisely where the theorems are: admissibility certifies procedure, never truth, and the seal says so on its face.

1.3 Contributions

1. A three-layer, fail-closed transition system in which **identity, scrutiny and standing** are each enforced by a kernel with inductive invariants (I1–I17, R1–R13, C1–C7), composed by two non-interference lemmas (R11, C7) so that each layer's proofs survive composition unchanged (§6).
2. **Measured doubt as a carried record**: power enters a seal only as a kernel-counted kill rate against a content-addressed defect model, or a computed bound from declared, seal-visible parameters — never as a scalar anyone wrote down — write-once, with composition across refuters only by measured union or labelled max (§4).
3. **Escapes as counterfactual trials**: a post-acceptance defect report that is a re-run of the *pinned* refuter at a finder-chosen input, adding no trust assumption the seal did not already carry; from it,

non-discretionary impeachment, per-cell charge totality, a coverage ratchet on policy succession, and query-shaped demotion (§5). To our knowledge the non-discretion and charge-totality properties have no prior formalization (§10).

4. A **verification methodology** matched to the claims: every guard is an individually deletable method with a red-first deletion proof; every proof citation is a file:line:symbol triple bound by a move-detecting test; premises were attacked by adversarial review *before* each kernel was written and the finished kernels re-attacked after, with all confirmed findings applied and journaled (§7).

5. An honest boundary, stated as theorems' complement: an explicit, consolidated *what is not proved* (§11), including the two empirical hypotheses the numbers rest on (defect-model coupling; corpus representativeness) and every trust assumption (A0–A13, B0–B14).

2. Setting and threat model

Parties. An *operator* owns policy and adjudication (trusted for governance, made visible, not defeated — B13). *Specialists* (generators) are untrusted and stochastic. *Refuters* are deterministic programs under a pinned hermetic runtime (B2). A *harness* executes and reports (honest per B1/A10 — the classical adapter boundary; its lies about deterministic components are falsifiable by replay, which is strictly more than can be said of its lies about stochastic ones). *Finders* of escapes are untrusted; their filings are checkable.

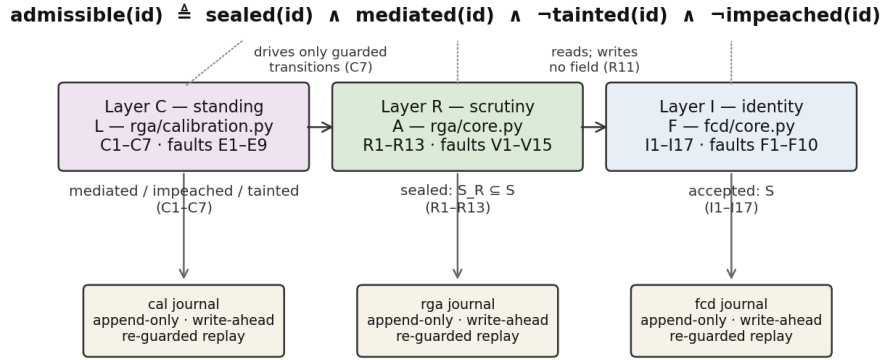
Machines. Three state machines with disjoint write sets, composed left-to-right by read-only observation and guarded interposition:

F (fcd/core.py) ← A (rga/core.py) ← L (rga/calibration.py)

A reads F's state and writes none of it (R11); L reads both and writes neither directly, driving A only through its own guarded transitions (C7). Every transition appends to an append-only journal; every machine rebuilds deterministically from its journal, re-checking on rebuild what it checked live (a property earned the hard way: the first two replay implementations were broken by review and repaired — §7).

Adversary. In scope: any sequence of public transitions by untrusted parties — mis-binding, fallback hopping, claim-shaping, refuter-shaping, tailoring outputs to known tests, witness multiplication, corpus flooding with true filings, inconsistent journal tampering detected at replay, and selection across retries. Coherent history replacement or deletion is detected only relative to a previously trusted authenticated publication. Out of scope, stated as assumptions: physical execution fidelity (A10/B1), runtime hermeticity (B2), hash second-preimage (A11/B7), author-identity strings (B11), operator governance (B13), authenticated-publication key/registry/bootstrap integrity (B14), and post-seal byte availability (B12). The full assumption tables are ../INVARIANTS.md §0 and ../RGA/INVARIANTS.md §0/§8.0.

Figure 6. One machine, three layers, one predicate



Each machine rebuilds from its journal re-checking every live guard; inconsistent records are refused, while historical authenticity requires an external current head.

3. Layer I: identity (fail-closed class dispatch)

A work item has a class c and a frozen body; policy gives $\pi(c)$, $\delta(c)$ and $\phi: A \rightarrow M$. The machine's rows are Open, Admit, Bind, Observe, Pass/PassRefuse, Close, Retry, Accept — with the load-bearing split that Bind writes only the *declared* identity and Observe alone writes the *executed* one, so the Pass guard $\text{norm}(m_{\text{exec}}) = \text{norm}(m_{\text{decl}})$ compares two independently sourced witnesses and can genuinely fail.

Selected theorems (full statements and proofs: ../INVARIANTS.md, ../PROOFS.md):

- **I1** $pc = \text{Passed} \Rightarrow \text{norm}(m_{\text{exec}}) = \text{norm}(m_{\text{decl}}) = \text{norm}(\phi(a))$.
- **I3** No Passed state has $\text{norm}(m_{\text{exec}})$ outside $\{\text{norm}(\phi(x)) : x \in \pi^*(c) \setminus \delta(c)\}$ — this machine has no leftover-hop edge.
- **I5/I8** Accepted means every required stage Passed; the store has exactly one writer.
- **I6** A check-stage admit excludes the item's authors: dual control.
- **I10–I17** pin the context envelope: package hashes, receipts, steering acknowledgement and cache identity all bind to the current attempt and nonce; memory promotes only through serialized compare-and-swap by accepted work.

Layer I is Clark–Wilson's enforcement rules applied to LLM binds — CDIs, TPs, validated writes, separation of duty — and its paper says so rather than claiming a new access-control algebra. Two of its guards were repaired during this project's reviews (a pc guard on NoAdmit; a fresh-id guard on Open), because the upper layers' proofs lean on I4 and I8 as *state* invariants; the repairs carry red-first tests.

4. Layer R: scrutiny (refutation-gated admission)

Layer I verifies the die, not the roll. Layer R moves the gate to the claim. A class pins, at Open and before any generation: formal claims (spec-hashed; the generator never authors what it is measured against), the refuter versions that attack each claim, sample count k , concordance threshold θ , and

power floor $p_{\min}$.

Samples are k FCD-passed write stages of one item under one bind key — identity inherited per sample by citation of I1/I2/I3 — each registered (kernel-hashed, single-holder) before the next stage may run, with a post-artifact nonce. **Trials** are one refuter against one claim on one sample, seeded by $H(v_i | h_i | r | \text{claim})$ so no seed exists before its artifact; verdicts are refuted/survived/inconclusive and only survival counts. **Power** enters only as a record: kills counted by the kernel over a ledger against a content-addressed defect model D whose id-set *and* *author* are fixed by first record, exact on D ; or $1-(1-\epsilon)^N$ computed from declared (ϵ, N) that the seal carries. **Concordance** is agreement of witness vectors with the *designated* sample (stage 0, fixed before anything ran) — a precondition that closes the gate, never an election.

Selected theorems (full: ../RGA/INVARIANTS.md §3, proofs with line citations in ../RGA/PROOFS.md):

- **R1** No admission without survival in every (claim, refuter, sample) cell; refuted and inconclusive close, published.
- **R2** Power is carried, never inferred: every figure on a seal equals a write-once, kernel-computed record; nothing raises it after the fact; composition across refuters is measured union over a shared D or a labelled max — never a product.
- **R3** Separation of duty, extended: refuter fixed before generation; authored by neither generator nor defect-model author; generator's package provably excluded refuter material; samples interleaved with their stages so slots cannot be assigned after seeing the batch.
- **R4** Concordance against the designated sample at θ , with (agreeing, k) on the seal so $k \geq 1$ is visibly trivial.
- **R5/R6** A replay divergence refuses a refuter monotonically, everywhere; every refuter used was replayed at least once on this line.
- **R8** $S_R \subseteq S$: everything sealed was first accepted by layer I .
- **R11** A writes no field of F — the composition lemma.

The seal carries: the artifact hash; per claim, each refuter's mode, power, D (or ϵ, N), kills and $|D|$; the composite and its label; (agreeing, k); the sampling configuration; the FCD identity; and an explicit list of what was *not* attacked, with its disposition. Faults V1–V15 name every forbidden step; V1–V5 publish, the rest refuse.

Figure 7. Anatomy of a seal (write-once; nothing on it can be raised later)

identity (layer I, by citation)	work item · class · frozen body hash · fcd policy version · generator · executed model
artifact	hash of the exact bytes accepted — the served bytes, byte-checked at Accept
sampling	k samples · designated sample = stage 0 · sampling config hash · post- artifact nonces
per claim	claim id · spec hash (authored outside the generator) · pinned refuter versions
per refuter	mode (ledger bounded) · power = kills/ D against content-addressed D (author fixed by first record), or $1 - (1 - \epsilon)^N$ from declared (ϵ , N) · kills · D
composite	union over shared D, or labelled max — never a product · (agreeing, k) so k=1 is visibly trivial
residual	every claim NOT attacked, with disposition: check_stage unreviewed — the seal names what it does not cover
standing (layer C, queries)	mediated? impeached? tainted? — never stored on the seal; computed against the retained escape ledger, with authenticated current heads making deletion of an anchored prefix loud

5. Layer C: standing (the escape ledger)

R leaves one loop open, and its own limits name it: kill-rate on D is power against real defects only under the coupling hypothesis, and nothing forces a found miss to have consequences. Layer C closes the loop that can be closed.

An **escape** against a sealed claim is, in its automatic form (tier A), a *counterfactual trial*: the claim's pinned refuter re-run at a finder-chosen nonce on the sealed bytes — the finder supplies only the nonce and the bytes; the kernel hashes the bytes against the seal, derives the seed itself, and requires verdict refuted plus one identical replay. Kills are sound regardless of proposer, and this channel adds no assumption the seal did not already carry. Any other checker is tier B: no effect of any kind until a named, journaled adjudication — the per-escape claim-fidelity judgment made visible rather than pretended away.

From one valid escape, mechanically:

- **C3 Impeachment, non-discretionary**: impeached(id) is entailed by the escape — a pure query; the seal is immutable; no authority can decline the revocation, because the revocation carries its own replayable proof. Validity degrades only by journal facts (checker discredited or refused).
- **C2 Charge totality**: every refuter that vouched is charged — read from the journal, never declared — exactly once per (line, claim, refuter\ version) cell, so witness and checker multiplicity cannot inflate.

- **C4 The ratchet:** no successor policy installs unless its ledger defect models cover the class's valid corpus minus journaled, attributed exclusions; a class owing coverage cannot be dropped; the install event (which now exists) carries the primaries and the diff. *Forgetting is loud.*
- **C6 Demotion:** charges past a declared budget block new pins unconditionally and gate sealing where the class declared it — a pure function of the valid charge set, falling when validity degrades (refusal-shaped facts outrank the count), never an event cascade, never a rewrite.
- **C5** Every seal issued through the authority is stamped, same-step, with its instruments' track record and the corpus's finder-provenance split — primaries, never a rate, with the stated reading that zero charges is absence of filed evidence.
- **C7** L writes no lower-layer field directly — the second composition lemma.

6. The composed guarantees

The unified results are compositions, proved by citation of the layer theorems plus the two non-interference lemmas. They are the paper's claim stated once:

Theorem 1 (Soundness of the record). If admissible(id) holds at journal position t , then the journals up to t contain, re-derivably: (i) for each of the k samples, a bind and an observation whose normalized identities agree with ϕ of a specialist admitted from $\pi^{\wedge}(c) \setminus \delta(c)$ [I1–I3, via R7]; (ii) *for every pinned claim and refuter, a surviving trial on every sample and at least one replayed trial per refuter on this line, seeded after the artifact it attacked, under power records the kernel computed, with concordance $\geq \theta$ against the designated sample and composite power $\geq p_{\min}$ [R1–R10, R12–R13]; and (iii) no valid escape standing against any sealed claim, no reliance on a refuter caught nondeterministic, and a same-step stamp of every instrument's charge record as of sealing — required by the predicate itself, not merely emitted beside it: a seal the calibration authority never mediated carries no stamp, and admissible refuses it as layer IR rather than IRC [C1–C3, C5].* Proof.* Each conjunct of admissible is the conclusion of the cited family; R11 and C7 guarantee the families hold on the combined trace because each upper machine's writes are disjoint from the lower machines' state. QED

Theorem 2 (Loudness of deviation). Every transition sequence that would falsify a conjunct of Theorem 1 either writes nothing (a guard refusal: faults F5–F10, V6–V15, E4/E6/E7/E9 raise) or appends a named fault to the journal before any effect (F1, F3, F4, V1–V5, E5 publish; F2 has no runtime-instance field to fire on and is listed as unproved, and E1–E3 and E8 are query-time validity and writer inspection rather than transitions, so they appear in neither bucket); and replay is *transition-local*: each of the three from_events re-checks, before every write, the guards live execution ran, so derived/output fields altered inconsistently, forged or duplicated transitions, reordering against preconditions, and removals that later events recompute against are refused — and every journal a live machine accepted replays unchanged. Replay does not authenticate which internally valid history is the anchored one. **Unauthenticated-history residue** includes a coherent rewrite of root transition inputs, a journal shortened at the tail, or a coherent removed group no surviving event recomputes against; each is indistinguishable from another honest history to replay alone. Deletion is the direction that can *raise* standing, since dropping an escape un-impeaches its line and dropping a refusal un-taints every seal that relied on it. Replay alone cannot close it. v0.5 adds a conditional authenticated-publication theorem: immutable authority-owned journals are sealed into three namespace-scoped ordered heads, successor receipts prove extension of the anchored cumulative event chain, the heads are accepted atomically, and the exact heads plus kernel-derived I/R/C predicates are bound into one signed receipt; verification rejects stale, coherently shortened, or longer-forked records. The theorem applies only when callers issue and verify that receipt, and still trusts signer/key custody, registry monotonicity, and

the single-writer authority model. One appended event has the same character: a calibration stamp forged for the most recent unmediated seal contradicts nothing earlier, so it is accepted, while one forged for any earlier seal breaks stamp order and is refused. Finally, membership checks on rebuild read the final Admission registry, so an install refused live for a missing Measure record can replay once a later record supplies it. Everything else in the transition-consistency class — inconsistent derived fields, forged transitions, duplicates, and deletions any later event recomputes against — is refused, because each machine compares its re-driven transitions to the journal field by field rather than trusting the file. *Proof.* By the fault tables and their executable checks: every V- and E-fault names its enforcing method, and tests/ demonstrates, for each method singly, that the forbidden state is unreachable with it and reached without it; every F-fault is driven to its refusal by the identity kernel's transition tests (`tests/test_invariants.py`, `tests/test_core.py`), which exercise the same table without per-guard deletion. QED

Neither theorem mentions truth. That is the point, and the limit (§11).

7. Methodology: executable proofs and adversarial rounds

The verification discipline is a contribution independent of the theorems it checks:

1. **Delete-the-guard.** Every guard of the scrutiny and standing kernels is a named method. For each, the suite runs a scenario twice: with the guard intact (forbidden state unreachable) and with the guard replaced by a no-op (forbidden state reached). A guard no deletion turns red fails the suite as dead code — and the discipline is applied to itself: clauses proved unreachable by construction were removed and derived instead, with the removals documented. The identity kernel predates this discipline: its guards are driven to refusal by transition tests over the same fault table, without per-guard deletion — a difference stated, not papered over.
2. **Citation binding.** Every R- and C-proof step cites file:line:symbol; a test opens the file and fails if the line no longer contains the symbol (the layer-I proofs predate the binder and carry file-level citations only). A move-detector, not a semantic check — the sentence claims enforcement, the test keeps it pointed at live code. It fired repeatedly during this project's own refactors, exactly as designed.
3. **Premise before kernel.** Each layer's design brief was attacked by independent adversarial reviewers *before* any code — five lenses and 38 verified findings for R; four lenses and 28 for C — with every finding handed to a skeptic instructed to refute it. Two of the author's pre-registered designs were killed at this stage (a Wilson-interval power bound; sibling-item sampling), which is the cheap time to kill them.
4. **Review after build.** The finished kernels were re-attacked (five lenses for R; two for C). Both rounds returned REVISE; all confirmed findings were applied, including five reproduced R-kernel defects, three of which invalidated published proofs, two latent soundness bugs in layer I itself, and a C-replay that trusted its own journal. Every round is journaled in `eval/reviews/`, and every repair carries a red-first test.

Current state: 524 executable Python checks (487 kernel/server/context/RGA/calibration/paper/custody and 37 evidence-store), of which 43 are per-guard deletion proofs and 2 are citation binders; all green. The cockpit adds 71 UI checks, for 595 repository checks total.

8. Implementation

Three kernels, pure Python 3.10+, no I/O, injectable clocks and nonce sources: `fcd/core.py` (identity), `rga/core.py` (scrutiny), `rga/calibration.py` (standing). State enters only as arguments; every fact about

the world is a report crossing a named assumption; every transition appends journal events matched by an explicit contract (metrics/SCHEMA.md); every machine rebuilds from its journal with re-guarded replay. The reference server and cockpit sit above the kernels and cannot pass, accept, seal or promote; a skin is a read-only projection. The store queries a deployment must consult — admissible, mediated, check_dependencies, suspect, tainted, impeached — are pure functions of journal state.

9. Evaluation protocol

Rates are computed only on a *named cut* $[t_0, t_1]$ with a class-derived window, from a write-ahead journal, with policy evaluated as-of each event — otherwise zeros are estimates biased clean, not results. Under that rule the repository carries three empirical studies, of decreasing internal validity and increasing external relevance, and the full account of every run — including the ones that turned out to be measuring the instrument — is eval/LOG.md, an append-only log whose convention, from the first real-defect detection run onward, is that each run's hypothesis, metric, adjudication rule and stopping rule are committed *before* the run and its result appended after; the four earlier entries are marked retrospective in their own headers.

The kernel bench (eval/bench/, Figure 8) drives all three layers over reference implementations of four stdlib-specified functions (median, wrap, normpath, quantiles), with the standard library as the strong refuter's differential oracle and a seeded AST mutator building the defect models. Its numbers — carried power, seal/refutation/discord outcomes, escapes established, demotions, ratchet coverage — are kernel queries on the named cut, reproduced byte-identically by tests/test_bench.py. Two of its three generator conditions are close to definitional and the figure now says so: the *honest* sample **is** the reference implementation that serves as the oracle, so 32/32 sealing is true by construction, and an honest line that failed to seal would be an instrument fault — as one was, when a property refuter flagged normpath("/") and refuted the reference implementation itself. The *sloppy* mutant is drawn from the defect model D that power is measured against, so its refutation is a within-model result that says nothing about a defect outside D. What the bench does establish is that the machinery is internally consistent under adversarial conditions, and it establishes one thing that is not definitional: **every latent-defect seal was impeached afterwards, 7 of 7**, while three unstable seals were not and stand as misses. That is layer C working, and its residue, on the same figure.

A real generator (eval/generators/) replaced the simulated one for 48 samples across those four targets, recorded into content-addressed corpora and replayed through the same seam, so determinism survives. **Not one sample was defective**, across 12 textually distinct normpath implementations, all 48 matching their oracle on 3,000 generated inputs. The gate refused none of them: 16 of 16 lines sealed, zero false refusals, which is the first measurement of the false-positive half against code a generator actually wrote — read with the corpus's own limitation, that nothing committed establishes the k samples within a line were independent draws, so concordance there is untested and the discord layer is not exercised by this corpus. The detection half could not be measured, and the reason is structural rather than a matter of adding targets: a target whose oracle is a well-known library function is one whose answer a modern generator recalls rather than derives, so the differential refuter ends up comparing the oracle to itself. No claim about any model follows; the corpora name no vendor and the run used one configuration throughout.

Real historical defects (eval/realdefects/, Figure 9) are the study that bears on coupling directly: 63 module-level functions drawn from BugsInPy, of which 42 could be run as a differential between their buggy and fixed commits under one interpreter, with inputs found by search rather than sampling and with no knowledge of the bug. It yields **eight defects separated and then hand-verified against the patch that closed them**, in youtube-dl, scrapy and thefuck, each with a re-runnable witness. It does **not** yield a detection rate, and the figure is drawn to refuse one. Five successive runs each carried an instrument fault invisible in its own summary statistic, in this order: a stubbed parent package that cost

two thirds of the corpus, a filter too narrow, an argument mutated between the two sides, a filter too broad, and finally an artifact rule that measurement afterwards showed never fires at all. Of the last run's 42 testable candidates, three are excluded by a guard bug rather than by the corpus, two of its ten separations are not the documented defect, and of six negatives adjudicated by hand three were overturned by an input written after reading the patch: "not separated" mostly means "not reached". The honest deliverable is the verified cases and a characterised instrument, and eval/realdefects/README.md names each remaining fault with its measured exposure.

None of this is admissible evidence and none of it is part of the gate; it is research code that fetches from the network and executes third-party source. The deployment-facing surface remains defined and empty: misbind, silent-fail, bleed and time-to-stage (layer I); refutation, discord, miss-observed, replay-divergence and escape-replay rates (layers R and C); all with their selection biases stated beside them. A deployment still owes the two empirical studies the theorems cannot supply: coupling of its defect models to its generators' real faults, and the escape corpus's coverage of what its finders actually find.

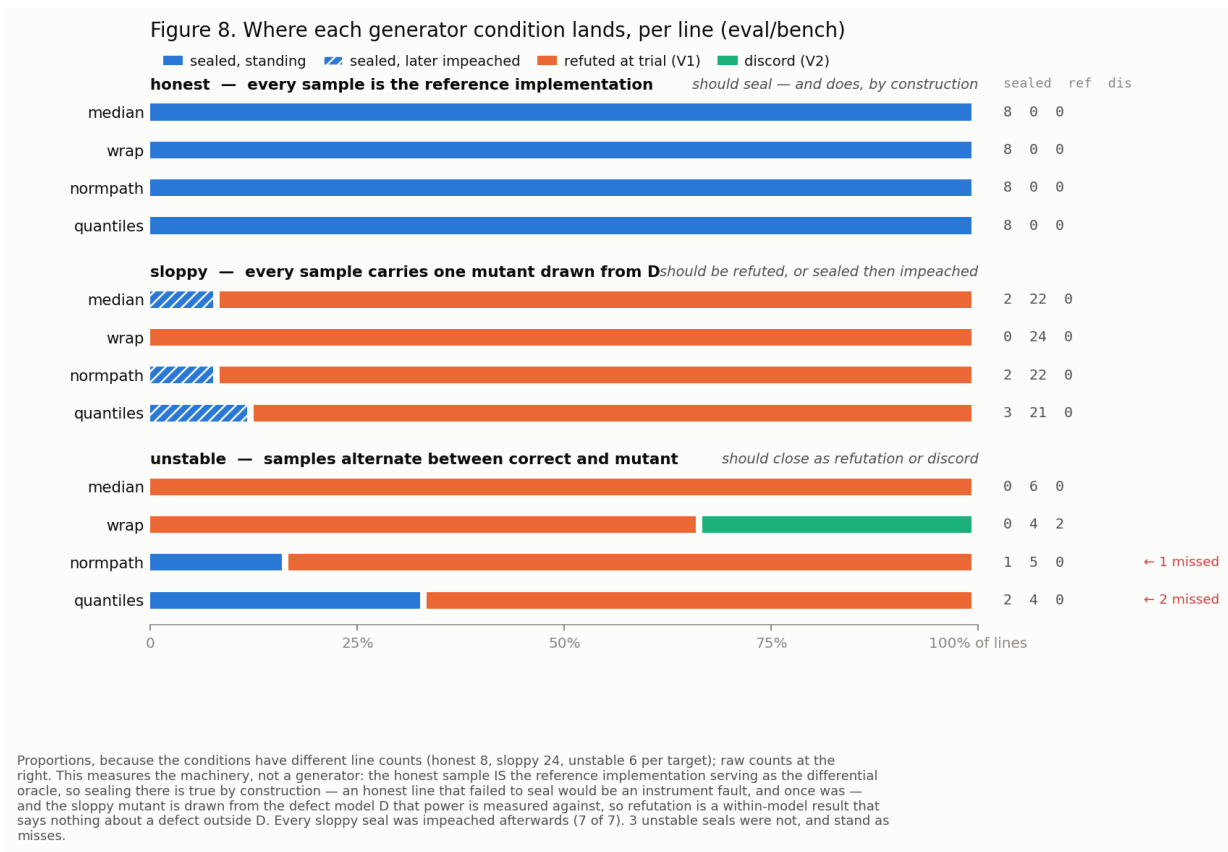
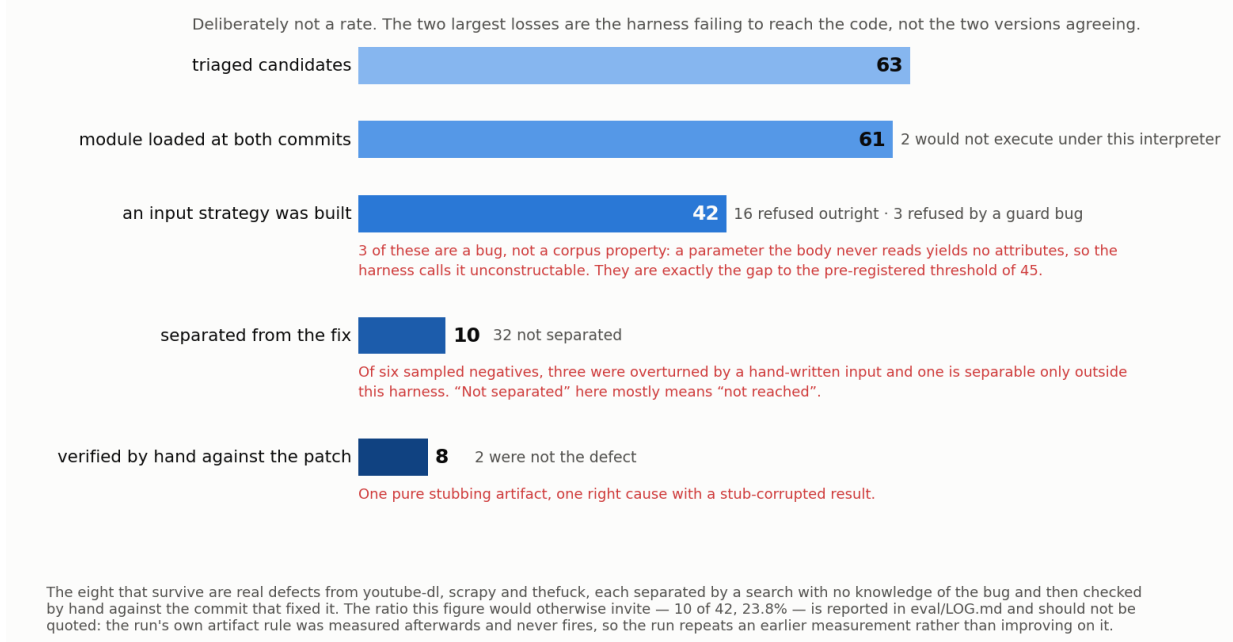


Figure 9. Where 63 real defects went (eval/realdefects)



10. Related work

Consolidated from the layer papers, which carry the full discussions. Foundations: Clark & Wilson (1987) — this stack is their model completed for stochastic workers: layer I is the enforcement rules, layer R the integrity verification procedure with its certification record, layer C the certification-maintenance discipline; Saltzer & Schroeder (1975) fail-safe defaults; Anderson's reference-monitor triad (1972) — complete mediation, tamper-evidence, verifiability — is what §6–§7 instantiate for work-products. Scrutiny: mutation analysis (DeMillo, Lipton & Sayward 1978; Jia & Harman 2011; Just et al. 2014), property testing and randomized checkers (Goldreich–Goldwasser–Ron 1998; Freivalds 1977), proof-carrying code as the power-1 corner (Necula & Lee 1996), semantic entropy as the graded statistic of which concordance is the zero-entropy gate (Kuhn, Gal & Farquhar 2023; Farquhar et al. 2024), N-version limits (Knight & Leveson 1986). Standing: eliminative argumentation and Assurance 2.0 defeater discipline (Goodenough, Weinstock & Klein 2015; Bloomfield & Rushby 2024) — mechanized here, with kernel-refused rather than assessor-flagged forgetting; regression-from-escapes and mined-fault mutation (Tufano et al. 2018; Beller et al. 2021); revocation infrastructure (CRL/OCSP/Sigstore) minus discretion; flaky-test quarantine and SRE error budgets as demotion's fail-open ancestors; adaptive conformal inference (Gibbs & Candès 2021) as the statistical-calibration theorem this stack deliberately does not claim. Contemporary neighbours (2026) circle the gap without the enforcement object — adversarial stage-gated review with LLM refuters, self-consistency admission as a soft score, attestation-aware promotion gates — surveyed in ../RGA/DRAFT.md §10. Evidence-law framing: *Daubert v. Merrell Dow* (1993); Popper (1959). To our knowledge, non-discretionary revocation entailed by a replayable counterexample, and mechanical charge totality against vouching checkers, have no prior formalization.

11. Limitations, and what is not proved

Stated before anything here is read as more than it is. **Truth:** survival is not correctness; impeachment's absence is not soundness; nothing in I, R or C says an artifact is good. **Coupling:** power is exact on a named D and transfers to a generator's real defects only under an empirical hypothesis, stated, not proved — and now attacked directly rather than left standing. §9's real-defect study is that attack, and its result is that the hypothesis remains open: the differential separates eight real historical defects from their fixes, which is evidence that the mechanism reaches real faults at all, but no rate survived scrutiny, four runs each hid an instrument fault behind a plausible-looking figure, and the corpus is filtered to module-level functions (63 of 501), so nothing about a population follows. Coupling is unproved as before; what has changed is that the gap has been attacked with real defects rather than left as an assertion, and that the instrument which would close it is characterised in `eval/realdefects/README.md`, including the faults that stop it. **Representativeness:** the escape corpus is a selected, adversarially steerable subsample of real defects; every aggregate built on it can be steered by parties who file only true things; silence is unreadable except through journaled audits whose reference is the corpus itself. **Reports:** harness honesty, runtime hermeticity, author identity, nonce unpredictability, byte availability and operator governance are assumptions (A10, B1, B2, B7, B9, B11–B13) — with the one asymmetry this stack buys: lies about deterministic components are falsifiable by replay after the fact. **Exposure:** a generator that has memorized a public refuter can game it; the post-artifact seed and package exclusions narrow this and do not close it. **Consumer redirection:** the guarantees stop at the kernels' queries; a deployment whose DAG edges and promotion predicates still read the layer-I store has layer-I guarantees only. The reference server now routes promotion and per-line state through `admissible()`, with unsealable lines labelled layer-I and their reason stated; consumers outside this repository remain an obligation. The server's admission and calibration state is process-lifetime only — the kernels rebuild from their journals (`from_events`, `re-guarded`), but the reference server persists none of them, so a restart empties the escape ledger and everything C promises about forgetting holds only within one process. **Record completeness:** replay proves the events present form a history the live machine would have accepted, not which history actually ran. A coherent rewrite of root inputs and deletion — of a journal's tail, or of any coherent group of events that no surviving event recomputes against — are indistinguishable from another honest history to replay alone; deletion can raise standing: a dropped escape un-impeaches its line, a dropped refusal un-taints every seal that relied on it. A calibration stamp appended for the most recent unmediated seal is accepted for the same reason: nothing earlier contradicts it. v0.5's authenticated publication path is the external anchor: immutable authority journals, three registry-current monotone heads, and a signed composed receipt close these deletions only when callers issue and verify that receipt (`tests/test_authenticated_receipt.py` pins positive, stale, truncation, cross-wiring, atomicity, and wrong-key cases). Replay without that path retains the residue; signer/key and registry compromise or rollback remain assumptions. **Liveness:** nothing here makes any gate reachable. The complete lists are the three "What is not proved" sections, each written before its kernel and extended by its reviews.

12. References

Anderson, J. P. Computer security technology planning study. ESD-TR-73-51, 1972.

Beller, M. et al. What it would take to use mutation testing in industry — a study at Facebook. ICSE-SEIP, 2021.

Bloomfield, R., and Rushby, J. Defeaters and eliminative argumentation in Assurance 2.0. arXiv:2405.15800, 2024.

Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE S&P, 1987.

Daubert v. Merrell Dow Pharmaceuticals, 509 U.S. 579, 1993.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. Hints on test data selection. IEEE Computer, 1978.

Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature, 2024.

Freivalds, R. Probabilistic machines can use less running time. IFIP, 1977.

Gibbs, I., and Candès, E. Adaptive conformal inference under distribution shift. NeurIPS, 2021.

Goldreich, O., Goldwasser, S., and Ron, D. Property testing and its connection to learning and approximation. JACM, 1998.

Goodenough, J., Weinstock, C., and Klein, A. Eliminative argumentation: a basis for arguing confidence in system properties. SEI, 2015.

Jia, Y., and Harman, M. An analysis and survey of the development of mutation testing. IEEE TSE, 2011.

Just, R. et al. Are mutants a valid substitute for real faults in software testing? FSE, 2014.

Knight, J. C., and Leveson, N. G. An experimental evaluation of the assumption of independence in multiversion programming. IEEE TSE, 1986.

Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty. ICLR, 2023.

Necula, G. C., and Lee, P. Safe kernel extensions without run-time checking. OSDI, 1996.

Popper, K. The Logic of Scientific Discovery. 1959.

Saltzer, J. H., and Schroeder, M. D. The protection of information in computer systems. Proceedings of the IEEE, 1975.

Tufano, M. et al. Learning how to mutate source code from bug-fixes. arXiv:1812.10772, 2018.

Full per-layer bibliographies: ../DRAFT.md §11, ../RGA/DRAFT.md §12.

Part II — Scrutiny and standing

Layer I verifies the die, not the roll. Layer R moves the gate to the claim: an artifact seals only when pinned, deterministic, replayable refuters were given a fair chance to kill it and failed, with measured power carried on the seal. Layer C keeps the seal honest afterwards: a defect found later, demonstrated as a counterfactual trial of the very instrument the seal trusted, impeaches it — and no successor policy can quietly forget it.

Refutation-gated admission

Integrity for agents that do not repeat.

Roque Briceño

Version 0.8.0. 30 August 2026. Licensed under CC BY 4.0.

Abstract

Fail-closed class dispatch proves that a work item's stage ran on the specialist and model it was bound to. With a deterministic function, knowing which function ran says what came out. With a language model it says almost nothing: the same specialist, prompt and context produce different artifacts, and the dispatch passes all of them identically. FCD verifies the die, not the roll. Its own proofs list the quality and provenance of a passed body as unproved. What this paper proves is integrity of the refutation record, not truth of any claim.

Refutation-gated admission moves the gate from the binding to the claim. A class fixes, before any artifact exists, the formal claims an artifact must satisfy and the deterministic refuters that will attack them. A refuter is versioned, replayable, declared as authored by neither the generator nor the defect model's author (author identities are strings the kernel compares, B11), and carries a power as a labelled record, never a bare figure: a kill-rate the kernel counts from a ledger of seeded defects against a named defect model, or $1-(1-\epsilon)^N$ from a declared (ϵ, N) the seal carries. The same bind is run k times; a sample is an FCD-passed stage. Every refuter is tried on every sample. A trial is refuted, survived, or inconclusive; only survived counts. Admission requires every cell survived, every refuter replayed once with the same outcome, concordance with the designated sample at or above a declared threshold, and power at or above a declared floor. The seal carries the artifact hash, the sampling configuration, the power per claim with its defect model or its (ϵ, N) , the concordance as (agreeing, k), and the class's declared list of what was not attacked. Nothing raises a sealed power afterwards.

This is mutation analysis, hermetic replay and semantic entropy applied as guards on an FCD line, not a new integrity model. Clark and Wilson already have the integrity verification procedure and the certification rules; FCD modelled the enforcement half and left the IVP out. RGA models the IVP and its certification record, and says which half is enforced and which is certified out of band. The theorems are about the refutation record — that this refuter version, at this measured power, ran on these bytes under this seed and fuel, and a third party can re-derive the verdict. They are not about the truth of the claim. RGA extends FCD by composition: it reads FCD state, writes none of it, and inherits I1, I2, I3, I5 and I8 by citation; the check stage's dual control (I6) is inherited unchanged and no R-proof cites it.

1. Problem

An agent that does not repeat cannot be certified by naming it. Two runs of one bind differ; a gate that passes on identity passes both the right artifact and the wrong one, and publishes the same green. Test suites distinguish them, but a green from an empty suite and a green from a suite that kills

ninety-five per cent of seeded defects look identical at the merge. Reviewers distinguish them, but a reviewer is another agent that does not repeat. Self-consistency votes among samples, and a vote is a tiebreak that picks a winner rather than a precondition that refuses.

The required property: nothing is admitted except through a declared refutation that was given a fair chance to kill the claim and failed; the seal records how strong that chance was; and the generator's own instability on the claim is a fault, not a nuisance.

Three inversions, each with prior art named in §10:

1. The generator is untrusted and stochastic; the refuter is deterministic and replayable. Integrity attaches to the refutation record, the only part that can be re-executed to the same answer.
2. Acceptance carries measured power against a declared alternative. A survival at 0.2 and a survival at 0.98 cannot share a seal; neither can a survival against a small adversarial defect model and one against a large redundant one, because the seal names the model.
3. Variance is a fault. The same bind is sampled k times; disagreement on the claim — not the prose — closes the gate.

PREMISE.md records the attack on those inversions before anything was built. The second survives with *measured* relativised; the fifth proposed theorem, that determinism is checkable, was restated as an assumption plus a falsifier.

2. Objects

A **line** is one FCD work item whose first k stages are write stages. Its **bind key** is $(c, \text{body}, \text{FCD policy version}, a, \text{norm}(\phi(a)), \sigma)$ with σ the sampling configuration. A **sample** is stage $i < k$ after FCD Pass with the line's bind key; its artifact is hashed by the kernel over the bytes it is handed, and a nonce is drawn after the hash. Sample 0 is **designated**: it is stage 0, fixed before any sample exists.

A **claim** is a formal statement over a fixed artifact format, named by a spec hash. A class pins its claims and, per claim, the refuters that attack it and the defect model they are measured against. The generator does not author the claim it must meet.

A **refuter** $r = (\text{id}, \text{version}, \text{author}, \text{mode})$ is deterministic under B2 and replayable. Mode *ledger*: measured against a defect model. Mode *bounded*: a seeded sampler with a declared (ϵ, N) .

A **defect model** $D = (\text{hash}, \text{author})$ names a finite set of seeded defects, each with a killing witness so that none is equivalent to the original (B5).

A **power record** $P[(r, D)]$ is write-once. Ledger: the kernel counts kills over a ledger of per-defect verdicts; inconclusive entries count in the denominator and not the numerator; power is $\text{kills}/|D|$, exact on D . Bounded: power is $1 - (1-\epsilon)^N$, the standard bound for any defect of failure mass at least ϵ under the declared distribution. No interval is carried: a binomial interval assumes i.i.d. draws from the population the seal speaks to, and a mutant set is not that.

A **trial** is one refuter against one claim on one sample, with a seed the kernel derives from the sample's nonce and artifact hash, and a verdict in $\{\text{refuted}, \text{survived}, \text{inconclusive}\}$ plus a witness hash — the canonical form of the claim-relevant observable the refuter computed.

Concordance for a claim is agreeing/k where agreeing counts samples whose witness vector on that claim equals sample 0's. Compared against the designated sample, never a plurality.

A **seal** is the record written to S_R (§5).

3. Process

1. Declare refuters. Measure each against the class's defect model from a ledger, or bound it from (ϵ , N). Records are write-once.
2. Open an FCD work item; open the line on it before any sample stage is attempted. The class's claims and refuters are pinned. A refuter declared after this point, authored by the generator, or refused, cannot be pinned.
3. For $i < k$: run the FCD write stage (Admit, Bind, Observe, Pass — I1 holds of it); hand the kernel the artifact bytes and the package categories; the kernel hashes, draws the nonce, and registers the sample. A package that carried refuter material is refused.
4. For every (claim, refuter, sample): obtain the seed from the kernel; run the refuter under B2; report the verdict and witness. Refuted or inconclusive closes the line, published.
5. Replay at least one trial per refuter with identical outcome. A divergence refuses the refuter for every line.
6. Run the FCD check stage, if the class has one (I6 excludes the generator). FCD Accept writes S.
7. Seal. Requires every cell survived, every refuter replayed and measured, the defect model independent of generator and refuter, $id \in S$, concordance $\geq \theta$ on every claim, and $\min_j \text{composite}_j \geq p_{\min}$. Below θ or $p_{\min}$ the line closes with V2 or V5; it does not choose a better sample or a better number.

4. What holds

Machine: rga/core.py. Proofs: PROOFS.md. Checks: tests/test_rga_invariants.py, tests/test_rga_mutation.py, tests/test_rga_citations.py.

Under B0–B11, R1–R13 are inductive on the combined machine. An independent five-lens review (round 1) broke three of the first proofs — a defect-model author free per record, a caller-supplied journal position, an unreplayable discord close — and the repairs are cited in PROOFS.md. R1: no admission without survival in every cell. R2: power is carried from a write-once record the kernel computed, never inferred. R3: the refuter was fixed before generation, not authored by the generator, and measured against a defect model authored by neither. R4: concordance with the designated sample is a precondition. R5: a replay divergence refuses the refuter monotonically. R6: every refuter was replayed at least once. R7: every sample satisfies I1, I2, I3 — by citation. R8: $S_R \subseteq S$, hence I5, I8. R9, R10: trials are bound to kernel-hashed bytes through a seed that cannot exist before the artifact. R11: RGA writes no FCD field; the FCD proofs hold on the combined machine because its FCD projection is the FCD machine. R12, R13: frozen line, bounded samples and trials, write-once power.

Every guard the fault table names is a named method. tests/test_rga_mutation.py replaces each with a no-op — one at a time — and shows the forbidden state becomes reachable; a guard method no deletion turns red fails the suite as dead code. Transition-shape preconditions (wrong pc, unknown ids, malformed verdicts, an index outside the sample range) are inline raises witnessed by the invariant

tests. tests/test_rga_citations.py opens every file:line:symbol in the proofs and fails if the line no longer contains the symbol — a move-detector; the proofs' sentences, not that test, claim enforcement.

5. The seal

S_R[id] carries: the designated sample's artifact hash; k , θ , $p_{\min}$, the sampling configuration, the RGA policy version; the FCD identity (class, body, specialist, executed model, FCD policy version); per claim, each refuter as (id, version, mode, power, D, kills, |D|, ϵ , N), the composite with its label — *single*, *union* over a shared D, or *max* — and (agreeing, k); $\min_j$ composite $_j$; the residual — the class's declared not-attacked intents, whose *check_stage* disposition is refused unless an FCD check stage Passed; and the journal position.

A seal is a record. A later refusal of a refuter does not rewrite it; tainted is a pure query. A deployment's DAG edge and promotion predicate must read S_R with a floor; one that leaves them reading S has FCD's guarantee at the edge, not this one.

6. Faults

Each is a forbidden transition, in its own namespace (V) so fault codes never collide with theorem numbers (R). V1–V5 are observed and publish a close; V6–V15 are guard refusals that write nothing.

V1 — Seal after a trial on the line returned refuted

V2 — Seal with agreeing $_j/k < \theta$ on any claim

V3 — Seal counting an inconclusive trial as survival

V4 — Agreement recorded for a replay that diverged; Measure, Bound or Replay on a refused refuter

V5 — Seal with $\min_j$ composite $_j < p_{\min}$

V6 — Sample from a stage not Passed or bound to a different specialist than the line

V7 — Sample whose generator package contained a refuter category

V8 — Open after a sample stage was attempted; a sample registered after a later sample stage was attempted; Trial against an unpinned claim or refuter

V9 — Trial whose seed is not the kernel-derived seed

V10 — Seal without k samples or with a cell lacking a surviving trial

V11 — Seal using a refuter with no identical replay on this line

V12 — Seal using a refuter with no power record

V13 — Measure or Bound on an existing key; a sample beyond k ; a second trial in one cell; a ledger whose id-set, or a record whose author, differs from the first record

V14 — A pinned refuter authored by the generator; a defect model authored by its recording refuter, or whose fixed author is the generator

V15 — Seal of a line not in S; a residual claiming a review that did not run; any write to S_R but Seal

7. The boundary

A claim is RGA-attackable iff its falsity has a finite witness that a total, fuel-bounded, hermetic, replay-stable checker accepts. That line is intrinsic to a formal claim. What is relative is the gap between an intent and the formal claims that stand in for it: "this is the correct fix" has the attackable shadow "the issue no longer reproduces, the held-out tests pass, the diff touches only the named files", and the residual "that the held-out tests test the issue". Every interesting intent has only a shadow. A seal that attacked the shadow at power 1 must not be read as a verdict on the intent, so the seal lists what it did not attack and who, if anyone, looked at it.

A language model can propose counterexamples. A checker-verified witness is a sound, replayable kill whoever proposed it. A survival under proposed attack carries power only if the proposer is pinned and its catch rate measured on a held-out seeded set, the rule for every stochastic witness source. The model is never the refuter.

Prose is not excluded by medium. Perturbation families for faithfulness exist, and fixed-weight consistency checkers that are bitwise replayable when pinned report agreement with human-labelled errors in the 0.7–0.85 range (balanced accuracy on the published benchmarks — not a kill-rate, which a deployment would measure against its own D); omission and adequacy defects have near-zero measured power today, and a seal over them says so.

Determinism is not checkable; it is assumed by construction (B2: pinned toolchain, no clock or network, runtime-counted fuel — refusing a refuter written outside that subset is the harness's obligation, not a kernel check) and falsified by replay. Exhaustion is a third outcome that never counts as survival. The artifact runs inside the refuter's sandbox; the refuter's parser is a trusted computing base against hostile input.

8. Measurement

Rates on a named cut, as in FCD §7, never theorems: refutation rate per class and refuter version; discord rate at the declared (k, σ) ; *miss observed* per refuter — where the witness is a function of the claim value, a discordant pair of survivals is a detected miss by that refuter, and a V2 close journals exactly the refuters whose witnesses differed; below $\theta = 1$ a discordant survival can seal, visibly, through (agreeing, k); replay divergence per refuter version, the refuter analogue of misbind; escape replay for a successor refuter version against defects later found under its predecessor, informative only across versions and doubly selected. Schema: `../metrics/SCHEMA.md`. No numbers here.

9. Calibration — the escape ledger

B6 leaves the defect model's resemblance to the generator's real defects an assumption, and nothing above forces a found miss to have any consequence. The calibration layer (`rga/calibration.py`, theorems C1–C7, faults E1–E9, INVARIANTS.md §8) closes the loop that can be closed and says plainly which loop cannot. "Calibration" is this repository's metrological sense: the reference corpus is ratcheted against verified real defects. No statistical sense is claimed.

An **escape** is a counterfactual trial: the sealed claim's *pinned* refuter, run at a finder-chosen nonce against the sealed bytes — the finder authors nothing but the nonce, the kernel hashes the bytes and derives the seed itself, and the verdict refuted plus one identical replay establishes it. That channel adds no assumption the seal did not already carry. Any other checker is tier B and has effect only

through a journaled, attributed adjudication — the per-escape claim-fidelity judgment made visible instead of pretended away. A **charge** is the wrong-verdict cell (line, claim, refuter version), one per cell however many witnesses prove the miss. **Impeachment** is a pure query entailed by a valid escape — the seal is immutable, revocation is consulted at use, and it cannot be declined: unlike a certificate authority's revocation list, the entry is entailed by a replayable proof, and the miss propagates mechanically to every checker that vouched. The **ratchet** refuses any successor policy whose defect models forget a valid escape without a named, journaled exclusion, and the install event — which now exists — carries the coverage primaries and the dropped-id diff. **Demotion** is a query, never a cascade: charges past a declared budget block new pins, and gate Seal exactly where the class declared it, closing with a published fault that carries the instrument's primaries. Every seal issued through the authority is stamped, in the same step, with each pinned refuter's track record — primaries, never a rate; a zero is absence of filed evidence and the record says so.

What this proves is one sentence shorter than it sounds: **forgetting is loud**. Every found miss *that stays in the record* has a mandatory, attributed, replayable consequence — and the qualification is load-bearing: replay re-derives every transition and compares it to the journal field by field, so alteration, forgery and duplication are refused, but a deletion that leaves a coherent shorter history is not detectable from the journal alone, and that is the direction which raises standing (§11). What was never found, the ledger cannot see — the corpus is a selected, steerable subsample of real defects, silence is unreadable except through journaled audits whose reference is the corpus itself, and every aggregate an adversary can steer by filing only true things. The C-theorems are about the ledger of found misses, never the distribution of real ones, and reading seven theorems under a calibration heading as "the coupling gap is closed" is exactly the laundering the seal's power field was built to prevent. Premise round and its one authored adjudication: `eval/reviews/rga-calibration-premise-SYNTHESIS.md`.

10. Related work

Clark and Wilson (1987): certification rules and the integrity verification procedure. FCD's specialist is a TP; RGA's refuter is an IVP; its power is a certification made by a party with I/O, and the kernel enforces that the certified IVP of the pinned version ran and that its record is on the seal.

Mutation analysis (DeMillo, Lipton & Sayward 1978; Jia & Harman 2011) owns the ledger number. Just et al. (2014) measured its coupling to real faults; Papadakis et al. (2016) its threats to validity; Kurtz et al. (2014) dominator mutants. Property testing (Goldreich, Goldwasser & Ron 1998), Freivalds (1977) and Miller–Rabin own the bounded number. Proof-carrying code (Necula & Lee 1996) is the power-1 corner and the seal still carries the checker's identity there.

Semantic entropy (Kuhn, Gal & Farquhar 2023; Farquhar et al. 2024) is the graded statistic of which concordance is the zero-entropy special case — exact agreement with a designated sample, as a gate rather than a signal; self-consistency (Wang et al. 2022) is the vote RGA refuses to take; AlphaCode's output clustering, CodeT and universal self-consistency are agreement on claim consequences without an oracle. N-version programming (Avizienis 1985) and Knight & Leveson (1986) are why power does not compose by independence. Flaky-test quarantine (Luo et al. 2014) and reproducible builds are the practice behind R5. In-toto (Torres-Arias et al. 2019) is the provenance the trial record resembles; what it lacks is concordance under a claim normaliser and a typed store a DAG edge reads. Statistical model checking (Younes & Simmons 2002) is the formal home for a stamped survival rate with error bounds. Conformal factuality (Mohri & Hashimoto 2024) is concordance plus calibration with no refuter. Metamorphic testing (Chen, Cheung & Yiu 1998) and FactCC (Kryscinski et al. 2020) supply claims and defect models where no oracle exists. Automated-repair overfitting (Smith et al. 2015) is why the generator's package excludes the refuter and why the seed is drawn after the artifact.

The calibration layer's loop has its own ancestry: eliminative argumentation and Assurance 2.0 defeater discipline (Goodenough, Weinstock & Klein 2015; Bloomfield & Rushby 2024) — confirmed, refuted, or accepted with recorded rationale, never silently dropped — mechanized, with the non-covering successor refused by a kernel rather than flagged for an assessor. The corpus is escape-to-test regression practice and mined-from-real-faults mutation (Tufano et al. 2018; Beller et al. 2021; Defects4J) applied as a journal-derived ratchet. Demotion is flaky-test quarantine and the SRE error budget with the sign flipped — those practices lack a verified false-negative channel, which is what the escape object creates — and browser root-program distrust of certificate authorities is the institutional precedent. Impeachment is the CRL/OCSP/Sigstore shape minus discretion: the two properties for which no prior formalization was found are non-discretion (revocation entailed by a replayable proof) and charge totality (mechanical consequence for every vouching checker). Adaptive conformal inference (Gibbs & Candès 2021) is the shape a statistical calibration theorem has, and why none is claimed here.

The 2026 neighbours circle the same gap without the enforcement object. Refute-or-Promote (arXiv:2604.19049) puts adversarial kill mandates at promotion gates, but its refuters are LLM agents — stochastic, unreplayable, with no carried power. ConsistencyGate (arXiv:2607.22962) is admission control by self-consistency, but as a thresholded soft score — a vote, where concordance here is a fault against a designated sample. Attestation-aware promotion gates (FSE 2026, arXiv:2603.28988) bind provenance of training and release claims — the B1/B2 boundary, complementary, carrying no power. And a contract-grade verifier for generated GPU kernels (arXiv:2608.12700) is the premise measured in one domain: the field's loose test accepted 1,487 kernels the rigorous verifier rejected, against 14 the other way — two greens that today print identically, which is what the seal's carried power exists to separate. Mutation-kill certification is reported absent from LLM generation benchmarks (arXiv:2608.12635). What none of these carry is the seal: kernel-counted power against a named defect model, concordance as a precondition, refuter refusal on replay divergence, and a proved fail-closed table underneath.

11. Limits

Record completeness. Replay proves the events present form a history the live machine would have accepted, not that it is the history that actually ran. A coherent rewrite of root transition inputs is another valid history; deletion — the journal's tail, or any coherent group no surviving event recomputes against — is likewise undetectable from replay alone and can raise standing. v0.5 adds an authenticated publication path: each authority owns an immutable tuple journal, each stack has a stable registry namespace, successor heads prove prefix extension through a cumulative event chain, three heads advance atomically, and a signed composed receipt derives its predicates from the exact current I/R/C state. After a trusted first anchor or continuous publication, verification rejects truncation, coherent deletion, longer forks, state-state receipts, namespace collisions, and cross-wired stacks. *Stale* here means not registry-current for the current journal state; *issued_at* is signed metadata, not expiry. The first receipt cannot distinguish coherent histories that predate its trusted bootstrap. The closure is conditional on using that path, HMAC-SHA256 unforgeability, SHA-256 collision and second-preimage resistance, honest key custody, and durable registry state; registry plus signer/key compromise or rollback and mutation outside the atomic single-writer authority model remain assumptions. **Mediation.** A seal produced by calling `Admission.seal` directly is layer R only: `mediated(id)` — one stamp bound to that seal — is a conjunct of admissible, so such a line reads IR, not IRC. **Budgets.** `e_max` and `demotion_gate` are explicit per class (E9); an unconfigured class is refused, never treated as unlimited, and the budgets are journaled so a rebuild runs under the ones the record names. No dataset. No quality theorem. No liveness. R1–R13 hold on the combined Python machine under B0–B11; B1 (harness honesty) and B2 (hermetic runtime) are physical-boundary assumptions of A10's standing, and author identities are declared strings (B11), not a closed namespace as FCD's specialist ids are. Power is relative to a declared alternative the seal names; its relevance to the generator's actual

defects is the coupling-effect hypothesis, stated, not proved. Claim fidelity to intent is not proved. Body provenance is not closed; it is moved to the seal's hash and still rests on l1 holding of the report. A refuter the generator already knows can be special-cased; the seed and the package exclusion narrow that and do not close it. The FCD check stage is inherited as a dispatch-integrity gate and carries no content authority.

"Subsumes" is by composition. RGA extends FCD, writes no FCD field, and needs FCD underneath as a separate, cited layer.

Site names do not belong in this voice.

12. References

- Avizienis, A. The N-version approach to fault-tolerant software. IEEE TSE, 1985.
- Chen, T. Y., Cheung, S. C., and Yiu, S. M. Metamorphic testing. HKUST-CS98-01, 1998.
- Chen, B. et al. CodeT: Code generation with generated tests. 2022.
- Chen, X. et al. Universal self-consistency for large language model generation. arXiv:2311.17311, 2023.
- Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE S&P, 1987.
- DeMillo, R. A., Lipton, R. J., and Sayward, F. G. Hints on test data selection. IEEE Computer, 1978.
- Farquhar, S. et al. Detecting hallucinations in large language models using semantic entropy. Nature, 2024.
- Freivalds, R. Probabilistic machines can use less running time. IFIP, 1977.
- Bloomfield, R., and Rushby, J. Defeaters and eliminative argumentation in Assurance 2.0. arXiv:2405.15800, 2024.
- Goodenough, J., Weinstock, C., and Klein, A. Eliminative argumentation: a basis for arguing confidence in system properties. SEI, 2015.
- Gibbs, I., and Candès, E. Adaptive conformal inference under distribution shift. NeurIPS, 2021.
- Tufano, M. et al. Learning how to mutate source code from bug-fixes. arXiv:1812.10772, 2018.
- Beller, M. et al. What it would take to use mutation testing in industry — a study at Facebook. ICSE-SEIP, 2021.
- Goldreich, O., Goldwasser, S., and Ron, D. Property testing and its connection to learning and approximation. JACM, 1998.
- Jia, Y., and Harman, M. An analysis and survey of the development of mutation testing. IEEE TSE, 2011.
- Just, R. et al. Are mutants a valid substitute for real faults in software testing? FSE, 2014.

Knight, J. C., and Leveson, N. G. An experimental evaluation of the assumption of independence in multiversion programming. IEEE TSE, 1986.

Kryscinski, W. et al. Evaluating the factual consistency of abstractive text summarization. EMNLP, 2020.

Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty. ICLR, 2023.

Kurtz, B. et al. Mutant subsumption graphs. ICSTW, 2014.

Li, Y. et al. Competition-level code generation with AlphaCode. Science, 2022.

Luo, Q. et al. An empirical analysis of flaky tests. FSE, 2014.

Mohri, C., and Hashimoto, T. Language models with conformal factuality guarantees. ICML, 2024.

Necula, G. C., and Lee, P. Safe kernel extensions without run-time checking. OSDI, 1996.

Papadakis, M. et al. Threats to the validity of mutation-based test assessment. ISSTA, 2016.

Smith, E. K. et al. Is the cure worse than the disease? Overfitting in automated program repair. FSE, 2015.

Torres-Arias, S. et al. in-toto: Providing farm-to-table guarantees for bits and bytes. USENIX Security, 2019.

Wang, X. et al. Self-consistency improves chain of thought reasoning in language models. 2022.

Younes, H. L. S., and Simmons, R. G. Probabilistic verification of discrete event systems using acceptance sampling. CAV, 2002.

Refute-or-Promote: an adversarial stage-gated multi-agent review methodology for high-precision LLM-assisted defect discovery. arXiv:2604.19049, 2026.

ConsistencyGate: preventing memory contamination in LLM agents via self-consistency admission control. arXiv:2607.22962, 2026.

Attesting LLM pipelines: enforcing verifiable training and release claims. FSE, arXiv:2603.28988, 2026.

A contract-grade verifier for LLM-generated GPU kernels. arXiv:2608.12700, 2026.

GateTruth: auditing the rigor of RTL design benchmarks via mutation testing. arXiv:2608.12635, 2026.

Appendix A — Premise, attacked before the kernel

Premise

Roque Briceño. The attack on the RGA brief, written before the kernel. Companion to INVARIANTS.md, PROOFS.md and DRAFT.md in this directory.

The brief asked four questions and said: if the premise does not survive, the honest paper says why. This file is that report. It records what was attacked, what fell, what survived, and what the survivors force on the design.

0. Method

Five adversarial readings of the brief, one per question plus one whose only job was to argue that RGA is FCD with ceremony. Thirty-eight findings. Each finding was handed to an independent reviewer instructed to refute it, defaulting to *refuted* unless the argument survived a genuine attempt. All readers had paper/INVARIANTS.md, paper/PROOFS.md, paper/DRAFT.md, fcd/core.py and eval/reviews/r4-formal.md; several ran constructions on the unmodified machine.

Question	Findings	Refuted	Survive, narrowed	Survive as stated
Power	10	5	5	0
Cheaper formulation	6	2	4	0
Boundary	7	2	5	0
Subsumption	7	1	6	0
Collapse	8	4	4	0

Six findings were filed as *kills-premise*. None survived at that severity. Every one was either refuted or reduced to a design constraint. Nothing survived unchanged either: the brief as written over-claims in four places, listed in §6.

Positions I held before the attack are in the session scratchpad. Two were wrong and are marked below.

1. Is power measurable, or declared?

Verdict: measurable, relative to a declared alternative. Never a measurement of the generator's actual defects. The kernel can make the bare number unrepresentable; it cannot make the alternative honest.

The brief defines power as the probability the refuter would have caught the defect had it been there. That is Neyman–Pearson power: a function of the alternative hypothesis, not a number. The first attack said a scalar exists only after someone declares a prior over defects, so power collapses to a declaration. The refutation: a composite alternative is scalarised by *infimum over a declared class*, not by expectation over a prior. Every clinical power calculation is a scalar at a declared effect size. Nobody concludes power does not exist.

Three modes survive, and the kernel must carry which one it is:

1. **Exhaustive.** The refuter enumerates a finite domain. Power against any defect in that domain is 1 by construction. No sample size. This is the proof-carrying-code corner (Necula & Lee 1996): the seal still carries power 1 and the checker's identity, because a seal with no power field is the laundering channel the brief forbids.

2. **Bounded.** A seeded refuter draws N inputs from its declared distribution Q . Against any defect whose failure mass under Q is at least ϵ , the kill probability is at least $1 - (1 - \epsilon)^N$ (Goldreich–Goldwasser–Ron 1998; Freivalds 1977; Miller–Rabin). The seal carries (ϵ, N) ; the seed rule is the kernel's (R10); Q is whatever the pinned refuter version samples from and is not a kernel field. ϵ is itself a declaration and the seal shows it (§9). The number is refuter-intrinsic and invariant to the generator by construction. The seed must be drawn *after* the artifact hash is frozen, or the generator can tailor to it; §5.

3. **Calibrated.** A refuter is run against a declared, content-addressed defect model D — a finite set of seeded defects, each with a killing witness so the denominator is exact. Power is $\text{kills}/|D|$, exact on D . The seal carries $(D, \text{kills}, |D|, \text{refuter version})$. Its transfer to the generator's real defects is the coupling-effect hypothesis (DeMillo, Lipton & Sayward 1978), which Just et al. (2014) found holds for roughly three quarters of real faults under common operators. That transfer is an empirical claim and goes under *What is not proved*, beside FCD's A10.

What fell:

- *The Wilson lower bound as the carried number.* I pre-registered this: carry kills, $|D|$ and a 95% lower bound, threshold on the bound. Refuted. A binomial interval assumes i.i.d. draws from the population the seal speaks to; a mutant set is not that. A refuter killing 4900 of 5000 redundant operator mutants carries a lower bound of 0.976; a refuter killing 49 of 50 adversarial defects carries 0.895. The interval ranks the first above the second while the first has power near zero against the class it misses. A threshold on the interval does not remove laundering; it re-laundered under a statistical badge. The coordinate that separates two seals at the same kill-rate is D , not n . So: $|D|$ is derived from the content-addressed D , not carried as a free variable; the class policy names which D it certifies; the number is exact on D and the seal says so. `rga/core.py` carries kills and $|D|$ and no interval.

- *Power composes by independence.* Refuted, and the repo's own ethos agrees (`fcd/core.py:254` recomputes `on_bind` rather than trusting a submitted boolean). Two refuters at 0.8 give 0.96 only if their blind spots are disjoint; Knight & Leveson (1986) measured correlated failure far above the independence prediction. Composite power is exact only as the union of kill vectors over one shared D . From scalars alone the sound bound is $\max(\pi)$. The kernel computes the union when D is shared and otherwise carries `max`, labelled.

- *A trivially weak refuter passes.* A refuter that always returns *survived* kills nothing: power $0/|D|$, carried. A refuter weak in a targeted way — strong on D , blind to the defect class the generator actually emits — is not caught, and that is the Goodhart residue under *not proved*. Its oracle must derive from the Claim, never from the artifact (a golden-copy refuter needs information flow from the artifact into the check, which pinning the refuter before generation forbids), and the defect model must be authored by a party that is neither the generator nor the refuter's author. Three-way separation of duty.

- *Prose has no D .* Refuted by the factual-consistency literature: FactCC's perturbation family (entity, number, pronoun swap, negation), FRANK's error typology, and fixed-weight checkers (SummaC, AlignScore) that are bitwise replayable when pinned, with kill rates against human-labelled faithfulness errors in the 0.7–0.85 range. RGA scopes admission by *defect family with measured power*, not by medium. Omission and adequacy defects have near-zero measured power today and the seal shows that.

What the kernel can do: refuse a bare float; accept only a ledger of per-defect verdicts and count the kills itself; bind the record to a refuter version and a D hash; make the record write-once per (refuter version, D) so nothing raises it after the fact; compute the bounded-mode number from declared (ϵ , N) rather than accepting it, and carry (ϵ , N) in the seal so the declaration is visible. Two representations cover the three modes — the exhaustive mode is a ledger enumerating the whole domain — and the seal labels which. What the kernel cannot do: verify the ledger was produced by running anything. That is the same boundary as FCD's A10, and it is named as such.

Is RGA then FCD with ceremony? It is, exactly when power enters the seal as a scalar somebody wrote down. It is not when power enters as a replayable certification record a third party can falsify — the same move FCD makes with the package hash. The brief's phrase "measured doubt" must be read as *measured power against a declared alternative*, and the paper says so in its first paragraph.

2. Is there a cheaper formulation?

Verdict: no candidate gives the same guarantee. Three are subcomponents, one is the power-1 corner, one is a different object. The name stays, with one sentence of DRAFT §9 discipline added.

Quorum of generators / N-version (Avizienis 1985) — *Gives*: Cross-model agreement; each sample inherits 11/13 · *Lacks*: No refuter, no power, no replay; Knight–Leveson correlation unmeasured · *Relation*: Subcomponent: concordance without refutation. Expressible as FCD stages with author exclusion on write stages.

Acceptance criteria fixed at Open (Meyer 1992; Nosek 2018) — *Gives*: Claim not authored by the generator · *Lacks*: No power: Hypothesis at 100 vs 10 000 examples is the same green. No concordance. No determinism audit. · *Relation*: The skeleton RGA measures the strength of. Does not close the Goodhart hole; relocates it from "generator writes a weak claim after" to "generator writes an artifact after seeing a fixed public refuter".

Plain CI + hermetic builds + in-toto/SLSA provenance — *Gives*: T1, T3 provenance, T5 re-run, trial record · *Lacks*: Concordance under a claim normaliser (in-toto thresholds are byte identity); a typed store read by the DAG gate; verdict replay from the journal · *Relation*: Residue is three guards, not one. Whether that is a paper or a section is a judgement the result must earn.

Proof-carrying code (Necula & Lee 1996) — *Gives*: Power 1 on the checked predicate · *Lacks*: Nothing, where the predicate is human-authored. Where it is autoformalised by the generator, kernel power on the proof is 1 and power on the claim is not. · *Relation*: The exhaustive corner of mode 1. Cited.

Self-consistency / semantic entropy (Wang 2022; Kuhn, Gal & Farquhar 2023) — *Gives*: Disagreement detection, AUROC $\approx$ 0.7–0.8 as a confabulation signal · *Lacks*: No refuter; a vote, not a fault · *Relation*: Inversion 3 is semantic entropy applied as a fail-closed gate with a threshold. The paper says so.

Conformal / selective prediction (Vovk; Mohri & Hashimoto 2024) — *Gives*: A marginal error rate with a theorem · *Lacks*: Per-item refutation; a replayable verdict on one artifact; the best scorer is k-resample frequency, i.e. concordance, plus a labelled calibration set whose labeller is a refuter · *Relation*: A class-level rate that can sit beside the seal. Not a guard on Accept.

SPRT / statistical model checking (Younes & Simmons 2002) — *Gives*: Early-stopping bound on $P(\text{survive } R)$ with declared α , β · *Lacks*: Refuter-independent concordance · *Relation*: The formal home

for a stamped survival rate with error bounds. Complementary.

Two alternative names were proposed from the mathematics: *Seeded-Refutation Admission: Observed Power, Replayable Verdicts* and *Verdict-Bound Acceptance: a refutation stage for fail-closed class dispatch*. Both name mechanisms. *Refutation-Gated* names the shape of the guarantee: nothing is admitted except through a declared refutation that was given a fair chance and failed. The algorithm keeps its name. The subtitle *Integrity for Agents That Do Not Repeat* states the problem precisely and is kept; what it must not be read as is a theorem that the claim is true. §6.

The sentence the paper owes, in the voice of DRAFT §9: this is mutation analysis, hermetic replay and semantic entropy applied as guards on an FCD line, not a new integrity model. Clark and Wilson already had the integrity verification procedure and the certification rules; FCD modelled the enforcement half and listed body quality as unproved; RGA models the IVP and its certification record, and says which half is enforced and which is certified out of band.

3. Which claims can a deterministic refuter attack?

Verdict: the boundary is intrinsic to a formal claim and sharp; what is relative is the gap between an intent and the formal claims that stand in for it. That gap is the engineering content, and the seal must carry it as an explicit field.

Criterion. A claim C over a fixed artifact format is RGA-attackable iff its falsity has a finite witness that a total, fuel-bounded, hermetic, replay-stable checker accepts. Run the brief's list through it:

"Passes tests T" / "type-checks" / "validates against schema S" — *Attackable shadow*: The claim itself; witness is the failing run · *Residual*: none

" $\forall x. P(x)$ " over an infinite domain — *Attackable shadow*: Kills (any counterexample); survivals only at bounded or calibrated power · *Residual*: absence, per Dijkstra

" $f(x) = f(\text{permute}(x))$ ", " $\text{decode}(\text{encode}(x)) = x$ " — *Attackable shadow*: The relation; witness is the violating pair · *Residual*: that the relation is the right one

"Cites only files that exist; touches only the files the contract named; every number in the summary occurs in the anchored source span" — *Attackable shadow*: The claim itself · *Residual*: none

"This summary is faithful" — *Attackable shadow*: Anchored-span checks on entities, numbers, negations; pinned NLI checker at calibrated power · *Residual*: reversed causality, omission

"This is the correct fix" — *Attackable shadow*: Issue no longer reproduces; held-out tests pass; diff scope · *Residual*: that the held-out tests test the issue

"This design is good" — *Attackable shadow*: Named constraints: latency under a fixed load script, no dependency cycle, schema backward-compatible · *Residual*: everything else

Every interesting intent has only an attackable shadow — necessary conditions, never the intent. A seal that attacked the shadow at power 1 must not read as a verdict on the intent. So the seal lists what was attacked, at what power, against which D, and carries an explicit **not-attacked** field for the residual with its disposition: reviewed by a non-author under I6, or unreviewed. FCD's check stage is where that residual is decided; it is inherited unchanged as a dispatch-integrity gate and is not asked to carry content trust.

Who authors the claim. A generator that authors the claim it must satisfy will author the weakest claim the refuter family can check, survive at power 1, and the seal will say 1. Not vacuous in the round-3 sense — refusal stays reachable — but worthless. So the claim schema and the refuter set are pinned by class at Open, by content hash, before any sample exists. The generator may instantiate a claim within the schema for discovery-shaped work; it may not select the schema or the refuters. rga/core.py v1 fixes claims at Open outright and treats within-schema instantiation as the documented extension.

What an LLM can do. Propose counterexamples. A checker-verified witness is a sound, replayable kill regardless of who proposed it. A survival under LLM-proposed attack carries power only if the proposer is pinned and its catch rate calibrated on a held-out seeded set — the same rule as any stochastic witness source (Miller–Rabin picks random bases). The LLM is never the refuter. It is a witness source with a calibration record or it is nothing.

Determinism. "Checkable" was over-claimed and was the one place the brief's theorem list had to be restated rather than narrowed. Re-running a refuter is a falsifier: a divergence is a sound refusal; an agreement is a sample. k agreeing re-runs bound the flake rate, never prove determinism. So:

- **A-det**, a physical-boundary assumption of A10's standing: a pinned runtime (toolchain hash, sealed clock, no network, runtime-counted fuel, not wall-clock — a 2M-iteration loop measured 86–91 ms across 25 idle runs, so any wall-clock deadline near the run time is a coin flip) makes a trial a function of (refuter, artifact, fuel).
- **Check 1**, static, is the harness's obligation under that assumption: the kernel never sees the refuter's text and cannot refuse one written outside the deterministic subset. The kernel's own check is the falsifier below.
- **Check 2**, dynamic: every refuter used on a line has at least one trial on that line replayed with an identical outcome; a divergence refuses the refuter, monotonically, for every line.
- **Exhaustion is a third outcome.** Fuel or harness exhaustion is neither kill nor survive. It closes the line with its own fault and never counts as survival. Lineage: A2, the watchdog's over-closing rule. Survival is a positive completed verdict, never the absence of a kill.
- The artifact runs inside the refuter's sandbox under the same denials and fuel; the refuter's parser is a trusted computing base against hostile input.

4. Does RGA subsume FCD?

Verdict: by composition, not replacement. RGA writes no FCD field. I1–I6, I8 and I9 are inherited unchanged by the same no-other-writer inspection; I7, I10, I16 and I17 are restated per sample; A1, A2, A10 and A13 gain per-sample forms, and A11's collision clause is honestly *weakened* here (B7: toward acceptance — and FCD's own I16/I17 proofs use A11 fail-open, filed). The honest verb is *extends*.

What "run the same bind k times" means on the machine. Not k Admits of one specialist on one stage (A7, I7 forbid it). Not k Observes in one Running stage (last-writer-wins, no artifact on the call event). A reviewer built both surviving constructions on the unmodified machine with the suite green:

- *k write stages of one class.* Required(c) = [(write,s0), ..., (write,s_{k-1}), (check,...)]. tried is per stage, so the same specialist is admitted on each; each stage gets its own Bind, Observe, Pass (I1 per sample), attempt, nonce and receipt (I10, I17 per sample); the item stays open until the check stage; one (P_v,K_v) pin, one policy version, one body hash; the check stage's π_{chk} excludes the generator

automatically, so the residual's reviewer is not the generator (I6). The refuter itself is not an FCD stage; its author is a declared string, and the refuter-is-not-the-author rule is a new guard of I6's shape, only as strong as the author identities are honest (B11, added in round 1).

- *k sibling items* with one body hash. Works, but pins may differ across a promotion (A13) and a losing sibling has no terminal status.

I pre-registered sibling items. The first construction is stronger and is what rga/core.py composes over: a **Sample** is a Passed write stage of the line's FCD item whose bind key matches the line's. One item, one pin, one id in both stores.

What RGA reads from FCD, and never writes: stages[i].pc, stages[i].kind, stages[i].a, stages[i].m_decl, item.cls, item.body, item.policy_version, membership in S. The Sample guard is FCD's Passed state; the Seal guard is FCD's store membership. That is the literal form of inheriting I1, I3, I5 and I8 as premises.

Where the gate sits. There is no state between the last Pass and Accept on the Python machine (decide_pass calls accept in the same step). An RGA gate "after Accept" is post-store. A reviewer first argued the seal must therefore replace Accept; the refutation showed the third placement the repo already uses — the server interposes receipt refusal and drift review in state Running $\wedge$ m_exec≠none, before decide_pass — and built a side authority with $S_R \subseteq S \times \text{Power}$ on the unchanged machine. RGA's seal is that refinement: $(id, p) \in S_R \Rightarrow id \in S \wedge$ a surviving trial at power p. S stays a set of ids; Accept is untouched; every consumer that today reads S as a boolean — the DAG edge at fcd/core.py:136-139, Promote through the injected is_accepted — must be redirected to S_R with a power floor, or inversion 2 holds at the seal and fails at composition.

What FCD lacks that RGA must add. FCD has no output object. Item.body is the input hash; AdapterReceipt binds the package the executor *received* and the model that ran; S is a set of ids; the reference server keeps the produced artifact in a plain dict and flips it to accepted after the fact. A refuter that "ran against the accepted artifact" has nothing in FCD to cite. So the kernel content-addresses the candidate at Sample and every Trial carries that hash; the Seal records the hash of the bytes the surviving trials examined; a store that serves anything else is outside the theorem. This reuses A10 rather than widening it: the hash is computed by the authority over bytes it holds, as compile_package already is.

Why I1 and I3 are premises — corrected. The brief's reason was "you cannot measure a refuter's power without knowing what ran." For a script refuter, what ran is a content hash and I1 is irrelevant to it. The real reasons: (a) power is P over the bind's output distribution conditioned on a defect, so the number is meaningful only because I1 fixes which generator that is and I3 keeps it inside the calibrated allowed set; (b) concordance is a generator fault only if the k samples are draws from one distribution, which needs I1, I3, I10, I11 and I15 together plus a **bind key** FCD does not expose: envelope_hash includes attempt id, nonce, counter and channel, so it differs per attempt by construction. RGA defines the bind key as the envelope projected onto the fields that are not attempt-local, plus the sampling configuration, and guards Sample on equality.

Strengthened assumptions. One new trust assumption: the refuter runtime is honest and hermetic (A-det). The rest are design obligations: the call event and receipt need a sample identifier (the same species as runtime_instance for F2); k samples share one envelope and hence one pin by the definition of "same bind"; refuter death is no-admission by the watchdog's rule; the determinism check is a falsifier.

One reviewer reproduced a defect in the reference server while doing this: a steer between the write and check stages makes the served-as-accepted bytes differ from the bytes the reviewer's package

hashed, with every guard green. And A11's clause "a collision fails toward refusal" is used fail-open by the I16 and I17 proofs. Both are FCD defects, out of RGA's scope, filed separately.

5. What the survivors force on the design

Each row is a constraint that was not in the brief and is now in INVARIANTS.md — as a guard, a seal field, a kernel query, or an entry under *Not theorems* naming it a deployment obligation. §9 records the round-1 corrections to these rows.

1. Power is a record, never a float: (mode, refuter version, D hash, kills, |D|) or (mode, Q, ϵ , N). Write-once per (refuter version, D). Kills counted by the kernel from a per-defect ledger.
2. Composite power per claim: union of kill vectors over a shared D, else max, labelled. Never a product.
3. Claim schema and refuter set pinned by class at Open — claims by content hash, refuters by declared (id, version) whose binding to bytes is the pinned-runtime assumption; a trial against an unpinned claim or refuter is refused, not scored.
4. Refuter author $\neq$ generator; defect-model author $\notin$ {generator, refuter author}; refuter version registered before the line opens.
5. Generator package excludes refuter source, version and results (I12 in the reverse direction); checked at Sample as a receipt field under A10.
6. Seeded refuters take a seed derived by the kernel from a nonce drawn *after* the artifact hash is recorded; the trial's seed is equality-checked. The envelope nonce predates generation and cannot serve.
7. Trial verdict $\in$ {refuted, survived, inconclusive}; only *survived* counts; *inconclusive* and *refuted* close the line, published.
8. Every refuter used on a line has ≥ 1 trial on that line replayed with identical (verdict, witness); a divergence refuses the refuter for every line, monotonically.
9. Concordance is agreement with the designated sample (stage 0, fixed before any sample exists), not plurality; measured on the trial witness vector per claim; the seal carries (agreeing, k) so $k = 1$ is visibly trivial. Canonicalisation is conservative: inequality is discord.
10. Sampling configuration is part of the bind key and the seal, so the fault is relative to a declared regime; under deterministic decoding the discord row is dead and the seal says k, σ .
11. The seal carries: artifact hash; per claim (refuter versions, D, kills, |D|, composite power and its label, agreeing/k); min power over claims; the not-attacked residual and its disposition; the FCD identity (class, body, specialist, executed model, policy version).
12. $S_R \subseteq S$ is a theorem. The kernel exposes the power-aware DAG gate (check_dependencies, which also refuses tainted seals) and the promotion predicates (is_sealed, admissible); that a deployment wires them in place of the FCD edge and is_accepted is an obligation listed under *Not theorems*, not a theorem. The reference server has since discharged it for promotion and state (tests/test_server_admissibility.py).

13. Discordance among surviving samples is a detected miss by every refuter on that claim; it is journaled against those refuters as a rate, not silently absorbed.

14. Retry after a refutation with the refutation trace visible to the generator makes the next sample adaptive. The kernel refuses only the categories excluded(c) names (B1); beyond that the flow is unlabelled, and INVARIANTS.md §7 and *Not theorems* say so. A retry that changes the claim is a new claim.

6. Where the brief over-claimed

1. "An accepted artifact is stamped with the power it survived." It is stamped with the calibrated power of the refuter version whose attested trial it survived, against a named alternative. The number is about the refuter, not the artifact; "negative on a test with sensitivity 0.98" is how a negative result is made informative about one patient, and no more than that.

2. "Determinism of the refuter is checkable." Falsifiable. A refuter that fails is refused; a refuter that passes is not proved deterministic.

3. "Integrity attaches to the test." Integrity attaches to the refutation *record*: that this refuter version, at this calibrated power, ran on these bytes under this fuel, and a third party can re-derive the verdict. It does not attach to the claim's truth, and the title must not be read that way, exactly as round 4 said "no leftover hop" must not be read as a theorem about fallback architectures.

4. "RGA subsumes FCD." Extends. Composes over. Writes no FCD field. Needs FCD underneath as a separate, cited layer.

The two inversions that survive cleanly are the first and third — the third as the zero-entropy special case of semantic entropy, exact agreement with a designated sample, applied as a gate. The second survives with the word *measured* relativised. "Fail-closed under measured doubt" is kept because every word in it is now defined: fail-closed on refutation, inconclusion, discord, nondeterminism and underpower; doubt measured against a declared alternative the seal names.

7. What goes under "What is not proved"

Written now, before the proofs, as the brief asked.

- Relevance of D to the generator's actual defects (coupling effect).
- Refuter over-fit to D; Goodhart outside $\text{span}(D)$.
- Claim fidelity: a correct artifact for the wrong claim; the intent-to-claim gap per row.
- Harness honesty: a trial verdict is a report (A10 analogue); the ledger was produced by running something.
- Determinism beyond replayed inputs; behaviour under the harness equals behaviour in deployment.
- Halting: the fuel bound is enforced by the harness, not proved by the kernel.
- Sample independence: k draws from a provider that caches whole responses are one draw; receipt-visible as a repeated run id under I17, not proved.

- Refuter independence: combined power beyond a shared D.
- Generator exposure to a public refuter through pretraining (A10-class residue).
- Quality. Survival is not correctness. The brief did not claim it and neither does the paper.
- Body provenance is not closed; it is moved to the calibration record and still rests on I1 holding of the report.

8. Decision

The premise survives in the narrowed form above. Build it. The kernel is rga/core.py; the theorems are INVARIANTS.md R-invariants over the transition table there; the proofs inherit I1, I2, I3, I5 and I8 by citation; the check stage's dual control (I6) is inherited unchanged and no R-proof cites it.

9. Round-1 addendum

This file was written before the kernel, as the brief asked. An independent five-lens review of the finished kernel and papers (formal, enforcer, contrarian, attacker, document-versus-code; every finding handed to a skeptic) then broke parts of both, and the corrections are recorded here rather than silently absorbed.

Kernel defects, all reproduced, all repaired with tests: the defect model's author was a free label per record, so a second record on a clean first hash could carry the generator's authorship past both guards; Open accepted a caller-supplied journal position, bypassing the before-generation guard; replay could not rebuild a journal containing a discord or underpower close; a negative sample index slipped the range guard; the rebuilt machine's nonce source was a constant. Two FCD kernel defects surfaced the same way and are repaired in this branch because R7 and R8 lean on them: no_admit could write status=failed on an accepted item (no pc guard), and open silently replaced an existing id.

Corrections to this document: mode 2's seal contents are (ϵ , N), not (Q, ϵ , N, seed rule) — Q is the refuter's, the seed rule the kernel's; "Check 1, static" is the harness's obligation, not a kernel refusal; row 3's refuters are pinned by declared identity, not content hash; rows 12 and 14 are deployment obligations the kernel exposes as queries, not guards; §4's "A11 strengthened" was backwards — B7 states the collision direction honestly toward acceptance, weaker than A11's clause, which FCD's own I16/I17 proofs use fail-open (filed); "the refuter-is-not-the-author rule is I6 verbatim" over-claimed — it is I6's shape on declared strings (B11). The kernel has two power representations, not three; the exhaustive mode is a ledger over the whole domain. Fault codes were renamed V1–V15 so they never collide with theorem numbers R1–R13. And "measured doubt" required one more honesty note the review forced: with $N = 1$, a bounded refuter's carried figure equals its declared ϵ — the seal shows both, which is the whole defence.

10. The calibration round

RGA's own "What is not proved" names its successor gap the way FCD's named RGA's: B6 — whether D resembles the generator's real defects — is assumed, and nothing forces a found miss to have any consequence. A proposed escape-ledger layer (C1–C7: verified escapes, charge totality, a coverage ratchet, demotion, impeachment) went through the same premise discipline before any kernel: four adversaries, twenty-eight findings, each to a skeptic. Full record: eval/reviews/rga-calibration-premise-SYNTHESIS.md.

What survived changes the design: an escape is a **counterfactual trial of the pinned refuter at a finder-chosen nonce** — kernel-derived seed over the sealed bytes, refuted verdict, identical replay — adding no assumption the seal did not already carry; any other checker acts only through a journaled adjudication. Charges are write-once per wrong-verdict cell. The ratchet moves to Install with a Measure-first precondition, and install finally gets a journal event. Count-budget demotion was amputated to a pure query with a declared per-class Seal gate. The loop's ancestor is Bloomfield & Rushby's eliminative argumentation, mechanized. The honest sentence shrank again, as it should: not "the system cannot silently absorb evidence" but **forgetting is loud** — the theorems are about the ledger of found misses, never the distribution of real ones.

Appendix B — Invariants: R1–R13, C1–C7

Invariants — Refutation-Gated Admission

Roque Briceño. Safety only. Companion to PREMISE.md, PROOFS.md and DRAFT.md. Machine: rga/core.py, composed over fcd/core.py. Round 1: an independent five-lens review found the defect-model author was a free label per record, the before-generation guard trusted a caller-supplied journal position, and replay could not rebuild a discord or underpower close; all three are repaired below and their traces are in tests/.

RGA extends fail-closed class dispatch. It writes no FCD field. The FCD theorems I1–I17 (../INVARIANTS.md) are premises here, cited, never re-derived. Where this file says *inherited*, the guard in rga/core.py reads an FCD state variable and the proof cites the FCD theorem that constrains it. Theorems are **R1–R13**; faults are **V1–V15** — separate namespaces, as FCD separates I from F.

0. Assumptions

B0. FCD underneath. A0–A13 hold. The FCD machine is fcd/core.py. Every RGA guard that mentions pc, kind, a, m_decl, cls, body, policy_version or S reads that machine's state. No RGA transition writes any of it (R11).

B1. Harness honesty. A trial verdict, witness hash, calibration ledger, package-category set and sampling configuration are reports from a harness that ran what it names on the bytes the kernel hashed. This is the A10 boundary moved to the refuter runner. A forged *survived* satisfies the abstract guards; a forged *survived* on a deterministic refuter is a replay divergence when a third party re-runs it (R5), which is more than FCD can say of a forged m_exec.

B2. Pinned hermetic runtime. A refuter runs under a pinned toolchain with no clock, no network, no environment, and a runtime-counted fuel budget — never a wall-clock deadline. Under B2 a trial's observable is a function of (refuter version, artifact bytes, seed, fuel). This is a physical-boundary assumption of A10's standing; binding a refuter *version string* to refuter *bytes* is part of it. The kernel checks it only by falsification (Replay). A refuter outside the deterministic subset of the runtime is the harness's obligation to refuse; the kernel has no static check.

B3. Exhaustion is reported. Fuel or harness exhaustion, refuter crash, and refuter death are reported as inconclusive, never as survived. Survival is a positive completed verdict.

B4. Sample independence. k samples under one bind key are k draws from the bind's output distribution at the declared sampling configuration. A whole-response cache replay is one draw presented k times; it is receipt-visible as a repeated run id under I17 and is outside this machine.

B5. Defect model identity. A defect-model hash names one finite set of seeded defects, each shipped with a killing witness so that none is equivalent to the original. A calibration ledger enumerates exactly that set. The kernel fixes the id-set and the author by the first record against the hash (V13); that the hash actually names those contents is part of B1.

B6. Coupling. Kill-rate on a defect model is read as power against the generator's actual defects only under the coupling-effect hypothesis (DeMillo, Lipton & Sayward 1978). That reading is empirical. The seal carries the defect-model identity so the reading is the reader's, not the kernel's.

B7. **Hash.** sha256 is second-preimage resistant against an adversary who chooses the artifact bytes. A collision here fails toward *acceptance* — a refuted artifact could wear the seal of an unrefuted one — so B7 is an assumption, not a fail-closed property.

B8. **Claim language.** A class's claims are formal statements over a fixed artifact format. A refuter's witness is the canonical form of the claim-relevant observable it computed. Two witnesses agree iff their canonical forms are byte-equal. Inequality is discord; no canonicaliser launders a syntactic difference into agreement.

B9. **Nonce.** The sample nonce is drawn from an injected source after the artifact bytes are hashed and every Sample guard has passed, and is unpredictable to the generator at generation time. It is journaled, so replay reads it rather than redrawing it.

B10. **Seed discipline.** A seeded refuter uses exactly the seed the kernel derives (R10). A refuter that ignores its seed is a fixed-corpus refuter and is declared as one.

B11. **Author identity.** Refuter.author and DefectModel.author are declared strings in one namespace with the FCD specialist ids. The kernel compares them byte for byte and does not resolve, normalise or close them. R3 holds of the strings; that a string names the party that wrote the refuter or the defect model is assumed, as A10 assumes m_exec names the model that ran.

1. Objects

Refuter $r = (\text{id}, \text{version}, \text{author}, \text{mode})$, $\text{mode} \in \{\text{ledger}, \text{bounded}\}$. Registry-level, write-once per (id, version). Carries declared_at (journal position) and refused $\in \{0,1\}$, monotone. The version is a declared identity; binding it to bytes is B2.

Defect model $D = (\text{hash}, \text{author})$. Named by content hash. Its ledger id-set **and its author** are fixed by the first power record written against the hash (V13), so no two records disagree about either.

Power record $P[(r, D)]$, write-once. Ledger mode: (kills, size, killed_ids) with kills = $\{e \in \text{ledger} : e.\text{verdict} = \text{killed}\}$, size = $|\text{ledger}|$, computed by the kernel; inconclusive entries count in size and not in kills; carried power is kills/size, exact on D. Bounded mode: a declared (ϵ, N) ; carried power is $1 - (1-\epsilon)^N$, computed by the kernel, with ϵ and N carried into the seal so the declaration is visible — the figure is as declared as ϵ is, and the seal labels the mode. No interval is carried: a binomial interval assumes i.i.d. draws from the population the seal speaks to, and a mutant set is not that.

Claim spec (id, spec_hash, refuters $\subseteq R$, defect_model_hash). Fixed per class in the admission policy; ids unique per class; refuters nonempty. Claims are fully fixed at Open in this machine. Within-schema instantiation for discovery-shaped work — a claim whose instance is a sample-projected value with its own concordance — is *not modeled*; nothing in v1 represents it.

Admission policy per class: claims(c), $k(c) \geq 1$, $\theta(c) \in (0,1]$, $p_{\text{min}}(c) \in [0,1]$, excluded(c) (package categories the generator must not receive: refuter source, version, results, defect model), residual(c) (intents not attacked, each with disposition check_stage | unreviewed; possibly empty). Versioned. Installing a version string that already exists with different content is refused.

Line = one FCD work item id plus: generator (the specialist a), m_decl = $\phi(a)$ under the item's pinned FCD policy, sampling_hash, policy_version (RGA), claims snapshot, opened_at (RGA journal position) and the FCD journal position recorded at Open, samples, trials, pc $\in \{\text{Open}, \text{Sealed}, \text{Closed}\}$, pub, fault.

Bind key of a line: (cls, body, fcd_policy_version, generator, norm(m_decl), sampling_hash). This is the FCD-core form; a deployment that runs the context envelope additionally projects ExecutionEnvelope onto its non-attempt-local fields, which this kernel does not model. sampling_hash is a harness report equality-checked at Sample (B1), not an FCD field.

Sample i = FCD stage i of the item with pc = Passed, bound to the line's generator, plus artifact_hash = sha256(bytes) computed by the kernel, nonce drawn after every guard passed, m_exec copied from the stage. Write-kind and declared-model equality hold by construction (Open requires the first k stages to be write stages; both m_decl values come from $\phi(\text{generator})$ under the pinned policy). Sample i must be registered before any later sample stage is attempted (V8), so bytes cannot be assigned to slots after seeing the batch. samples[0] is the **designated** sample; it is stage 0, fixed before any sample exists.

Trial (r , claim, i , seed, inputs_hash, verdict, witness_hash, replays), verdict $\in \{\text{refuted, survived, inconclusive}\}$. At most one per (r , claim, i); $0 \leq i < |\text{samples}|$. inputs_hash is carried for audit and not compared by the kernel.

Seal = the record written to S_R (§4).

Store S_R : id $\rightarrow$ Seal. Written only by Seal (R8).

2. Transitions

Journal position is the index of the event a transition emits; "before" is strict order in that journal. The FCD journal positions recorded at Open and Sample are read by the kernel on the live path (len(fcd.events)) and from the journal on replay; no public transition accepts one from the caller.

Declare(r) — *Guard*: ($r.\text{id}$, $r.\text{version}$) $\notin R$; $r.\text{author} \neq \emptyset$; $r.\text{mode} \in \{\text{ledger, bounded}\}$ · *Effect*: $R \cup = \{r\}$; $r.\text{declared_at} \leftarrow \text{position}$; refused=0

Measure(r , D, ledger) — *Guard*: $r \in R$, $r.\text{mode}=\text{ledger}$, $\neg r.\text{refused}$; (r, D) $\notin P$; ledger $\neq \emptyset$, ids distinct, verdicts $\in \{\text{killed, survived, inconclusive}\}$; if the hash has a record: same id-set and same author; $D.\text{author} \neq r.\text{author}$ · *Effect*: $P[(r, D)] \leftarrow (\text{kills, size, killed_ids})$ computed from ledger; id-set and author fixed on first record

Bound(r , ϵ , N) — *Guard*: $r \in R$, $r.\text{mode}=\text{bounded}$, $\neg r.\text{refused}$; ($r, \perp$) $\notin P$; $0 < \epsilon \leq 1$; $N \geq 1$ · *Effect*: $P[(r, \perp)] \leftarrow (\epsilon, N)$; power $1 - (1 - \epsilon)^N$

Open(id, generator, sampling) — *Guard*: item id exists in FCD; line id not yet open; cls(id) $\in$ policy; generator $\in \pi(c) \setminus \delta(c)$ of the pinned FCD policy; stages[0..k) all kind=write; no FCD stage event for stages 0..k of id precedes the FCD position the kernel reads here; every pinned $r \in R$ (hence declared before this position — the journal is append-only), $\neg \text{refused}$, author $\neq$ generator; every claim's D with a fixed author: author $\neq$ generator · *Effect*: line created; claims $\leftarrow$ policy.claims(c); m_decl $\leftarrow \phi(\text{generator})$; pc=Open; positions recorded

Sample(id, bytes, categories, sampling) — *Guard*: pc=Open; $i = |\text{samples}| < k$; FCD stages[i]: pc=Passed $\wedge$ a=generator; categories $\cap$ excluded(c) = $\emptyset$; sampling = line.sampling_hash; no FCD stage event for stages $i+1..k$ precedes the FCD position read here · *Effect*: $h \leftarrow \text{sha256}(\text{bytes})$; $v \leftarrow \text{nonce}()$ (drawn after the guards); samples $\cup = (i, h, v, m_exec)$; position recorded

Trial(id, r, claim, i, seed, inputs, verdict, witness) — *Guard*: pc=Open; claim $\in$ claims; $r \in \text{claim.refuters}$; $0 \leq i < |\text{samples}|$; seed = H($v_i \mid h_i \mid r.\text{id} \mid r.\text{version} \mid \text{claim.id}$); no trial for (r , claim, i);

verdict $\in \{\text{refuted}, \text{survived}, \text{inconclusive}\}$ · *Effect*: append trial; if refuted $\rightarrow \text{Close}(V1)$; if inconclusive $\rightarrow \text{Close}(V3)$

Replay(id, t, verdict, witness) — *Guard*: trial t exists on line; $\neg r_t.\text{refused}$. Enabled in any line state: a post-seal audit replay that diverges still refuses the refuter (see tainted) · *Effect*: if (verdict, witness) $\neq (t.\text{verdict}, t.\text{witness}) \rightarrow r_t.\text{refused} \leftarrow 1$, monotone; every Open line pinning $r_t \rightarrow \text{Close}(V4)$; else $t.\text{replays} += 1$ (a counter outside every seal)

Seal(id) — *Guard*: $\text{pc} = \text{Open}$; $|\text{samples}| = k$; $\forall \text{ claim } j, \forall r \in j.\text{refuters}, \forall i < k$: trial (r, j, i) exists with verdict=survived; $\forall r$ used: some trial of r on this line has $\text{replays} \geq 1$; $\forall \text{ ledger } r \text{ of } j$: $P[(r, D_j)]$ exists; $\forall$ bounded r: $P[(r, \perp)]$ exists; $\forall j$: D_j 's fixed author $\neq$ generator; $\text{id} \in S$ (FCD); a residual disposition of check_stage requires a Passed FCD check stage · *Effect*: compute agreeing_j, composite_j; if some agreeing_j/k $< \theta \rightarrow \text{Close}(V2)$, naming in miss_observed exactly the refuters whose own witnesses differ from their sample-0 witness; elif $\min_j \text{composite}_j < p_{\min} \rightarrow \text{Close}(V5)$; else $S_R[\text{id}] \leftarrow \text{Seal}$; $\text{pc} = \text{Sealed}$

Close(id) — *Guard*: $\text{pc} = \text{Open}$ · *Effect*: $\text{pc} = \text{Closed}$; $\text{pub} = 1$; no fault (operator stop, published)

A guard failure not listed as a Close raises and writes nothing (V6–V15). A Close is published: it emits rga_close with the fault code; the Seal attempt that observes V2 or V5 publishes the Close and then raises to its caller. A refuter refused while a line is Open closes that line in the same step; a refused refuter cannot be pinned (Open), measured (Measure/Bound) or replayed further. No Trial or Seal guard tests refused, because no trace reaches one with a refused refuter — a guard no trace can reach is dead code. For the same reason there is no write-kind or declared-model guard at Sample, no refuter-author clause at Seal (a record by a refuter whose author equals the model's author is refused at Measure, and Seal requires the record), and no separate declared_at $<$ opened_at clause at Open (membership implies it in an append-only journal). Each removal is derived in PROOFS.md.

Concordance for claim j: $\text{vec}_j = \text{sorted}\{(r, \text{witness}) : \text{trial } (r, j, i)\}$; $\text{agreeing}_j = |\{i : \text{vec}_j = \text{vec}_0\}|$. Compared against the designated sample, never a plurality.

Composite power for claim j: let L be the ledger refuters of j with records against D_j , B the bounded refuters. If $|L| \geq 2$ their killed-id sets are unioned over the shared D_j : $u = |\cup \text{killed}|/\text{size}$. $\text{composite}_j = \max(u, \max_{\{b \in B\}} \text{power}_b)$, labelled union if only L contributed with $|L| \geq 2$, single if one refuter, max otherwise. No product, no noisy-OR.

3. Theorem statements

Proofs in PROOFS.md. Executable checks in tests/test_rga_invariants.py; each guard has a deletion proof in tests/test_rga_mutation.py.

- **R1** $\text{id} \in S_R \Rightarrow$ every (claim, refuter, sample) cell has a trial with verdict=survived, and no trial on the line has verdict $\in \{\text{refuted}, \text{inconclusive}\}$.
- **R2** $\text{id} \in S_R \Rightarrow$ every power figure in $S_R[\text{id}]$ equals a record $P[(r, D)]$ written before Seal by Measure or Bound from a ledger or a declared (ϵ, N) the seal carries; no transition writes an existing key of P; hence no sealed power is raised after the fact.
- **R3** $\text{id} \in S_R \Rightarrow$ for every refuter r used: $r.\text{author} \neq \text{generator}$; r was declared before the line opened; the line opened before any sample stage was attempted and each sample was registered before the next sample stage was attempted; the defect model's fixed author is neither the generator nor (for any record that exists) the recording refuter's author; and no sample's package contained a category in

excluded(c). All authorship clauses hold of declared strings (B11).

- **R4** $id \in S_R \Rightarrow$ for every claim j , $agreeing_j / k \geq \theta$, measured against stage 0; no transition writes a sealed artifact hash other than `samples[0].artifact_hash`.
- **R5** A replay divergence sets `refused=1` on that refuter; `refused` is monotone; after it, no line can pin, measure, bound or further replay it, and every line Open at refusal closes in the same step.
- **R6** $id \in S_R \Rightarrow$ every refuter used has at least one trial on that line with an identical replay.
- **R7** (inherited) every sample i of a sealed line is FCD stage i with `pc=Passed` and the line's bind key; hence $norm(m_exec_i) = norm(\phi(generator))$ (I1) and $generator \in \pi^*(c) \setminus \delta(c)$ (I2, I3).
- **R8** $id \in S_R \Rightarrow id \in S$ (hence `status=accepted` by I8 and every required FCD stage Passed by I5, and neither can be unwritten: NoAdmit requires `pc ∈ {Open,Closed}` and Open requires a fresh `id`). `S_R` is written only by Seal.
- **R9** $id \in S_R \Rightarrow$ every trial on the line is bound, through its seed, to the artifact hash the kernel computed at Sample for that sample; the seal's artifact hash is sample 0's.
- **R10** every trial's seed is $H(v_i \mid h_i \mid r \mid claim)$ where v_i was drawn at Sample i after h_i was computed and after every Sample guard passed; no seed exists before its artifact hash.
- **R11** no RGA transition writes any field of the FCD machine. Non-interference lemma; it is what lets the FCD proofs carry to the combined machine.
- **R12** `claims`, `k`, `θ`, `p_min`, `generator`, `m_decl`, `sampling_hash`, `policy_version` are write-once at Open.
- **R13** a line has at most k samples and at most one trial per cell, hence at most $|claims| \cdot \max_j |refuters_j| \cdot k$ trials; a power record is written at most once per key; a defect model's id-set and author are fixed by its first record. Replays are unbounded (Ask-idle analogue).

Not theorems: quality; truth of a claim; relevance of D to the generator's defects (B6); refuter determinism beyond replayed inputs (B2); halting (B3); harness honesty (B1); sample independence (B4); claim fidelity to intent; refuter independence beyond a shared D ; generator exposure to a public refuter through training; author identity beyond string equality (B11); that any consumer — the FCD DAG edge at `fcd/core.py open(depends_on=...)`, or a promotion predicate — reads `S_R` rather than `S`: the kernel exposes `check_dependencies`, `is_sealed`, `tainted` and `admissible`, and wiring them is a deployment obligation; that a retry after a refutation hides the refutation trace from the generator — `excluded(c)` refuses only the categories the harness reports (B1), and an adaptive retry is otherwise unlabelled. See PROOFS.md "What is not proved".

4. The seal

`S_R[id]` carries, and nothing else may be read as RGA authority:

- `artifact_hash` of the designated sample; `k`; `θ`; `p_min`; `sampling_hash`; RGA `policy_version`.
- FCD identity: `cls`, `body`, `generator`, `m_exec` of sample 0, FCD `policy_version`. (I1, I4.)
- Per claim: `claim_id`, `spec_hash`, each refuter as (`id`, `version`, `mode`, `power`, `D_hash` | $\perp$, `kills` | $\perp$, `size` | $\perp$, ϵ | $\perp$, `N` | $\perp$), `composite`, composition label, (`agreeing`, `k`).

- `power_min = min_j composite_j`.
- `residual`: the class's declared not-attacked intents (possibly empty) with disposition; `check_stage` is refused unless an FCD check stage Passed on this item.
- `sealed_at` (journal position).

A seal is a record. A later refusal of a refuter (R5) does not rewrite it; `tainted(id)` is a pure query that reports whether a sealed line relied on a refuter refused after sealing, `admissible(id)` is `sealed` $\wedge$ $\neg$ `tainted`, and `check_dependencies(deps, floor)` — the power-aware DAG gate a deployment calls before `fcd.open(..., depends_on=deps)` — refuses an unsealed, underpowered or tainted dependency. With $k = 1$, $(\text{agreeing}, k) = (1, 1)$: nothing was measured and the record shows it.

5. Faults

Each is a forbidden step. V1–V5 are observed and publish a Close. V6–V15 are guard refusals that raise and write nothing. Every fault names the guard method(s) that enforce it; `tests/test_rga_mutation.py` deletes each method singly and proves the fault's forbidden state becomes reachable.

- V1** — Seal after a trial on the line returned refuted
- V2** — Seal with `agreeing_j/k <  $\theta$` on any claim
- V3** — Seal counting an inconclusive trial as survival
- V4** — Agreement recorded for a replay that diverged; Measure, Bound or Replay on a refused refuter
- V5** — Seal with `min_j composite_j < p_min`
- V6** — Sample from a stage that is not Passed or is bound to a different specialist than the line
- V7** — Sample whose generator package contained a category in `excluded(c)`
- V8** — Open after a sample stage was attempted; Sample *i* after a later sample stage was attempted; Trial against a claim not in the line's snapshot or a refuter not pinned to it
- V9** — Trial whose seed is not $H(v_i \mid h_i \mid r \mid \text{claim})$
- V10** — Seal without *k* samples or with a (claim, refuter, sample) cell lacking a surviving trial
- V11** — Seal using a refuter with no identical replay on this line
- V12** — Seal using a refuter with no power record
- V13** — Measure or Bound on an existing key; a sample beyond *k*; a second trial in one cell; a ledger whose id-set, or a record whose author, differs from the defect model's first record
- V14** — A pinned refuter authored by the generator; a defect model authored by its recording refuter's author, or whose fixed author is the generator

V15 — Seal of a line whose item is not in S; a residual that claims check_stage with no Passed check stage; any write to S_R other than Seal

6. Rates

Defined as in ../INVARIANTS.md §5, on a named cut [t0,t1] with window W. Policy as-of ts. None are theorems.

- **Refutation rate** per class and refuter version: trials with refuted over trials.
- **Discord rate** per class at declared (k, sampling_hash) (joined from rga_close to rga_open on the line id): lines closed V2 over lines reaching Seal.
- **Miss observed** per refuter version: V2 closes whose miss_observed names it — the refuter's own witnesses differed across samples it declared survived. For a claim whose witness is a function of the claim value, that is a detected miss: two claim-different samples both survived it. For other witness designs it is instability evidence only.
- **Replay divergence** per refuter version: Replays that diverged over Replays. The refuter analogue of misbind; always published with silent-fail.
- **Escape replay** per refuter version v' : of defects later found in artifacts sealed under v , the fraction v' kills on the pre-fix bytes. Requires a defect-report channel outside this machine (its numerator has no journal event); informative only for $v' \neq v$ (a deterministic v reproduces its own miss); doubly selected. A journal-plus-reports rate keyed to the successor version, never a rewrite of any seal.

7. Illustration (not a corollary)

A generator that has seen a fixed public refuter can special-case it and survive at the refuter's full carried power. R3 forbids the refuter being shaped by the artifact; nothing in this table forbids the artifact being shaped by the refuter, except the post-artifact seed (R10) for seeded refuters and excluded(c) (R3) for fixed corpora the generator's package would otherwise carry. Pretraining exposure is outside both. A retry opened after a refutation, with the refutation trace in the generator's context, is the same picture one step later: the samples become adaptive, and this table neither forbids nor labels it beyond excluded(c). Do not cite R3 as a theorem about Goodhart.

8. Calibration — the escape ledger

Machine: rga/calibration.py, composed over Admission the way Admission composes over FCD.
Premise round: eval/reviews/rga-calibration-premise-SYNTHESIS.md; corrections to the first sketch are recorded there, including one authored adjudication between conflicting reviewer verdicts.
Theorems are **C1–C7**; faults are **E1–E9**. "Calibration" is used in this repository's metrological, mode-3 sense — the reference corpus is ratcheted against verified real defects. The statistical sense is *not proved* and is filed as such: these theorems are about the ledger of found misses, never the distribution of real ones. What is proved is: forgetting is loud.

8.0 Assumptions

B12. Post-seal byte availability. Filing an escape requires the sealed bytes; whoever found the defect holds them, and any hash-matching copy suffices (B7). Loss of bytes censors the escape stream one-sidedly: it slows revocation and can never manufacture admission. This is the standing condition

RGAs post-seal audit replay already carried, now load-bearing.

B13. Governance visibility, not governance defeat. Budget values, Seal-gate bits, exclusion decisions and tier-B adjudications are operator acts. The kernel makes each one a journaled, attributed, primary-carrying event; it does not and cannot make the operator honest. The operator is the trust root here as A10's adapter is FCD's.

B14. Authenticated-publication trust boundary. The composed receipt theorem assumes one authority stack with a stable registry namespace, atomic single-writer authority transitions, an honest signer whose key is not disclosed, and an external monotone registry that cannot be rolled back with the journal. Unrelated stacks use distinct namespaces; the registry serializes concurrent updates across them. Historical authenticity begins at a trusted first anchor or continuous publication: a first receipt cannot prove which coherent pre-anchor history actually ran. The computational claim assumes HMAC-SHA256 unforgeability and SHA-256 collision and second-preimage resistance. The bundled HMAC signer authenticates within that trust domain; it is not public-key non-repudiation. Compromise or coordinated rollback of signer/key and registry defeats currency and is not proved away.

8.1 Objects

Escape = a filed counterexample against a sealed line's claim: $(\text{line_id} \in S_R, \text{claim_id}, \text{checker} = (\text{id}, \text{version}), \text{nonce}, \text{artifact bytes}, \text{witness_hash})$. Two tiers:

- **Tier A** — the checker is a refuter **pinned to that claim in the line's seal**. The finder authors nothing but the nonce and the bytes. The kernel computes $\text{sha256}(\text{bytes})$ and refuses unless it equals $\text{seal.artifact_hash}(\text{single-holder}, \text{the sample}())$ pattern; derives $\text{seed} = H(\text{nonce} \mid \text{artifact_hash} \mid \text{checker} \mid \text{claim})$ itself; records the reported verdict, which must be refuted. This is a counterfactual trial: the same pure function the seal's own trials evaluated (B2), at another point of its seed domain. It adds no assumption the seal did not already carry.
- **Tier B** — any other declared checker. A tier-B escape has **no effect of any kind** until a journaled adjudication event (named actor, decision, reason) accepts it. The adjudication is the per-escape claim-fidelity judgment (B8 of the checker), made visible instead of pretended away.

Established = the escape's run has one identical replay (verdict and witness equal). A divergent replay **discredits** the checker in this ledger, monotonically. **Valid** (read at query time, never stored): $\text{established} \wedge \text{checker not discredited} \wedge \text{checker not refused in the Admission registry} \wedge (\text{tier B} \Rightarrow \text{adjudicated accepted})$.

Charge = the journaled wrong-verdict cell $(\text{line_id}, \text{claim_id}, \text{refuter_version})$: the pinned refuter returned survived on the designated sample and a valid escape exists against that cell. **Write-once per cell** — witness multiplicity and checker multiplicity never multiply charges; the refuter's chargeable error is its single journaled verdict.

Corpus per class: every valid escape contributes a derived seeded defect whose killing witness is the escape witness (B5 by construction), identified by the escape's journal position, carrying the finder as **journal-cited provenance metadata** — never fed into any author gate, never deriving any exclusion (the adjudicated decision of the premise round: an added entry is a monotone kill obligation and cannot advantage its finder).

Exclusion = a journaled operator decision (class, escape ids, actor, reason) releasing named corpus entries from the successor-coverage obligation. It waives nothing else: charges, impeachment, track records and the corpus itself stay monotone.

Calibration policy per class: $e_max \geq 0$ (charge budget per refuter version within the class) and $demotion_gate \in \{\text{seal}, \text{carry}\}$. Both explicit; no defaults.

8.2 Transitions

The authority wraps Open, Seal and Install the way the reference server wraps `decide_pass` (the GatePass interposition pattern): each Cal-row is a guard conjunction in front of the corresponding Admission row, driven through Admission's own guarded transition. A deployment that bypasses the wrapper has RGA's guarantees and not these — the consumer-redirection obligation, one layer up, stated. Seal- and claim-membership at filing are structural preconditions (an unknown line or claim cannot be named), not deletable predicates. Replay re-checks every filing guard and entailment before writing, and requires a diverged replay to be adjacent to its discredit event; PROOFS.md, *Replay of the ledger*, states its one-sided registry residue.

FileEscape — *Guard*: $\text{line} \in S_R$; claim in the seal; checker declared; $\text{sha256}(\text{bytes}) = \text{seal.artifact_hash}$; tier A: checker pinned to the claim in the seal and filed seed equals the kernel's derivation; verdict = refuted · *Effect*: escape recorded with tier, finder, position

ReplayEscape — *Guard*: escape exists; checker not discredited · *Effect*: equal (verdict, witness) → established; unequal → checker discredited, monotone; every unestablished escape of a discredited checker is void

Adjudicate — *Guard*: tier-B escape established; named actor · *Effect*: decision accept or reject, with reason, journaled; accept makes it valid

Exclude — *Guard*: class has the named valid escapes in its corpus; named actor; reason · *Effect*: exclusion recorded with primaries

CalInstall — *Guard*: every class of the incoming admission policy carries an explicit calibration policy, supplied with it and installed in the same step (E9); every **ledger** claim's defect-model hash has a kernel-fixed id-set (Measure before Install — always achievable: measure reads no policy state); each ledger claim's id-set $\supseteq$ the class corpus's derived-defect ids minus excluded ids as of this position; no claim is bounded-only while the class corpus minus exclusions is nonempty; no class with a nonempty net corpus is absent from the incoming policy · *Effect*: drives Admission.install; emits the install event with coverage primaries (corpus size, excluded count, per-claim model map), the predecessor-policy diff of dropped defect ids, and any dropped classes

CalOpen — *Guard*: the class carries an explicit calibration policy (E9); no refuter pinned by the class is demoted here or refused in Admission · *Effect*: drives Admission.open

CalSeal — *Guard*: the class carries an explicit calibration policy (E9); the line is Open (checked before any emission — a close event for a line that stays Sealed would be a journaled step that never happened); if the class declares $\text{demotion_gate} = \text{seal}$: no pinned refuter demoted as of this position — else publish close, fault E5, carrying that refuter's primaries; always: stamp event with each pinned refuter's track-record primaries and the corpus finder-provenance split as of this position · *Effect*: drives Admission.seal; stamp emitted in the same step

Audit — *Guard*: $\text{line} \in S_R$; a checker pinned on the claim or the checker of a valid escape of the class, run against the sealed bytes (kernel hashes them; tier-A seed discipline applies to pinned checkers), verdict survived, replayed · *Effect*: audit event; exposure queryable

Pure queries, never writes: `charges(refuter_version, class)` — the valid charge cells; `demoted(r, class)` = `charges > e_max`; `impeached(id)` = some valid escape against the seal; `suspect(id)` = some direct dependency impeached or tainted (transitive coverage is the consumer's fold over `depends_on`, RFC 5280 shape); `mediated(id)` = exactly one stamp bound to this seal's position, so a seal produced by calling `Admission.seal` directly is layer IR and distinguishable from IRC; `admissible` extended to $\wedge$ `mediated` $\wedge$ $\neg$ `impeached`; `check_dependencies` refuses impeached dependencies; `audit_exposure(id)`.

8.3 Theorem statements

- **C1 Established, never asserted.** A tier-A escape has effect only past kernel-checked preconditions — seal membership, byte-hash equality, pinned checker, kernel-derived seed, refuted verdict, identical replay — none of which is a declaration; a tier-B escape has effect only past a journaled, attributed adjudication. No other path to effect exists.
- **C2 Charge totality and unit.** The charges standing against a refuter version are exactly the valid wrong-verdict cells derivable from the journal — read, never declared — and at most one per (line, claim, refuter_version).
- **C3 Impeachment is entailed and checkable.** `impeached(id)` is a pure query entailed by a valid escape; the seal is never rewritten; validity degrades (checker discredited or refused) by journal facts, never by discretion. Non-discretion is the property CRL/OCSP lack.
- **C4 The ratchet.** No policy installs unless every **ledger** claim's defect model is Measured and covers the class's valid corpus minus journaled exclusions as of the install position, no claim is bounded-only against a nonempty net corpus, and no class that owes coverage is dropped from the policy. The install event carries the coverage primaries, the per-claim model map, the dropped-id diff and any dropped classes. Hence: no successor forgets a valid escape without a named, attributed, journaled decision — forgetting is loud.
- **C5 Track record carried, and mediation provable.** Every CalSeal stamps the seal in the same step, and `mediated(id)` reads that stamp back: exactly one, bound to the seal's own position. A line sealed without passing through this authority has no stamp, so it is layer IR — `admissible` refuses it — and a consumer can tell the two apart. Re-pointing a stamp, duplicating one, or adding one out of seal order is refused on rebuild; adding one for the most recent unmediated seal is not — nothing earlier contradicts it — and removing one lowers the line to IR. Both residues are stated in PROOFS.md rather than claimed away.
- **C5a Primaries.** Every CalSeal stamps, in the same step as the seal, each pinned refuter's primaries as of that position — charged cells, seals participated in, corpus size and exclusions — and the class corpus's finder-provenance split for this seal's generator; never a rate; a zero is absence of filed evidence and the record says so.
- **C6 Demotion is a query with declared consequences.** `demoted` is a pure function of the valid charge set against the declared budget — rising as valid escapes accumulate, falling when a checker's discredit or refusal degrades their validity (refusal-shaped facts outrank the count: the fail-closed direction). It blocks new pins at CalOpen unconditionally; it gates CalSeal exactly where the class declared it, closing with a published fault that carries the primaries; it never cascades events and never touches an existing seal.
- **C7 Non-interference.** The calibration authority writes no Admission or FCD field directly; its only effect on the lower machines is through their own guarded transitions. Every R- and I-theorem holds on the combined trace.

Authenticated publication (conditional on B14). `issue_admissibility_receipt` accepts exactly one composed Enforcer → Admission → CalibrationAuthority stack and a stable namespace, snapshots its immutable journals, derives the subject's artifact and four predicates from that state, signs all three ordered heads plus the composed receipt, and advances the three registry heads atomically. Each successor head carries the new event digests; the registry recomputes a cumulative chain from its anchored head, so greater length without prefix extension is refused. It refuses if the journals or predicates move while signing. `verify_admissibility_receipt` authenticates the composed fields and requires every embedded head to equal the namespace's registry-current digest and the current journal body. Therefore, after a trusted anchor, a verified registry-current receipt cannot be produced by public rehashing, cross-wiring, truncation, coherent deletion, a longer fork, state-state replay, namespace collision, or predicate substitution unless B14 fails. Here *stale* means not registry-current for the present journal state; composed `issued_at` is signed metadata, not an expiry or wall-clock-freshness theorem.

Not theorems: corpus representativeness — B6 is relocated to the corpus, not closed; the corpus is a selected, adversarially steerable subsample of real defects, and flooding it with true entries steers every aggregate built on it. Search intensity — zero charges is unreadable between sound and unsearched except through journaled audits, and those measure power against the corpus, never against undiscovered defects (B6 one level up). Audit-campaign scope honesty (B1 one level up). The residual disincentive to reveal defects only one's own instruments miss. Tier-B checker claim-fidelity — adjudicated, never kernel-checked. Statistical calibration — no bounded escape rate, no convergence of D to the generator's defect distribution, and none soundly statable under an adversarially censored escape channel (Gibbs & Candès 2021 is the shape such a theorem has, and why it does not fit here). Operator honesty (B13). Authenticated publication when the receipt path is bypassed, signer/key custody, registry durability, rollback of both key and registry, or concurrent writers outside the authority model (B14).

8.4 Faults

E5 publishes a close through CalSeal. E4, E6, E7 and E9 are guard refusals that raise and write nothing. E1–E3 are validity semantics read at query time: an invalid escape simply has no effect — there is nothing to publish or refuse — and `_check_valid` is the deletable method that holds them. E8 is the writer inspection, held by C7's proof and by replay re-guarding every write. Every fault names the method(s) that enforce it; `tests/test_rga_calibration.py` deletes each singly and proves the fault's forbidden state becomes reachable.

E1 — Effect from an escape that is not established, or whose checker is discredited or refused

E2 — Effect from a tier-B escape with no accepted adjudication

E3 — A second charge in one (line, claim, refuter_version) cell

E4 — An install whose ledger defect models omit a valid, unexcluded corpus entry — or reference an unmeasured D, leave a claim bounded-only against a nonempty corpus, or drop a class that owes coverage

E5 — CalSeal under a demoted refuter where the class declared the gate

E6 — FileEscape whose bytes do not hash to the seal's artifact hash, whose checker is not pinned (tier A), or whose seed is not the kernel's derivation

E7 — Exclusion or adjudication without a named actor and reason; a second adjudication of one escape

E8 — Any write to charges, corpus, exclusions or stamps except through the named transitions

E9 — A CalOpen, CalSeal, budget query or install for a class carrying no explicit calibration policy — e_max and demotion_gate have no defaults, and an unconfigured class must never behave as an unlimited one

8.5 Rates

All on named cuts; none theorems. Escape rate per class and tier; adjudication acceptance rate (tier B); exclusion load per install (primaries, already carried); charge accrual per refuter version; audit coverage per class (lines audited over lines sealed — one-sided, reads only with the audit events); the §6 escape-replay rate now has its journal numerator for tier-A escapes.

8.6 Ancestry

Eliminative argumentation and Assurance 2.0 defeater discipline (Goodenough, Weinstock & Klein 2015; Bloomfield & Rushby), mechanized: established escapes are replayed kills, not dispositioned prose, and the non-covering successor is refused by the kernel, not flagged for an assessor. Escape-to-test regression practice and mined-from-real-faults mutation (Tufano et al. 2018; Beller et al. 2021; Defects4J) for the corpus. Flaky-test quarantine and SRE error budgets for the budget-with-consequence shape — which lack the verified false-negative channel this layer creates. Browser root-program distrust of certificate authorities for the institutional precedent. CRL/OCSP/Sigstore for revocation-consulted-at-use, minus the two properties with no prior formalization found: non-discretion and charge totality. Mills 1972 for seeded audit controls.

Appendix C — Proofs

Proofs — Refutation-Gated Admission

Roque Briceño. Inductive proofs of R1–R13 on the machine in INVARIANTS.md, composed over the FCD machine. No quality. No truth of any claim. No liveness. Round 1: an independent five-lens review broke three of these proofs as first written — the defect-model author was a free label per record, Open trusted a caller-supplied journal position, and replay could not rebuild a V2/V5 close; the repairs are cited below and their traces are in tests/.

Notation. A **trace** is a finite sequence of transitions of the combined machine from the initial state (no lines, $R=\emptyset$, $P=\emptyset$, $S_R=\emptyset$, FCD initial). Each step applies exactly one enabled row of either table. A citation `rga/core.py:n:symbol` names the line that carries the claim; `tests/test_rga_citations.py` opens the file and fails if line n does not contain `symbol` — a move-detector that keeps citations from going stale, not a semantic check. The sentence, not the test, claims enforcement. FCD theorems are cited as premises from `../PROOFS.md` and are not re-derived.

Writer inspection, used throughout: S_R is written at `rga/core.py:604:sealed` and nowhere else; `sealed_put` raises at `rga/core.py:643:PermissionError`. P is written at `rga/core.py:404:power` (ledger) and `rga/core.py:426:power` (bounded) and nowhere else. `refused` is written at `rga/core.py:569:refused` and nowhere else. A line's `pc` becomes `Sealed` only at `rga/core.py:605:Sealed` and `Closed` only at `rga/core.py:635:Closed`.

R1 No admission without survival

Claim. $id \in S_R \Rightarrow$ every cell (claim, refuter, sample) of the line has a trial with verdict = survived, and no trial on the line has verdict $\in \{\text{refuted, inconclusive}\}$.

Proof. Induction on trace length. Base: $S_R = \emptyset$. Step: the only writer of S_R is `Seal` (`rga/core.py:604:sealed`). `Seal` is enabled only past `_guard_seal_complete` (`rga/core.py:795:_guard_seal_complete`), which requires `|samples| = k` (`rga/core.py:797:samples`) and, for every claim, every pinned refuter and every $i < k$, a trial in that cell whose verdict is survived (`rga/core.py:806:survived`). That gives the first conjunct. For the second: a trial with verdict = `refuted` closes the line in the same step at `rga/core.py:848:refuted`, and `inconclusive` at `rga/core.py:853:inconclusive`; a `Closed` line cannot `Seal` (`rga/core.py:578:seal` requires `pc = Open`), and nothing reopens a line. Since at most one trial exists per cell (`rga/core.py:752:cell`), a cell that holds a surviving trial holds no other. QED

Remark. The close at `Trial` is the publication; the guard at `Seal` is the safety. Deleting either alone leaves the other standing; `tests/test_rga_mutation.py` deletes each separately and both together and shows all three go red.

R2 Power is carried, never inferred

Claim. $id \in S_R \Rightarrow$ every power figure in $S_R[id]$ equals a record of P written by `Measure` or `Bound` before `Seal`; no transition writes an existing key of P ; hence no sealed power is raised after the fact.

Proof. The seal's per-refuter power is read in `_claim_seal` (`rga/core.py:901:_claim_seal`) from `_record_for` (`rga/core.py:369:_record_for`), which returns $P[(r, D_j)]$ or $P[(r, \perp)]$; `_guard_seal_measured` (`rga/core.py:820:_record_for`) refuses `Seal` if any is absent, so every figure comes from P . P is written only at `rga/core.py:404:power` and `rga/core.py:426:power`. The ledger writer computes kills as the count

of killed entries (rga/core.py:403:kills) and carries size = |ledger|; the figure is kills/size (rga/core.py:104:kills), exact on D. The bounded writer stores the declared (ϵ , N) and the figure is $1 - (1 - \epsilon)^N$ (rga/core.py:105:epsilon), computed in the kernel, with ϵ and N themselves carried into the seal by `_claim_seal` so the declaration is visible: ledger mode accepts no scalar; bounded mode accepts two labelled ones and shows them. Write-once: `_guard_power_once` refuses an existing ledger key (rga/core.py:762:power) and `_guard_bound_once` an existing bounded key (rga/core.py:780:power). A seal is a frozen record written once; a later Measure or Bound on the same key is refused, and on a different key does not touch the seal. QED

Remark. The composite per claim is the union of killed-id sets over a shared defect model (rga/core.py:925:killed) or a max (rga/core.py:929:composite), both functions of records in P. No product, no independence assumption. Inconclusive ledger entries count in size and not in kills — toward doubt.

R3 Separation of duty, extended

Claim. $\text{id} \in S_R \Rightarrow$ for every refuter used: its author is not the generator; it was declared before the line opened; the line opened before any sample stage was attempted and each sample was registered before the next sample stage was attempted; the defect model's fixed author is neither the generator nor any recording refuter's author; and no sample's package contained an excluded category. Authorship clauses hold of declared strings (B11).

Proof. Open pins the claim snapshot (rga/core.py:464:lines) and is enabled only past `_guard_pinned_before_open` (rga/core.py:687:_guard_pinned_before_open), which requires every pinned refuter to be a member of the registry (rga/core.py:693:refuters) and to have `author  $\neq$  generator` (rga/core.py:690:author). Membership implies declared strictly before Open: `declared_at` is written once at declaration (rga/core.py:385:declared_at) and the journal is append-only, so a member's position precedes any later transition's — no separate ordering clause is needed and none exists. Temporal ordering against generation: Open is enabled only past `_guard_open_before_generation` (rga/core.py:704:stage), which reads the FCD journal up to the position the kernel itself recorded — the public row takes no position from the caller (rga/core.py:438:_position), which round 1 showed would be a bypass — and refuses if any stage event for the item's first k stages precedes it; FCD emits stage on every Admit (fcd/core.py:242:admit). The same construction at Sample (rga/core.py:738:stage) refuses a sample registered after a later sample stage was attempted, so bytes cannot be assigned to slots after seeing the batch. Defect-model authorship: the author is fixed by the first record against the hash (rga/core.py:406:defect_authors); a later record with a different author is refused by `_guard_model_author_fixed` (rga/core.py:783:author) — round 1 reproduced a seal through a second, generator-authored record before this guard existed; a record whose author equals the recording refuter's author is refused by `_guard_independent_model` (rga/core.py:792:author); a fixed author equal to the generator is refused at Open (rga/core.py:700:defect_authors) for records that exist then and at Seal (rga/core.py:829:defect_authors) for records written after Open — both read the one fixed value, so they cannot disagree. Package exclusion: Sample is enabled only past `_guard_sample_package` (rga/core.py:730:leaked). All of these hold at the step that writes the fact and nothing rewrites it; S_R is written only past all of them. QED

Remark. This is I6 (fcd/core.py:58:authors) in shape — separation of duty — lifted from the checker's stage identity to the refuter's declared provenance, plus temporal clauses FCD has no need of. It is only as strong as the author strings are honest (B11). It forbids the refuter being shaped by the artifact; it does not forbid the artifact being shaped by a refuter the generator already knows. See "What is not proved".

R4 Concordance is a precondition

Claim. $\text{id} \in S_R \Rightarrow$ for every claim, $\text{agreeing}/k \geq \theta$, where agreeing counts samples whose witness vector equals sample 0's; and the sealed artifact hash is sample 0's.

Proof. agreeing is computed at `rga/core.py:903:agreeing` as the number of $i < k$ with `_witness_vector(i) = _witness_vector(0)` (`rga/core.py:897:_witness_vector`), the vector being the sorted (refuter, version, witness) triples of that sample's trials on that claim. Seal proceeds only if `_check_concordance` (`rga/core.py:862:_check_concordance`) returns true, which is false — and closes the line with V2 — if any claim has $\text{agreeing}/k < \theta$ (`rga/core.py:863:theta`). The V2 close names in `miss_observed` exactly the refuters whose own witness differs across samples (`rga/core.py:868:misses`), not every refuter on the claim. The seal's artifact_hash is `samples[0].artifact_hash` (`rga/core.py:578:seal`, effect block). Sample i is FCD stage i — the index is forced to `|samples|` under the k bound (`rga/core.py:715:k`) — so sample 0 is stage 0, which is fixed before any sample exists; no transition chooses it. QED

Remark. A plurality against sample 0 is discord, not a winner: with $\theta = 0.6$, samples 1 and 2 agreeing with each other and not with 0 gives $\text{agreeing} = 1$ and closes (`tests/test_rga_invariants.py`, `test_majority_against_designated_is_discord_not_a_winner`). With $k = 1$, $\text{agreeing}/k = 1$ and the seal shows (1, 1); nothing was measured and the record says so. Below $\theta = 1$, a discordant survival can seal — the miss is visible in ($\text{agreeing}, k$), and only a V2 close journals `miss_observed`.

R5 Nondeterminism refuses

Claim. A replay divergence sets refused on that refuter; refused is monotone; afterwards no line can pin, measure, bound or further replay it, and every line Open at refusal closes in the same step.

Proof. Replay compares the reported (verdict, witness) to the recorded pair (`rga/core.py:858:witness_hash`). On inequality, `_refuse` (`rga/core.py:568:_refuse`) adds the key to refused (`rga/core.py:569:refused`) and closes every Open line that pins it. No transition removes from refused. Measure, Bound and Replay refuse a refused refuter through `_guard_not_refused` (`rga/core.py:676:refused`); Open refuses to pin one (`rga/core.py:690:refused`). A line Open at refusal is Closed in the same step and cannot Seal; a line opened later cannot pin the refuter; a line sealed earlier is a frozen record, and tainted (`rga/core.py:651:tainted`) reports it, admissible (`rga/core.py:663:admissible`) excludes it, and `check_dependencies` (`rga/core.py:667:check_dependencies`) refuses it as a dependency. No Trial or Seal guard tests refused: no trace reaches either with a refused refuter (the pinning line is already Closed), and a guard no trace can reach is dead code. QED

Remark. Agreement on replay increments replays (`rga/core.py:559:replays`) and proves nothing beyond that one input. This is B2 checked by falsification. Replay is enabled in any line state — a post-seal audit replay that diverges still refuses the refuter, which is what tainted exists to report; the counter it increments is outside every seal.

R6 Replay exercised

Claim. $\text{id} \in S_R \Rightarrow$ every refuter used on the line has a trial on that line with replays ≥ 1 whose replay returned the recorded outcome.

Proof. Seal is enabled only past `_guard_seal_replayed` (`rga/core.py:812:replays`), which requires, for every pinned refuter, some trial of that refuter on this line with replays ≥ 1 . `replays` is incremented only at `rga/core.py:559:replays`, on the branch where `_check_replay` returned false, i.e. the replay equalled the recorded (verdict, witness). QED

R7 Sample integrity, inherited

Claim. Every sample i of a sealed line is FCD stage i of the line's item with $pc = \text{Passed}$ and the line's bind key. Hence, for each sample, $\text{norm}(m_exec_i) = \text{norm}(\phi(\text{generator}))$ and $\text{generator} \in \pi^*(c) \setminus \delta(c)$.

Proof. Sample i is registered against stages $[i]$ (rga/core.py:482:_register_sample) only past _guard_sample_stage, which requires $pc = \text{Passed}$ (rga/core.py:723:Passed) and $a = \text{generator}$ (rga/core.py:725:generator). Write-kind and declared-model equality hold by construction rather than by guard: Open requires the first k stages to be write stages (rga/core.py:440:_open), the sample index is forced below k (rga/core.py:715:k), and both m_decl values are $\phi(\text{generator})$ under the one policy the item pinned — a guard for either would be unreachable, and none exists. FCD sets $pc = \text{Passed}$ only at fcd/core.py:373:Passed, inside decide_pass, past the guard $\text{norm}(m_exec) = \text{norm}(m_decl)$ (fcd/core.py:362:norm), with m_decl written only by Bind as $\phi(a)$ (fcd/core.py:319:phi) and m_exec only by Observe (fcd/core.py:339:m_exec). That is I1 (./PROOFS.md, I1), applied per stage. a is written only by Admit under $a \in \pi^* \setminus \delta \setminus \text{tried}$ (fcd/core.py:248:allow): I2, hence I3. RGA writes none of these fields (R11), and a Passed stage of an accepted item cannot later be rewritten — NoAdmit now requires $pc \in \{\text{Open}, \text{Closed}\}$ (fcd/core.py:269:no_admit) and Open a fresh id (fcd/core.py:219:exists), the two FCD repairs round 1 forced — so the copied m_exec is the Pass-time value I1 speaks of. QED

R8 Seal implies Accept

Claim. $\text{id} \in S_R \Rightarrow \text{id} \in S$; hence status = accepted (I8) and every required FCD stage Passed (I5), and neither can be unwritten afterwards. S_R is written only by Seal.

Proof. Seal is enabled only past _guard_seal_accepted (rga/core.py:834:store), which requires $\text{id} \in S$. S is written only by FCD Accept (fcd/core.py:418:store), which sets status = accepted in the same step (fcd/core.py:417:accepted) and is enabled only if every stage Passed (fcd/core.py:415:Passed): I5, I8. Once accepted, no writer of status is enabled — NoAdmit requires $pc \in \{\text{Open}, \text{Closed}\}$ (fcd/core.py:269:no_admit) and every stage is Passed — and the item itself cannot be replaced under the seal, because Open refuses an existing id (fcd/core.py:219:exists). Round 1 reproduced both rewrites on the unguarded FCD kernel; tests/test_core.py (KernelIdentityGuards) and tests/test_rga_invariants.py (test_accepted_item_cannot_be_failed_or_reopened_underneath_a_seal) witness the repairs. S_R is written only at rga/core.py:604:sealed; sealed_put raises (rga/core.py:643:PermissionError). QED

Remark. $S_R \subseteq S$. A consumer that reads S gets FCD's guarantee; one that reads S_R gets this paper's. The kernel exposes the power-aware DAG gate check_dependencies (rga/core.py:667:check_dependencies), which also refuses tainted dependencies, and the predicates is_sealed (rga/core.py:647:is_sealed) and admissible (rga/core.py:663:admissible). Wiring them — in place of the FCD DAG edge and the injected is_accepted — is a deployment obligation, not a theorem of this machine; it is listed under "Not theorems".

R9 Artifact binding

Claim. $\text{id} \in S_R \Rightarrow$ every trial on the line is bound, through its seed, to the artifact hash the kernel computed at Sample for that sample, and the seal's hash is sample 0's.

Proof. sample hashes the bytes it is handed (rga/core.py:474:sample, sha256(artifact)) and _register_sample stores that hash (rga/core.py:498:samples). Trial is enabled only past _guard_trial_seed (rga/core.py:751:_guard_trial_seed), which requires the trial's seed to equal

derive_seed(nonce_i, artifact_hash_i, r, claim) (rga/core.py:754:derive_seed), a sha256 over those fields (rga/core.py:56:derive_seed). Under B7 no other (nonce, hash) yields that seed. The seal copies samples[0].artifact_hash. QED

Remark. FCD has no output object (./PROOFS.md, "Body provenance"). This is the field it lacked, computed by the authority over bytes it holds, as compile_package already does for inputs. Which stage produced which bytes remains a report (B1); the interleaving guard (R3) forces the assignment before later artifacts exist, no more.

R10 Seed after artifact

Claim. Every trial's seed is a function of a nonce drawn at Sample after the artifact hash was computed and after every Sample guard passed; no seed exists before its artifact hash.

Proof. _register_sample (rga/core.py:482:_register_sample) runs every guard first and draws the nonce only on the live path afterwards, so a refused Sample consumes nothing (tests/test_rga_invariants.py, test_refused_sample_consumes_no_nonce); the nonce is stored on the sample (rga/core.py:498:samples). seed_for refuses an index outside [0, |samples|) (rga/core.py:509:sample_index). derive_seed takes both fields. A seed for sample i therefore exists only after sample i exists, which is after its hash. Replay reads the nonce and the journal position from the rga_sample event rather than redrawing either (rga/core.py:1004:nonce). QED

Remark. Unpredictability of the nonce to the generator is B9, an assumption about the injected source. The theorem is the ordering.

R11 RGA writes no FCD field

Claim. No RGA transition writes any field of the FCD machine.

Proof. Inspection. Every reference to self.fcd in rga/core.py is a read: items and stages (rga/core.py:482:_register_sample), store (rga/core.py:834:store), events (rga/core.py:704:stage, rga/core.py:738:stage), policy_for (rga/core.py:440:_open). The only assignments in the file target self._events, self.lines, self.sealed, self.refuters, self.declared_at, self.refused, self.refused_at, self.power, self.defect_ids, self.defect_authors, self._policies, self.policy, and fields of Line, Sample, Trial. tests/test_rga_invariants.py (R11RGAWritesNoFCDField) snapshots the FCD enforcer's items, store, events and policy across every RGA transition — declare, measure, bound, install, open, sample, trial, replay, seal, a published close and an operator close — and asserts equality. QED

Remark. This is a non-interference lemma, and it is what "extends" means here: the combined machine's FCD projection is the FCD machine, so the FCD proofs hold on it unchanged.

R12 Frozen line

Claim. claims, k, θ , p_min, generator, m_decl, sampling_hash, policy_version are write-once at Open.

Proof. Written at rga/core.py:464:lines from the policy version current at Open and the FCD policy pinned by the item. No other transition assigns them (inspection: Sample appends to samples, Trial to trials, Replay increments replays, Seal and Close set pc, pub, fault). install (rga/core.py:342:install) changes self.policy for later lines and refuses to reuse a version string with different content; policy_for returns the pinned version (rga/core.py:349:policy_for). QED

R13 Bounded

Claim. A line has at most k samples and at most one trial per (refuter, claim, sample) cell; a power record is written at most once per key; a defect model's ledger id-set and author are fixed by its first record.

Proof. Sample refuses when $|\text{samples}| \geq k$ (rga/core.py:715:k). Trial refuses a duplicate cell (rga/core.py:752:cell) and an index outside $[0, |\text{samples}|)$ (rga/core.py:523:sample_index), so a negative index cannot alias a cell. Measure refuses an existing key (rga/core.py:764:power), an id-set differing from the first record (rga/core.py:774:fixed) and an author differing from the first record (rga/core.py:783:author); Bound refuses an existing key (rga/core.py:780:power). The id-set and author are fixed at rga/core.py:405:defect_ids and rga/core.py:406:defect_authors with setdefault. QED

Remark. Replays are unbounded. A harness that replays forever never seals and never lies; that is the Ask-idle shape, not a safety concern.

Replay of the machine

from_events (rga/core.py:941:from_events) rebuilds deterministically: re-driven transitions emit into the live journal so positions advance exactly as they did live, then the journal replaces the regenerated events, refusing on a count mismatch (rga/core.py:1032:rga_events). V1/V3 closes are regenerated by re-driving rga_trial, V4 by rga_replay, V2/V5 by re-driving the failed Seal attempt the close records — which must refuse with the journaled fault (rga/core.py:1015:fault); operator closes are re-driven. The rebuilt line is cross-checked against the rga_open event's recorded (k , θ , $p_{\min}$, claims, refuters), so a supplied policy that differs from the journaled one is an error, never a silent substitution (rga/core.py:986:policy); journal positions are range-checked; nonces are read from rga_sample events and the injected nonce source serves live use afterwards. Round 1 found the first version of this function could not rebuild any journal containing a V2 or V5 close; ReplayTests now drives both.

Calibration proofs

Machine: rga/calibration.py (CalibrationAuthority), composed over Admission. Premise round: eval/reviews/rga-calibration-premise-SYNTHESIS.md; round 1 of this layer (eval/reviews/rga-calibration-r1-SYNTHESIS.md) broke the first version's replay — it trusted the journaled tier, seed, checker standing and exclusion actor — and found a spurious close emission and three statement-versus-code divergences; the repairs are cited below. Writer inspection, used throughout: on the live table a Run enters the ledger only at rga/calibration.py:281:runs; established is set only at rga/calibration.py:308:established; discredited grows only at rga/calibration.py:292:discredited and nothing removes from it; adjudication is written only at rga/calibration.py:321:adjudication; exclusions grow only at rga/calibration.py:326:exclusions. from_events writes the same fields on rebuild and is therefore part of this inspection: it re-checks every filing guard and entailment first — see *Replay of the ledger*. Validity is computed at query time by _check_valid (rga/calibration.py:341:_check_valid), never stored.

C1 Established, never asserted

Claim. A tier-A escape has effect only past kernel-checked preconditions — seal membership, byte-hash equality, pinned checker, kernel-derived seed, refuted verdict, identical replay — none of which is a declaration; a tier-B escape has effect only past a journaled, attributed adjudication. No other path to effect exists, on the live machine or the rebuilt one.

Proof. Every effect (impeachment, charges, corpus membership, demotion arithmetic) quantifies over runs passing `_check_valid`, which requires established (`rga/calibration.py:342:established`). established is written only on a replay whose (verdict, witness) equals the filed pair (`rga/calibration.py:612:witness_hash`); a divergence instead discredits the checker, monotonically, and a discredited or Admission-refused checker fails validity at every later read (`rga/calibration.py:341:_check_valid`). Filing itself is guarded: the line must be sealed and the claim in its seal (`rga/calibration.py:233:_seal_of`, `rga/calibration.py:239:_pinned_on_claim` — structural preconditions: an unknown line or claim cannot be named); the verdict must be the filing kind's (`rga/calibration.py:576:RUN_VERDICTS`); the checker declared and in good standing (`rga/calibration.py:540:refuters`); the kernel hashes the filed bytes itself and refuses unless they are the sealed bytes (`rga/calibration.py:588:artifact_hash`) — single-holder, B7-grade; and for tier A the seed must equal the kernel's own derivation over the seal's hash and the pinned identity (`rga/calibration.py:598:derive_seed`), so the finder authors nothing but a nonce and the run is a counterfactual trial of the instrument the seal already trusted (B1/B2, unchanged). A tier-B escape additionally fails validity until an adjudication (`rga/calibration.py:356:adjudication`), which requires tier B, establishment, a named actor and a reason, once (`rga/calibration.py:596:tier`, `rga/calibration.py:616:actor`). Replay re-derives the tier from the seal, re-runs the seed, standing and audit-checker guards, and requires a diverged replay to be followed by its discredit event, so a journal the live machine refuses is refused on rebuild. QED

Remark. Tier B is where checker claim-fidelity lives, and it is adjudicated — a visible governance act (B13) — never kernel-checked. Audits (survived filings) carry no consequence beyond exposure, so their checkers must be pinned or be a valid escape's checker (`rga/calibration.py:597:pinned`), and adjudication does not gate them.

C2 Charge totality and unit

Claim. The charges standing against a refuter version are exactly the valid wrong-verdict cells derivable from the journal — read, never declared — and at most one per (line, claim, refuter version).

Proof. `charge_cells` (`rga/calibration.py:373:charge_cells`) iterates valid refuted runs and adds, for every refuter pinned to the escaped claim in that line's seal, the cell (line, claim, id, version) into a set (`rga/calibration.py:373:cells`). By R1, each such refuter holds a surviving trial on the designated sample, so the cell names a journaled wrong verdict; by set semantics a second escape on the same cell adds nothing, so witness and checker multiplicity never multiply charges. No transition writes a charge anywhere: the quantity exists only as this query. QED

C3 Impeachment is entailed and checkable

Claim. `impeached(id)` is a pure query entailed by a valid escape; the seal is never rewritten; validity degrades by journal facts, never by discretion.

Proof. `impeached` (`rga/calibration.py:407:impeached`) is a read over runs and `_check_valid`; no field of any Seal is assignable (frozen dataclass, R2) and no calibration transition touches Admission.sealed (C7). The only ways impeachment lifts are the checker's discredit (`rga/calibration.py:292:discredited`) or Admission refusal — both journal facts with their own falsifiers — never an issuer's act. admissible (`rga/calibration.py:424:admissible`) and `check_dependencies` (`rga/calibration.py:436:check_dependencies`) read it; `suspect` (`rga/calibration.py:428:suspect`) folds it over direct dependencies, and transitive coverage is the consumer's fold — a query, never a write. QED

C4 The ratchet

Claim. No policy installs unless: every **ledger** claim's defect model is Measured and covers the class's valid corpus minus journaled exclusions as of the install position; no claim is bounded-only against a nonempty net corpus; and no class that owes coverage is dropped from the policy. The install event carries the coverage primaries, the per-claim model map, the dropped-id diff and any dropped classes. Forgetting is loud.

Proof. `install (rga/calibration.py:473:install)` is the only path that drives `Admission.install` from this authority (`rga/calibration.py:473:install`), and it is enabled only past three guards: every ledger claim's defect model has a kernel-fixed id-set (`rga/calibration.py:644:Measure`) — always satisfiable in advance, since `measure` reads no policy state; a bounded-only claim never has one, which is why the bounded-only guard exists separately (`rga/calibration.py:676:bounded`); that id-set contains the derived defect id (`rga/calibration.py:245:derived_defect_id`) of every corpus entry not excluded (`rga/calibration.py:674:omits`), where the corpus is the valid escapes of the class minus exclusions (`rga/calibration.py:365:corpus`); and a class with a nonempty net corpus cannot vanish from the incoming policy (`rga/calibration.py:657:dropped`) — round 1's attacker found the drop released a whole corpus with no journal trace. Exclusion is itself a guarded, journaled decision requiring a named actor, a reason, and valid escapes of the class (`rga/calibration.py:616:actor`). The install event carries per-class coverage primaries with the per-claim model map, the predecessor-diff of dropped defect ids and the dropped classes (`rga/calibration.py:495:cal_install`, `rga/calibration.py:657:dropped`). QED

Remark. The successor relation quantifies over journaled installs of this authority, not over a D-to-D lineage; a "new unrelated D" faces the identical check. A deployment calling `Admission.install` directly bypasses the ratchet — the interposition boundary, stated where RGA states consumer redirection.

C5 Track record carried

Claim. Every seal issued through this authority is accompanied, in the same step, by a stamp carrying each pinned refuter's primaries as of that position — charged cells, seals participated in, corpus size and exclusions — and the class corpus's finder-provenance split for this seal's generator. Never a rate.

Proof. `seal (rga/calibration.py:449:seal)` calls `_stamp (rga/calibration.py:534:_stamp)` immediately after `Admission.seal` returns, emitting `cal_stamp (rga/calibration.py:543:cal_stamp)` with `track_record` primaries (`rga/calibration.py:448:track_record`) for every pinned refuter and the provenance split (`rga/calibration.py:545:_corpus_provenance`) — corpus entries found by this seal's generator versus independent finders, carried as counts; the finder string gates nothing. No ratio is computed anywhere, and a zero `charged_cells` is absence of filed evidence — the record's documentation says so and nothing in the kernel reads it as power. Replay recomputes both the records and the split and refuses a stamp that does not recompute (`rga/calibration.py:859:recompute`), bounding `seals_participated` to seals at or before this seal's position so what the live stamp saw is what replay recomputes — without that bound every honest journal with two seals was refused, which the round-2 review's boundary test caught. QED

Corollary (mediation is provable). `mediated (rga/calibration.py:412:mediated)` reads the stamp back: a line is mediated exactly when one `cal_stamp` exists for it and its `sealed_at` is the seal's own position. `admissible (rga/calibration.py:424:admissible)` conjoins it, and `check_dependencies` refuses an unmediated dependency. Hence a line sealed by calling `Admission.seal` directly — layer R's own transition, which this authority cannot forbid — is visible as layer IR and never answers `admissible`, which is the property the exact-head review found missing: before this, a bypassed seal and a mediated one were indistinguishable to every consumer. Adding a second stamp, or re-pointing one at another seal, is refused on rebuild; removing the only stamp is not detectable — an unstamped seal is

exactly what a never-mediated line looks like — but it lowers the line to IR and can never raise standing. QED

C6 Demotion is a query with declared consequences

Claim. demoted is a pure function of the valid charge set against the declared budget — it rises as valid escapes accumulate and can fall when a checker's discredit or refusal degrades their validity, which is the fail-closed direction: refusal-shaped facts outrank the count. It blocks new pins at CalOpen unconditionally; it gates CalSeal exactly where the class declared it, closing with a published fault that carries the primaries; it never cascades events and never touches an existing seal.

Proof. demoted (rga/calibration.py:392:demoted) compares the charge count to the class's declared e_max (rga/calibration.py:405:e_max); charges are the valid cells (C2), so the crossing tracks validity exactly — no stored flag exists to go stale in either direction. open refuses a class whose current policy pins a demoted refuter (rga/calibration.py:687:demoted), unconditionally. seal first refuses a line that is not Open (rga/calibration.py:515:seal) — round 1 found the first version emitted a close event for a line that stayed Sealed — then consults _check_seal_demotion (rga/calibration.py:694:_check_seal_demotion), which acts only where the class declared demotion_gate = seal (rga/calibration.py:698:demotion_gate); on a crossing it publishes cal_close with fault E5 and the refuter's primaries (rga/calibration.py:528:cal_close) and closes the line through Admission's own operator close (rga/calibration.py:521:close). No calibration transition iterates lines, so there is no refusal-style cascade; existing seals are untouched (C3, C7). Replay refuses an E5 close whose refuter is not demoted at that point (rga/calibration.py:838:demotion). QED

C7 Non-interference

Claim. The calibration authority writes no Admission or FCD field directly; its only effect on the lower machines is through their own guarded transitions. Every R- and I-theorem holds on the combined trace.

Proof. Inspection: every reference to self.adm and self.adm.fcd outside the four wrapped drives is a read (sealed, refuters, refused, defect_ids, policy, lines, items, _policies); the wrappers drive exactly Admission.install (rga/calibration.py:473:install), Admission.open (rga/calibration.py:503:open), Admission.seal (rga/calibration.py:515:seal) and Admission.close (rga/calibration.py:521:close), each a guarded public transition whose own theorems quantify over all callers. The only assignments in the file target the private journal storage self._events, self.runs, self.discredited, self.exclusions and fields of Run. tests/test_rga_calibration.py (C7NonInterference) snapshots FCD and Admission state — including the Admission event count — across every ledger transition and asserts equality. QED

Replay of the ledger

from_events (rga/calibration.py:731:from_events) rebuilds over an already-rebuilt Admission, re-checking before every write what the live table checked: the tier is re-derived from the seal and a journaled tier that disagrees is refused (rga/calibration.py:738:tier); the seed, checker-standing, verdict and audit-checker guards are re-run; a diverged replay must be immediately followed by its discredit event and a discredit must be preceded by its diverged replay (rga/calibration.py:750:pending_discredit) — round 1 demonstrated a deleted discredit silently un-discrediting; exclusions and adjudications are re-guarded; installs re-run the ratchet; stamps and E5 closes must recompute from the rebuilt ledger. Two residues, stated and executable (tests/test_rga_calibration.py, ExactHeadReviewRepairs). **Deletion.** Replay re-checks every event it is given; it cannot check an event it is not given. Removing an event that a later event recomputes against

is refused — a deleted run breaks the run-index sequence, a deleted discredit breaks its adjacency, a deleted or re-pointed stamp fails its binding, and every machine now compares its re-driven transitions to the journal field by field, so a deletion that changes any regenerated derived field is refused too. What survives replay alone is any coherent alternate history: root transition inputs rewritten consistently, or deletion that leaves a *coherent* shorter history: the journal's tail, the highest-indexed run, an adjudication no later event reads, or the whole diverged-replay/refuse/close group of a taint. That is the direction which raises standing — a dropped escape un-impeaches its line, a dropped refusal un-taints every seal that relied on it — and it is not detectable from the journal alone. One appended event has the same character: a stamp forged for the most recent unmediated seal contradicts nothing earlier and is accepted, where one forged for an earlier seal breaks stamp order and is refused. Replay alone cannot close any of this. v0.5's authenticated publication path does: immutable authority-owned event sequences are sealed into three namespace-scoped ordered heads, each successor carries the new event digests so the registry can recompute the cumulative chain from its anchored prefix, the heads are accepted atomically, and the exact heads plus derived I/R/C predicates are bound into one signed receipt. Verification rejects an old, coherently shortened, or longer-forked sequence. This is conditional on callers issuing and verifying the receipt; registry plus signer/key compromise or rollback, and mutation outside the single-writer authority model, remain assumptions. **Final registry.** The rebuilt Admission registry is the final one, so checker membership and Measure records are checked against a superset of the live registry — a journal the live machine accepted always replays, and most tampers are refused, but an install the live machine refused for a missing Measure record could replay if a later record filled it. Validity's Admission-refusal clause is read *as of the position each event records* — every event whose replay guard reads Admission state carries the position it was written at (as_of on runs' stamps, E5 closes, exclusions and installs) — because reading the final refused set made an ordinary post-seal audit divergence refuse honest journals that had already stamped, excluded or E5-closed. The round-2 review demonstrated four such refusals; each is now a regression test.

Calibration: what is not proved

§8.3's "Not theorems" in full, kept adjacent: corpus representativeness (B6 relocated, a selected and steerable subsample; flooding with true entries steers every aggregate); search intensity (zero charges is unreadable between sound and unsearched except through journaled audits, whose reference is the corpus, never undiscovered defects); audit-campaign scope honesty (B1 one level up); the residual disincentive to reveal defects only one's own instruments miss; tier-B checker claim-fidelity (adjudicated, visible, never kernel-checked); statistical calibration (no bounded escape rate, no convergence of D to the generator's defects; Gibbs & Candès 2021 is the shape such a theorem has and why it does not fit an adversarially censored channel); operator honesty (B13 — budgets, gates, exclusions and adjudications are governance made visible, not defeated); byte availability (B12 — loss censors the escape stream one-sidedly and can never manufacture admission); the replay registry residue above; and the interposition boundary — a deployment that bypasses this authority for open, seal or install has RGA's guarantees and none of these.

What is not proved

Written in PREMISE.md §7 before any of the above; extended by round 1; kept here in that order.

- **Relevance of D.** Kill-rate on a defect model is power against the generator's actual defects only under the coupling-effect hypothesis (B6). Just et al. (2014) found mutants coupled to roughly three quarters of real faults under common operators; for a generator whose characteristic defects are semantic, the uncoupled remainder may be what matters. The seal carries D so the reader makes that judgement; the kernel does not.

- **Refuter over-fit.** A refuter strong on D and blind outside $\text{span}(D)$ carries its full measured power. R3 stops the refuter being shaped by the artifact and D being authored by an interested party; nothing stops a refuter author who knows D from optimising against it.
- **ϵ honesty.** For a bounded refuter, that its failure-mass threshold under its own sampling distribution is ϵ , and that it draws N inputs, is a report (B1). The kernel computes $1-(1-\epsilon)^N$ and carries (ϵ, N) in the seal so the reader can judge the alternative; it does not verify it. With $N = 1$ the carried figure equals the declaration.
- **Ledger honesty and replay.** A calibration ledger is attested, not replayed by the kernel; R6 exercises trial replay only. A third party can replay a ledger because D is content-addressed and the refuter deterministic, but no guard requires that it happened, and no event records who performed any replay.
- **Author identity (B11).** Separation of duty holds of declared author strings compared byte for byte, not of a closed namespace as FCD's specialist ids are.
- **Claim fidelity.** A correct artifact for the wrong claim seals. Every interesting intent has only an attackable shadow; the seal's residual field names what was not attacked, and its `check_stage` disposition says only that a non-author reviewer ran (I6), not what they concluded.
- **Harness honesty (B1).** A trial verdict is a report. A forged survived satisfies the abstract guards, as a forged `m_exec` satisfies FCD's. What determinism adds is that a third party re-running the refuter on the journaled bytes and seed catches the forgery as a replay divergence (R5) — after the fact, unless the harness is required to replay before Seal, which R6 requires once.
- **Determinism beyond replayed inputs (B2).** Replay is a falsifier. A refuter deterministic on this host and not another, or on these inputs and not others, passes. Behaviour under the harness equal to behaviour in deployment is not proved. The static half — refusing a refuter written outside the runtime's deterministic subset — is the harness's obligation, not a kernel check.
- **Halting (B3).** The fuel bound is enforced by the harness and reported as inconclusive. The kernel proves only that inconclusive never counts.
- **Sample independence (B4).** k samples served from a whole-response cache are one draw presented k times; concordance is then k/k and measures nothing. A repeated run id on the FCD receipt (I17) makes this visible; it is not a guard here.
- **Refuter independence.** Combined power is measured only as a union over one shared D. Across defect models the seal carries `max`, labelled, which is a sound lower bound and not an estimate.
- **Generator exposure.** A generator that has seen a fixed refuter in training or in context can special-case it. The seed (R10) defeats this for seeded refuters; `excluded(c)` (R3) removes fixed corpora from the package; pretraining is outside both, as A10 is outside FCD. A retry opened after a refutation with the trace visible to the generator is adaptive and unlabelled beyond `excluded(c)`.
- **Consumer redirection.** That any DAG edge or promotion predicate reads `S_R` (via `check_dependencies / admissible`) rather than `S` is a deployment obligation. The reference server now discharges it: promotion and per-line state are gated on the calibration authority's `admissible()` (`tests/test_server_admissibility.py`). The FCD depends_on edge itself remains layer-I, and every consumer outside this repository remains an obligation.
- **Nonce unpredictability (B9).** An injected source; journaled; assumed.

- **Hash (B7).** A second-preimage fails toward acceptance here, unlike FCD's norm collisions. Assumed, not fail-closed.
- **Quality.** Survival is not correctness. Nothing in R1–R13 says the artifact is right.
- **Body provenance.** Not closed. Moved: the seal binds the trials to bytes the kernel hashed (R9) and forces slot assignment before later artifacts exist (R3), but which stage produced which bytes, and the link from m_exec to their production, still rest on reports (B1, I1).
- **Liveness.** A harness that never replays, a refuter that is never measured, a check stage that never runs: each strands a line Open. Nothing here makes Seal reachable.

Executable form

tests/test_rga_invariants.py drives the same transition guards with positive and negative traces, including the three round-1 defect traces; tests/test_rga_calibration.py does the same for C1–C7 and E1–E9, with its own per-guard deletion table over CalibrationAuthority (including the stamp effect). tests/test_rga_mutation.py replaces each guard method with a no-op — one at a time — and shows the corresponding forbidden state is reached, and fails if any guard method in rga/core.py has no such scenario or any fault V1–V15 no scenario. tests/test_rga_citations.py opens every path:line:symbol citation above and fails if the line does not contain the symbol. A failing test is a counterexample to the corresponding claim on the Python machine. That is a proof about the core, not about a harness that lies.

Part III — Identity

Fail-closed class dispatch binds each class to an allowed specialist, records declared and executed identity independently, and refuses fallback edges. The report, invariants and proofs below define the identity layer inherited by Parts I and II.

Fail-closed class dispatch

Class-admitted work, data-plane bind. This machine has no leftover-hop edge.

Roque Briceño

Version 0.8.0. 30 August 2026. Licensed under CC BY 4.0.

Abstract

A specialized model fleet is a checkable claim only if a work item of class c cannot complete a stage on a specialist or model outside the policy for c .

Fail-closed class dispatch is that check. A work item has a class and a frozen body. A policy gives each class an allow set $\pi(c)$ and a deny set $\delta(c)$. Each specialist a is bound to one API model identity $\phi(a)$. A stage may run only after a well-formed admit of some $a \in \pi(c) \setminus \delta(c)$ and only with executed model $\text{norm}(\phi(a))$. If that bind cannot be served, the stage fails closed and the failure is published. Retry, if any, is the same item, same class, a specialist not yet tried. It does not search a leftover fallback list.

This is Clark–Wilson integrity applied to LLM binds, not a new access-control algebra. Allow/deny, fail-safe defaults, and dual control are old. The added obligations are: ϕ is the identity Observe recorded from the inference client, leftover fallback is not an edge in this table, and the event contract is the witness set. An accidental hop is in scope. An adversarial worker that forges Observe is not.

Safety invariants hold on the abstract machine under stated enforcer obligations. A second envelope pins project state, accepted memory, instructions, agent, exact model, context policy, cache identity and steering at each gate attempt. This extends the same fail-closed rule from model identity to context identity without rebuilding the worker's tools or session system. Item liveness does not hold. There is no quality theorem and no empirical table in this draft.

1. Problem

Routing, cascades, and mixture-of-agents optimize score or spend (Chen et al., 2023; Ong et al., 2024; Wang et al., 2024). Mixture-of-experts is a learned gate inside one network (Jacobs et al., 1991; Shazeer et al., 2017). Orchestration says who calls whom. None of them forbid a hop to an unbound model when the bound provider is down. None of them treat a model that refuses a class as illegal rather than low-scoring.

The sentence “we run specialists” is then unfalsifiable. The control plane can name one model while the data plane calls another. A 403 can look like work still in progress. A reviewer can be the author.

The required property is class integrity: a pass record lies in $\pi(c) \setminus \delta(c)$ and uses $\phi(a)$. The process is the smallest transition system that keeps that property and refuses the leftover hop.

2. Objects

A **work item** has class c , charge, frozen body hash, author set, required-stage list $\text{Required}(c)$, and status $\text{open}|\text{failed}|\text{accepted}$.

$\pi(c)$ and $\delta(c)$ are disjoint. On a check stage the effective allow set is $\pi_chk(c, \text{authors}) = \pi(c) \setminus \text{authors}$.

$\phi: A \rightarrow M$ maps a specialist to an API identity. Display strings are compared after norm.

$u(m)=0$ iff bind time returns 401, 403, 404, 429, exhausted, or not_found.

tried is the set of specialists already admitted on this stage.

A stage is **well-formed** only if a control event names class, specialist, declared model, and body hash. Prose that mentions a name is not a stage.

Fail closed means: publish a fail_closed decision; next is ask, retry in $\pi(c) \setminus \delta(c) \setminus \text{tried}$, or stop.

An **accepted artifact** exists only when every stage in $\text{Required}(c)$ has passed. The **store** accepts only accepted artifacts.

3. Process

1. Open a well-formed work item.
2. Admit $a \in \pi^*(c) \setminus \delta(c) \setminus \text{tried}$. If none, fail closed.
3. Bind. Writes $m_decl = \phi(a)$. If $u(\phi(a))=0$, fail closed. Does not write m_exec .
4. Observe. Copies m_exec from the provider call (A1).
5. Pass only if $\text{norm}(m_exec) = \text{norm}(m_decl)$; else PassRefuse (F1).
6. When $\text{Required}(c)$ is covered, Accept into the store.

A death watchdog outside the worker records death while Running (A2, A9). Close publishes.

4. What holds

Proofs: PROOFS.md. Machine: fcd/core.py. Checks: tests/.

Bind writes declared only. Observe writes executed. I1 is then non-vacuous. Under A0–A9, I1–I6, I8, I9 are inductive. I7 bounds Admit count. Under A10–A13, I10–I17 prove work-snapshot pinning, package/receipt binding, steering scope, attempt-local cache identity and accepted-only memory promotion. Ask may idle.

The leftover-hop picture is an illustration, not a corollary.

5. Context, memory and cache envelope

A project has an accepted snapshot P_v and accepted semantic memory K_v . A work item pins (P_v, K_v) at Open. Its raw transcripts, candidate reasoning and failed attempts remain evidence; they are not project memory.

A gate attempt has a monotonic counter and unpredictable nonce. At Admit it freezes

X_g,a=(attempt,P_v,K_v,W_r,G_r,A_r,specialist,M_r,I_h,C_p,T_h,Q_c,S_0,channel).

M_r is the exact provider/API model reference. I_h is the layered instruction hash. C_p selects context categories. T_h binds tool authority. Q_c is the FCD attempt-local cache policy. S_0 is steering already present at Admit. A base-envelope change closes the attempt and creates a new nonce.

FCD builds canonical context bytes B_g,a from include minus exclude; exclude wins. The adapter independently hashes the bytes it submits. Pass requires the observed package hash, attempt/nonce, executor/run, latest steering continuation and executed-model receipt to match the current envelope. A previous-attempt receipt cannot authorize a new attempt.

The package modes are project_shared, fresh_scoped, fresh_blind, and contract_only. A fresh_blind review excludes author transcript, reasoning, prior verdict and unaccepted memory; it forbids executor continuity. Continuing or forking an opaque executor session is a declared adapter capability, not an FCD-owned session store.

Live steering is an ordered stream outside the frozen base envelope. Each event addresses project, work, gate, stage, artifact, evidence or failure scope. Pass requires acknowledgement of the latest continuation hash. Steering cannot write accepted state or sibling work.

FCD cache stays attempt-local and clears at Admit/Close. Provider or executor prefix/session reuse is reported telemetry, not Pass authority. This separates semantic context from an optimization.

Only Accept may promote a reviewed knowledge delta. Promotion is a serialized compare-and-swap on the exact project/memory head. If another accepted item advances the head, stale active work requires signed impact review. continue_pinned binds that review to one exact new head; another advance invalidates it. refresh does not authorize the stale final gate.

These are process theorems about manifests, receipts and state transitions. They do not prove physical prompt isolation, hidden executor residue, model quality or impact-review correctness.

6. Faults

Each fault is a forbidden transition, not a metaphor.

Id	Forbidden step
F1	Pass with $\text{norm}(m_{\text{exec}}) \neq \text{norm}(\phi(a))$
F2	Two specialists, one runtime instance (needs an instance field to measure)
F3	After $u=0$, another call without fail-closed, or a call outside unused allow
F4	Running exit with no published close
F5	$\phi(a)$ not an API identity
F6	Pass with $a \in \delta(c)$
F7	Check admit with $a \in \text{authors}$
F8	Run without a well-formed stage
F9	Stop in chat with status not accepted

Id	Forbidden step
F10	Same id, class changes; or retry of $a \in \text{tried}$

Weight-sharing across specialists is extra config. It is not in I6.

7. Measurement

A **named cut** is $[t_0, t_1]$ and window W taken from the class reply-time distribution (exclude $ts > t_1 - W$). Evaluate π, δ, ϕ **as-of event ts**. A pinned policy version is only a default when no as-of row exists.

- Misbind: first observed bind attempt per stage where $\text{norm}(m_exec) \neq \text{norm}(\phi(a))$. This is an observation-gap / A1-breach rate, not F1 (F1 requires Pass). Never publish without silent-fail.
- Silent fail: well-formed stages with no decide/accept within W , plus work items that open and never emit a stage. Fail-closed counts as published.
- Bleed: assigned a outside $\pi^*(c)$ as-of ts (use π_chk on check stages). Pair with silent-fail.
- Time-to-stage: duration from well-formed stage to decide or accept. Right-censored survival, paired with 1- silent-fail. Not a mean.

Zeros on misbind, bleed, and silent-fail are a proof that those faults did not fire only if stage is write-ahead and call/decide are total. Otherwise they are estimates biased clean. Schema: `../metrics/SCHEMA.md`. No numbers here.

8. Reference implementation

The repository includes the acceptance kernel, context authority, immutable project/evidence atlas, language-neutral schemas, an execution-adapter boundary, and a project/work/artifact cockpit. The cockpit loads and verifies a local/GitHub project before intake. Every work line stays selectable under its capability. A gate shows agent instructions and authority, exact provider/API model, context mode and exact-route readiness before Admit; an unavailable route cannot start. After Admit, the same tray locks to package/model/steering receipts. Questions are anchored to their node. Drift review and multi-scope steering remain inside the work surface.

The implementation does not widen the theorem. An execution adapter preserves an existing worker's tools, session implementation and provider cache, but cannot Pass, Accept or promote memory. A skin is a read-only projection and cannot hide model/context/receipt state or write policy, journal, or store. The artifact iframe is sandboxed. fcd remains the only writer of accepted state.

Replay, policy-version pinning, and the DAG gate make a project many dependent lines over time: a line opens only on accepted dependencies and finishes under the policy version pinned at Open.

9. Related work

Clark and Wilson (1987) already have constrained data items, transformation procedures, a certification relation, integrity verification, and mandatory separation of duty. A work item is a CDI. A bound specialist is a TP. Accept is a validated write.

Saltzer and Schroeder (1975) name fail-safe defaults. Deny-override is standard MAC/RBAC. Thomas and Sandhu (1997) bind permission to a task instance. F1/F2/F5 are TOCTOU / control-plane versus data-plane divergence. F7 is also LLM-as-judge self-preference.

MoE, MoA, FrugalGPT, RouteLLM, and MoMA optimize a different objective. Orchestrator-worker (Anthropic, 2024) is a topology. Topology is not the process.

10. Limits

No dataset. No quality theorem. No item liveness. I1–I6, I8 and I9 are proved on the Observe machine; I7 is a bound. I10–I17 prove FCD-owned envelope, manifest, receipt, cache and promotion transitions. A1/A2 and A10–A12 are observation/trust assumptions. A package receipt proves submitted-byte equality under an honest adapter, not absence of hidden executor context. Provider fidelity, cache semantic neutrality and impact-review correctness are not proved. The hop remark is an illustration.

Site names do not belong in this voice.

11. References

Chen, L., Zaharia, M., and Zou, J. FrugalGPT. arXiv:2305.05176, 2023.

Clark, D. D., and Wilson, D. R. A comparison of commercial and military computer security policies. IEEE Symposium on Security and Privacy, 1987.

Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. Neural Computation, 1991.

Ong, I. et al. RouteLLM. 2024.

Saltzer, J. H., and Schroeder, M. D. The protection of information in computer systems. Proceedings of the IEEE, 1975.

Shazeer, N. et al. The sparsely-gated mixture-of-experts layer. ICLR, 2017.

Thomas, R. K., and Sandhu, R. S. Task-based authorization controls (TBAC). 1997.

Wang, J. et al. Mixture-of-Agents Enhances Large Language Model Capabilities. arXiv:2406.04692, 2024.

Towards Generalized Routing: MoMA. arXiv:2509.07571, 2025.

Anthropic. How we built our multi-agent research system. 2024.

Figure 1. Stage automaton (Observe machine)

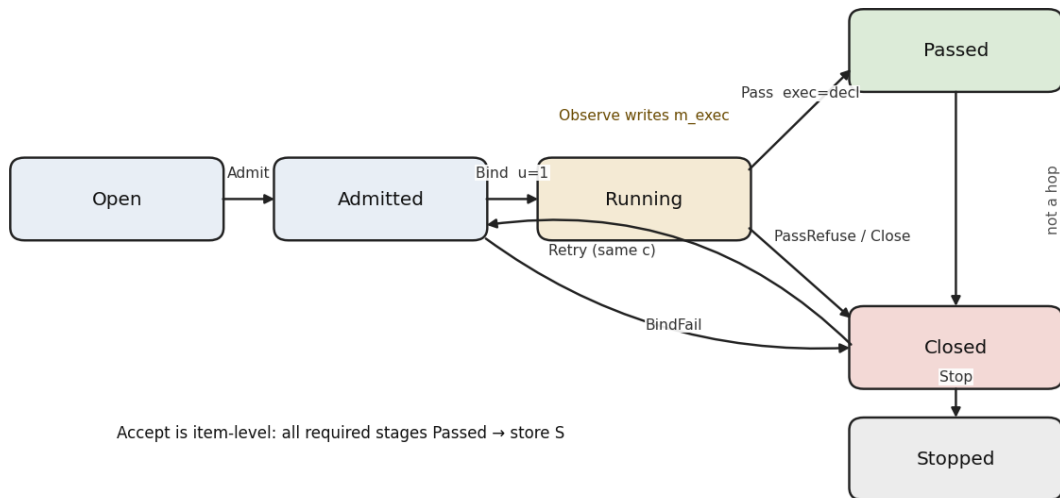
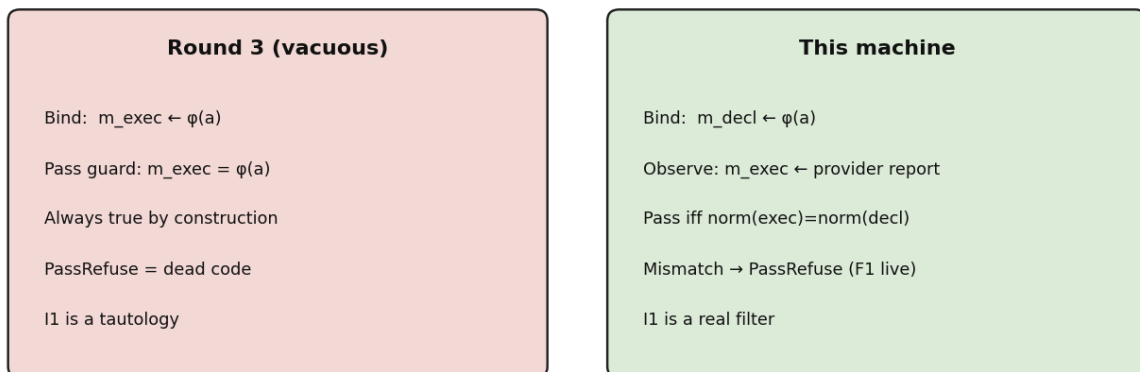


Figure 2. Why I1 is non-vacuous: two writers



Invariants

Roque Briceño. Safety only. Companion to DRAFT.md and PROOFS.md.

Round 3: Bind wrote $m_{\text{exec}} \leftarrow \phi(a)$, so PassRefuse was dead. Bind now writes **declared** only. **Observe** writes **executed** from the provider call.

0. Assumptions

A0. $\pi(c) \cap \delta(c) = \emptyset$. A and $\pi(c)$ finite.

A1. **Observation totality**. Every data-plane call while $pc = \text{Running}$ produces an Observe. A1 does not assert equality with $\phi(a)$.

A2. **Death observability.** A watchdog **outside** the worker process records death while $pc=Running$. A2 does not publish fail-closed. Close does.

A3. $u(m)=0$ iff bind returns 401, 403, 404, 429, exhausted, or not_found.

A4. Required(c) finite ordered. Stages of one item sequential. authors snapshotted at Admit.

A5. norm to API identities. Equality is $norm(x)=norm(y)$.

A6. Check admit: $\pi_chk(c, authors)=\pi(c)\backslash authors$. Write admit: $\pi(c)$.

A7. Retry only a $\notin$ tried.

A8. Store S written only by Accept: $S \leftarrow S \cup \{id\}$.

A9. Watchdog liveness for A2: if the worker pid is gone and $pc=Running$, the watchdog emits death_observed in finite time. Partition can delay this; F4 is then an estimate.

A10. **Adapter attestation honesty.** The adapter computes the observed package hash from bytes it actually submits; reports executor/run, steering acknowledgement and executed model truthfully; and does not inject hidden context against its declared capability. This is a physical-boundary assumption, not an FCD theorem.

A11. Hash and attempt-nonce collision resistance. A collision here would fail toward *acceptance* — a stale receipt could match a new identity (I17) or a differing attempt could hit the cache (I16) — so A11 is an assumption that collisions do not occur, not a fail-closed property.

A12. A signed impact review correctly classifies non-interference for continue_pinned. If incorrect, promotion safety relative to the newer project is not proved.

A13. Accept/memory promotion is globally serialized by compare-and-swap on one exact (P_v,K_v) head.

1. State

Per stage: $pc \in \{Open, Admitted, Running, Passed, Closed, Stopped\}$ c, body write-once. a or none. m_decl (Bind) or none. m_exec (Observe) or none. tried $\subseteq A$. pub $\in \{0,1\}$.

Item: id, authors, required, pointer, status $\in \{open, failed, accepted\}$. Store S.

Project context: accepted project version P_v, accepted memory version K_v, policy version.

Per work item: Open-pinned (P_v,K_v), contract revision.

Per gate attempt: monotonic counter, nonce, immutable base envelope X, expected canonical package hash, package categories, initial steering hash S0, latest continuation hash/sequence, adapter receipt or none, FCD cache identity, state Running|Passed|Closed.

Project memory changes only by accepted knowledge delta under CAS.

π^* is π_chk on check stages else π .

2. Transitions

Open — *Guard*: well_formed; id fresh · *Effect*: c, body; tried={}; pc=Open; status=open

Admit(a) — *Guard*: pc∈{Open,Closed}; a ∈ $\pi^* \setminus \delta \setminus \text{tried}$ · *Effect*: a; tried $\cup = \{a\}$; m_decl=m_exec=none; pc=Admitted

NoAdmit — *Guard*: pc∈{Open,Closed}; allow remainder empty · *Effect*: pc=Closed; pub=1; status=failed

Bind — *Guard*: Admitted $\wedge u(\phi(a))=1$ · *Effect*: m_decl $\leftarrow \phi(a)$; pc=Running

BindFail — *Guard*: Admitted $\wedge u(\phi(a))=0$ · *Effect*: pc=Closed; pub=1

Observe(m) — *Guard*: Running · *Effect*: m_exec $\leftarrow m$

Pass — *Guard*: Running $\wedge m_exec \neq \text{none} \wedge \text{norm}(m_exec) = \text{norm}(m_decl)$ · *Effect*: pc=Passed; write-stage: authors $\cup = \{a\}$

PassRefuse — *Guard*: Running $\wedge m_exec \neq \text{none} \wedge \text{norm}(m_exec) \neq \text{norm}(m_decl)$ · *Effect*: pc=Closed; pub=1; F1

Close — *Guard*: Running $\wedge (\text{refuse} \vee \text{death_observed})$ · *Effect*: pc=Closed; pub=1

Retry — *Guard*: Closed $\wedge$ remainder nonempty · *Effect*: back toward Admit; c unchanged

Ask — *Guard*: Closed · *Effect*: idle

Stop — *Guard*: Closed · *Effect*: pc=Stopped; status may be failed

Accept — *Guard*: status=open and all required Passed · *Effect*: status=accepted; S $\cup = \{\text{id}\}$

Bind does **not** write m_exec.

Context-envelope transitions:

WorkOpen — *Guard*: verified project selected · *Effect*: pin current (P_v,K_v) and contract revision

EnvelopeAdmit — *Guard*: gate editable; context policy valid; declared executor capability available · *Effect*: increment attempt; fresh nonce; freeze X and S0; state Running

BuildPackage — *Guard*: attempt Running · *Effect*: canonical bytes from include\exclude; write expected hash

AppendSteering — *Guard*: Running; scope belongs to work and is not accepted state · *Effect*: append ordered event; advance continuation hash; clear acknowledgement

Receipt — *Guard*: current attempt/nonce; package hash, model, executor and latest continuation match · *Effect*: record valid receipt/acknowledgement

ReceiptRefuse — *Guard*: any receipt field stale/mismatched · *Effect*: Close; no Pass/Accept

GatePass — *Guard*: valid current receipt and class-stage Pass guard · *Effect*: state Passed

ImpactReview — *Guard*: work pin behind current head · *Effect*: bind signed classification/decision to that exact current head

Promote — *Guard*: item accepted; CAS head matches; no drift or valid continue_pinned/owner review · *Effect*: advance project/memory once; append accepted knowledge delta

PromoteRefuse — *Guard*: CAS mismatch; stale/missing review; refresh decision · *Effect*: no project/memory write

fresh_blind also guards continuity=fresh and manifest categories disjoint from excluded author context. FCD cache is keyed to the current attempt, envelope and continuation; Admit/Close clears it. Executor cache/session reuse is telemetry outside this transition table.

3. Theorem statements

Proofs in PROOFS.md. Executable checks in tests/test_invariants.py and tests/test_context_envelope.py.

- **I1** $pc=Passed \Rightarrow \text{norm}(m_exec)=\text{norm}(m_decl)=\text{norm}(\phi(a))$
- **I2** $pc \in \{Running, Passed\} \Rightarrow a \in \pi^*(c) \setminus \delta(c)$ (Admit-time snapshot)
- **I3** no Pass with $\text{norm}(m_exec) \notin \{\text{norm}(\phi(x)) \mid x \in \pi^* \setminus \delta\}$
- **I4** c and body constant after Open
- **I5** $\text{status}=accepted \Rightarrow$ every required stage Passed
- **I6** check Admit $\Rightarrow a \notin \text{authors}$
- **I7** $|\pi^* \setminus \delta|$ Admits per stage (not termination)
- **I8** $id \in S \Rightarrow \text{status}=accepted$
- **I9** Retry does not write c
- **I10** work (P_v, K_v) is write-once at Open; attempt base envelope, nonce and $S0$ are immutable until Close
- **I11** Pass requires adapter-observed package hash equal to FCD expected hash for the current attempt/nonce
- **I12** fresh_blind manifest categories are disjoint from defined excluded author context and continuity is fresh
- **I13** project memory changes only through serialized Accept/CAS; failed CAS cannot promote
- **I14** a work pin cannot change silently; refresh is a new work revision/attempt and envelope
- **I15** steering cannot write outside scope or accepted state; Pass requires latest continuation acknowledgement

- **I16** an FCD cache hit implies full current attempt/envelope/continuation identity equality; otherwise miss and clear

- **I17** package, executor/run, steering and executed-model receipts all bind to the current attempt/nonce; prior-attempt receipts fail closed

Not theorems: quality, item liveness, F2 without instance field, hop-as-safety, physical prompt isolation, executor/provider cache neutrality, adapter honesty, or impact-review correctness.

4. Illustration (not a corollary)

A selector that may pick models outside $\phi(\pi \setminus \delta)$ can hop when every allowed bind is down. This table has no such edge. A selector confined to $\pi \setminus \delta$ also fail-closes. A cost minimizer fail-closes. Do not cite this as a safety theorem.

5. Rates

Cut: $[t_0, t_1]$. W = class p95 of completed stage durations in the previous cut, or 12 minutes if $n < 30$. Exclude $t_s > t_1 - W$. Policy as-of t_s .

Silent-fail includes: well-formed stage with no decide/accept in W , **and** open with no stage. Bleed uses π^* . Misbind = first Observe per stage with $\text{norm}(m_{\text{exec}}) \neq \text{norm}(m_{\text{decl}})$; always with silent-fail. Time-to-stage = survival to decide/accept, paired with 1–silent-fail.

Proofs

Roque Briceño. Inductive proofs of I1–I17 on the machines in INVARIANTS.md. No quality. No item liveness.

Notation. A **trace** is a finite sequence of transitions from the initial state (pc unset, $S=\emptyset$, no item). Each step applies exactly one enabled row. π^* at a step is the Admit-time snapshot (A4, A6).

I1 Bind integrity

Claim. $pc=Passed \Rightarrow \text{norm}(m_exec)=\text{norm}(m_decl)=\text{norm}(\phi(a))$.

Proof. Induction on trace length.

Base. Initial: $pc \neq Passed$. Holds.

Inductive step. The only transition that sets $pc=Passed$ is Pass. Pass is enabled only if $m_exec \neq \text{none}$ and $\text{norm}(m_exec)=\text{norm}(m_decl)$. m_decl is written only by Bind, which sets $m_decl=\phi(a)$. No transition after Bind overwrites m_decl until the next Admit, which also clears m_exec and leaves Passed. Hence at Pass, $\text{norm}(m_exec)=\text{norm}(m_decl)=\text{norm}(\phi(a))$.

Observe may set $m_exec \neq \phi(a)$. Then Pass is disabled and PassRefuse is enabled. That path never reaches Passed. QED

Remark. If Bind wrote m_exec , the Pass guard would be tautological. Observe is what makes I1 non-vacuous.

I2 Class admission

Claim. $pc \in \{\text{Running}, \text{Passed}\} \Rightarrow a \in \pi^*(c) \setminus \delta(c)$.

Proof. a is written only by Admit. Admit requires $a \in \pi \setminus \delta \setminus \text{tried}$. *Bind, Observe, Pass do not write a . A4: stages sequential*, so π is the snapshot at that Admit. QED

I3 No unbound hop

Claim. No state with $pc=Passed$ has $\text{norm}(m_exec) \notin \{\text{norm}(\phi(x)) \mid x \in \pi^*(c) \setminus \delta(c)\}$.

Proof. I1 gives $\text{norm}(m_exec)=\text{norm}(\phi(a))$. I2 gives $a \in \pi^* \setminus \delta$. QED

I4 Frozen class and body

Claim. After Open, c and body are constant on that item.

Proof. Inspection: no other row writes c or body . A class change is a new id by definition. QED

I5 Accept coverage

Claim. $\text{status}=\text{accepted} \Rightarrow$ every $s \in \text{Required}(c)$ has $pc=Passed$.

Proof. Accept is the only writer of $\text{status}=\text{accepted}$ and is enabled only while $\text{status}=\text{open}$ and every required stage has Passed. The status guard also makes the row one-shot: an accepted item cannot emit a second Accept. QED

I6 Dual control

Claim. On a check-stage Admit, $a \notin \text{authors}$.

Proof. A6: $\pi_chk=\pi(c)\backslash\text{authors}$. Admit requires $a \in \pi^*\backslash\delta\backslash\text{tried}$. A4: authors snapshotted at Admit. QED

I7 Bounded admits

Claim. A stage enables Admit at most $|\pi^*(c)\backslash\delta(c)|$ times.

Proof. A0: that set is finite. A7: Admit requires $a \notin \text{tried}$ and adds a to tried . Each Admit strictly decreases the remainder. Ask may idle; the bound is on Admit count, not on reaching Passed. QED

I8 Store only accepted

Claim. $\text{id} \in S \Rightarrow \text{status}=\text{accepted}$.

Proof. A8: Accept is the only writer of S . Accept sets $\text{status}=\text{accepted}$ in the same step. status is otherwise written only by Open (fresh id) and NoAdmit; NoAdmit requires $p \in \{\text{Open}, \text{Closed}\}$, and once Accept has fired every stage is Passed, so no writer of status is enabled on an accepted item and the implication holds of every later state. QED

I9 Retry preserves class

Claim. Retry does not change c .

Proof. Retry's effect list does not write c . I4. QED

I10 Work and envelope pinning

Claim. A work item's (P_v, K_v) is write-once at Open. Within one gate attempt, the counter/nonce, base-envelope hash and $S0$ remain constant until Close.

Proof. WorkOpen is the only transition that writes the work pin. EnvelopeAdmit reads that pin, creates a fresh counter/nonce, computes X , and freezes $S0$. BuildPackage, Receipt, AppendSteering and GatePass do not write those fields. AppendSteering writes a separate continuation chain. A base change requires Close and a new EnvelopeAdmit. QED

I11 Package/receipt compliance

Claim. GatePass implies the adapter-observed package hash equals the FCD expected hash for the current attempt/nonce.

Proof. BuildPackage deterministically writes the expected hash. Receipt is the only transition that writes a valid package acknowledgement and guards equality plus current attempt/nonce. GatePass requires that acknowledgement. ReceiptRefuse closes every mismatch. This proves receipt-level

equality under A10; it does not prove physical model input without A10. QED

I12 Fresh-blind manifest exclusion

Claim. For `fresh_blind`, manifest categories are disjoint from the defined excluded author-context set and no continuity hint is admitted.

Proof. Canonical package categories are `include\exclude`; `exclude` wins. `EnvelopeAdmit` rejects `fresh_blind` unless continuity is fresh. `BuildPackage` cannot add a category outside the effective set. A10 is still required to rule out hidden executor prefix/session residue. QED

I13 Accepted-only serialized promotion

Claim. Project memory changes only by accepted knowledge delta under one successful CAS. A failed CAS cannot promote.

Proof. `Promote` is the sole memory writer. Its guard requires accepted item status and expected head equal to the current head. A13 serializes competing transitions, so at most one transition from one base succeeds. The winner advances the head; every loser observes a mismatch and takes `PromoteRefuse`, which has no write. QED

I14 No silent context drift

Claim. A work item cannot silently change its Open-pinned (`P_v`, `K_v`).

Proof. I10 makes the pin write-once. `ImpactReview` records a decision against a newer head but does not rewrite the pin. `refresh` closes the old attempt and creates a new work revision/envelope. `continue_pinned` rebinds only the promotion CAS expectation to the exact reviewed head; another head advance invalidates the review. QED

I15 Steering scope and acknowledgement

Claim. Steering cannot write outside its declared scope or accepted state, and `GatePass` requires acknowledgement of the latest continuation hash.

Proof. `AppendSteering` guards scope membership in the same work item and rejects accepted targets. Its only write is an append plus chained continuation hash. Each append clears acknowledgement. `Receipt` records acknowledgement only for the latest hash. `GatePass` requires current acknowledgement. QED

I16 FCD cache identity

Claim. An FCD cache hit implies equality of the current attempt-local identity, including nonce, envelope, context mode, S0 and latest continuation hash.

Proof. The FCD key is the hash of those fields. A11 assumes collisions do not occur — a colliding key here would produce a spurious *hit*, so this proof uses A11 in the fail-open direction and holds only under it. `Admit/Close` clear the cache. A differing field either changes the key or crosses a clear boundary, producing a miss. `Executor/provider` cache telemetry is not an FCD hit and has no `Pass` guard. QED

I17 Current-attempt receipt binding

Claim. GatePass cannot use a package, executor/run, steering or model receipt from another attempt.

Proof. Receipt requires the current attempt ID and nonce for every bound field. I10 fixes them within an attempt; EnvelopeAdmit creates a fresh nonce on retry. A11 is used in the fail-open direction here too: a colliding stale receipt would be *accepted*, so the theorem holds only under the assumption that collisions do not occur. Any stale field takes ReceiptRefuse, not GatePass. QED

What is not proved

- **Quality** of a Passed body.
- **Item liveness.** BindFail / empty remainder / Ask idle can make Accept unreachable.
- **F2** without a runtime-instance field.
- **A1/A2 on a partitioned host.** A9 says the watchdog emits death_observed in finite time if it can see the pid. If it cannot, F4 is an estimate on the log.
- **Provider fidelity.** Observe records what the client reports. If the provider lies about the model that ran, I1 holds of the report, not of physics.
- **Norm injectivity.** norm strips a bracketed display suffix only; vendor and namespace prefixes are significant (fcd/core.py). Distinct API ids can still collide if they differ only inside brackets. I1/I3 hold up to that collision unless norm is injective on $\phi(\pi^*\delta) \cup \{\text{observed}\}$.
- **Re-observe.** Observe overwrites m_exec. I1 is the Pass-time value, not “no mismatched call ever occurred.” First-Observe mismatch is a misbind metric, not a Pass gate.
- **Body provenance.** Nothing ties m_exec to how body was produced.
- **Physical context isolation.** I11/I12 prove manifest/package/receipt properties. A10 is required to rule out hidden executor prefixes or session residue.
- **Adapter honesty.** A forged package/model/steering receipt can satisfy the abstract guards.
- **Executor/provider cache neutrality.** Internal cache/session behavior is telemetry and capability, not FCD authority.
- **Impact-review correctness.** continue_pinned promotion relative to a newer head relies on A12.
- **Instruction meaning.** Deterministic tool/authority conflicts block Admit; natural-language semantic contradiction is not generally decidable.

Executable form

tests/test_invariants.py, tests/test_core.py, and tests/test_context_envelope.py drive the same transition guards. A failing test is a counterexample to the corresponding claim on the Python machine. That is a proof about the core, not about an uninstrumented host.

Part IV — In plain words

The same guarantees without the notation. Nothing here may weaken a claim the proofs make, and every sentence restates something a proof or a transition already says.

The proofs, in plain words

Every theorem in paper/PROOFS.md (FCD, I1–I17) and paper/RGA/PROOFS.md (RGA, R1–R13, and the escape ledger's C1–C7), one plain sentence each. Rules, as in docs/UI_GLOSSARY.md: plain phrase first, exact term kept; a plain reading may never weaken the guarantee; every sentence restates something a proof already says, never more.

The two systems answer two different questions. **FCD answers: did the worker we chose actually do the work? RGA answers: was the work attacked hard enough, by an independent test, before we accepted it — and did the worker even agree with itself?** Neither proves the work is *good*. Both prove the paperwork cannot lie about what happened.

FCD — the identity proofs (I1–I17)

FCD treats "we run specialists" as a claim that must be checkable. Its machine only lets work through a gate when the identity story holds together.

I1 Bind integrity — A stage passes only when the model that *actually ran* is the model we *said* we'd use. The two names come from different witnesses — intent is written at Bind, reality at Observe — so the check can genuinely fail.

I2 Class admission — Whoever is running the work was picked from the approved list for this kind of work, never from outside it.

I3 No unbound hop — Putting I1 and I2 together: nothing ever passes on a model outside the approved list. When a bind dies, the system stops; it never quietly continues on a different model.

I4 Frozen class and body — Once work opens, what it is and what it's about can never be rewritten. A change is a new work item, full stop.

I5 Accept coverage — "Accepted" means every required stage passed — not most, not the important ones, all.

I6 Dual control — The reviewer of a piece of work is never one of its authors. You cannot grade your own homework.

I7 Bounded admits — Retries can't loop forever: each retry must use a worker not yet tried, and the list is finite.

I8 Store only accepted — The store of finished work has exactly one door, and only fully accepted work goes through it. Nothing gets in by a side path.

I9 Retry preserves class — A retry is the same job again, never a quietly different job.

I10 Pinning — The moment work starts, it locks the exact project state, memory and setup it will run under; nothing swaps those underneath it.

I11 Package receipt — The gate passes only if the worker reports back the exact fingerprint of the context it was handed. Different bytes, no pass.

I12 Fresh-blind — A blind review provably excludes the author's notes, reasoning and prior verdicts, and cannot resume the author's session.

I13 Serialized promotion — Project memory changes only when accepted work wins a one-at-a-time race; a loser writes nothing.

I14 No silent drift — If the project moved on while work was in flight, that work cannot pretend it didn't; someone must sign off on the difference.

I15 Steering acknowledged — Mid-flight instructions can't rewrite the contract, can't touch finished work, and the run must prove it saw the latest one before passing.

I16 Cache identity — A cache shortcut is honored only when *everything* about the attempt is identical. Close enough is a miss.

I17 Current-attempt receipts — Paperwork from a previous attempt can never authorize this one.

What FCD honestly does not prove: that the work is any *good*; that a worker who *lies* about what ran is caught (it proves the report is consistent, not that the report is true — that is assumption A10); that the work ever finishes. The paper says all of this itself.

RGA — the scrutiny proofs (R1–R13)

RGA's machine only seals work when a pre-registered, re-runnable attack on it was actually carried out, at a strength that was measured, on outputs that agree with each other.

R1 No admission without survival — Nothing seals unless every declared test was run against every one of the k outputs and every single one came back "survived." A failed test, a test that timed out, or a missing test each keeps the seal shut.

R2 Power carried, never inferred — The strength number on the seal is the one the kernel itself computed — by counting kills against a named set of planted defects, or from declared, visible parameters — written once, before sealing, and impossible to raise afterwards. Nobody hands the kernel a flattering number.

R3 Separation of duty, extended — The worker didn't write its own test, the test existed before the worker started, the planted defects weren't authored by the worker or the test's author, and the worker's inputs provably excluded the test materials. The exam was locked before the student saw the paper.

R4 Concordance is a precondition — The k outputs must agree on the substance with the *first* one, at a declared rate. Disagreement closes the gate; the machine never holds a vote and picks a winner.

R5 Nondeterminism refuses — Re-run any test and get a different answer once, and that test is banned everywhere, forever. An unreliable inspector loses the badge.

R6 Replay exercised — Every test used on this work was re-run at least once, on this work, with an identical result — the re-runnability isn't taken on faith.

R7 Sample integrity (inherited) — Every one of the k outputs came through FCD's identity gate first, so we know all k came from the same worker on the same job. This is where FCD's proofs are reused, not redone.

R8 Seal implies Accept — Everything RGA seals was already fully accepted by FCD — the strict store is a subset of the honest store — and once sealed, the underlying record can't be failed or swapped out from under it.

R9 Artifact binding — The tests ran against the exact bytes the kernel fingerprinted, and the seal names those bytes. Not "an output like this one" — *this one*.

R10 Seed after artifact — Test inputs that depend on randomness are derived only *after* the output exists, so the worker could not have tuned its output to inputs it had already seen.

R11 RGA writes no FCD field — RGA only reads the identity machine, never edits it — which is exactly why every FCD proof still holds with RGA on top.

R12 Frozen line — The rules of this run — which claims, how many samples, what threshold, what floor — are locked at the start and can't be adjusted mid-flight.

R13 Bounded — Sample counts, tests per slot, and strength records are all hard-capped and write-once. Nothing accumulates its way around a limit.

The faults (V1–V15), in one sentence: every way of cheating the above — sealing past a failure, counting a timeout as a pass, an under-strength seal, a plurality vote, a self-authored test, a tampered seed, an unmeasured or unreplayed test, a double-written strength record — is a named, forbidden step: the first five close the gate loudly, the rest are refused before they can write anything. And each protection is tested by *removing it* and proving the cheat then works (tests/test_rga_mutation.py).

What RGA honestly does not prove: that the work is *correct* — surviving a test is not being right; that the planted defects resemble the mistakes this worker really makes (that link is a stated assumption); that a harness reporting the verdicts is truthful (same boundary as FCD's A10 — but here a lie is catchable later, because the test can be re-run by anyone); that a worker who has memorized a public test can't game it. The seal also carries an explicit list of what was **not** attacked, so a pass on the checkable part can never be dressed up as a verdict on the judgment part.

The calibration layer — the escape ledger (C1–C7)

A third, thinner set of proofs sits on top of RGA and answers: *what happens when a sealed artifact turns out to be wrong anyway?*

C1 Established, never asserted — A "we found a defect in sealed work" report counts only if it is re-runnable: the very test the seal trusted, re-run at a new input the finder chose, killing the work — replayed identically. Anything else needs a named human decision on the record before it counts.

C2 Charge totality and unit — When a miss is proven, every test that vouched for that work is charged — read off the record, automatically — and one miss is one charge, no matter how many ways it is demonstrated.

C3 Impeachment entailed and checkable — A proven miss revokes the work's standing everywhere it is consulted, the certificate never gets rewritten, and nobody has the discretion to decline the revocation — the proof itself entails it.

C4 The ratchet — Nobody can quietly ship a new test battery that forgets a proven miss. Either the new battery covers it, or someone puts their name and their reason on the record for leaving it out. Forgetting is loud.

C5 Track record carried, and the stamp proves who approved — Every new approval is stamped with its instruments' history — how many proven misses, across how much work — as raw counts, never a percentage, with the stated warning that zero misses may only mean nobody looked. The stamp is also the proof that the ledger saw this approval at all: work that got its certificate without passing the ledger has no stamp, and the system says so out loud instead of treating it as fully approved.

C6 Demotion — Every kind of work must state its miss budget up front — a kind of work nobody configured is refused outright, rather than quietly treated as having an unlimited budget. A test past its declared budget can't be trusted with new work, and — where the class chose strictness — can't finish work in flight either; the shutout is loud, carries the numbers, and never rewrites anything already approved.

C7 Non-interference — The ledger only reads the layers below; every proof of the two systems underneath survives untouched.

What the ledger honestly does not prove: that the misses it knows about resemble the misses that exist. It is a ledger of *found* problems — attackers can pad it with true-but-chosen entries, silence in it is unreadable, and no one can be forced to look.

And one more, found by review rounds and worth stating plainly: replay alone detects internal inconsistencies, but it cannot distinguish a coherent replacement of root inputs or prove that no page is missing. v0.5 adds the missing publication layer: the three authority journals are read-only snapshots, each authority stack has its own registry namespace, successor heads prove the anchored prefix through a cumulative event chain, and one signed receipt carries those exact heads together with the kernel-derived result. After a trusted first anchor, a registry-current verified receipt rejects a cleanly shortened or forked history, a state-stale copy, a cross-wired stack, or predicates somebody merely typed. The first anchor cannot prove which coherent earlier history actually ran, and its signed time is metadata rather than expiry. This protection exists only when the receipt is issued and semantically verified against that external registry; schema validation alone cannot enforce the namespace/head equality. Bypass the path and the old replay-only limitation remains. If the signer/key and registry are both compromised or rolled back, the theorem has lost its trust roots. Everything else about forgetting stays loud.

The one-line version of each

FCD: the right worker provably did the work. RGA: the work provably survived a fair, measured, repeatable attempt to break it — several times over, consistently. The ledger: anything found against approved work afterwards sticks to it, automatically, and no new test battery can quietly forget it. The first is an ID check; the second is a crash test with the test's strength printed on the certificate; the third is the recall notice that nobody can tear up. None of the three will ever tell you the work is perfect, and all three are built to make that impossible to forget.

How to read the label. Work carries one of three marks. **I** — we know who did it, and nothing more is claimed. **IR** — it also survived the tests, but the ledger never counter-signed it, so its standing is not being tracked. **IRC** — all three: identity, scrutiny, and standing that stays live, so if a defect turns up next month the mark changes by itself. A missing letter is never a smaller version of the next one up; it is a different, smaller claim.

UI glossary

Maps the paper's vocabulary to what the cockpit shows. An engineer should be able to operate the instrument without reading paper/DRAFT.md; this file is the bridge. The UI ships its own copy of these readings in apps/cockpit/src/domain/glossary.ts; the two are mirrored by hand and kept aligned in review — where they differ, the glossary in the code is what the cockpit actually shows.

Sources of truth: paper/INVARIANTS.md (assumptions A0–A13, transitions, theorems I1–I17), paper/DRAFT.md (fault table), fcd/core.py, fcd/context.py.

Implementation: apps/cockpit/src/domain/glossary.ts holds the UI's table and feeds both the inline hover cards (Gloss) and the **Legend** panel in the command rail. Change one, change the other — by hand; there is no generator.

Rules for this vocabulary

1. **Plain phrase first, exact term kept.** The machine's words are load-bearing and appear in the journal, the schemas and the proofs. The UI keeps them and attaches a plain reading, rather than renaming them into something friendlier that no longer matches the record.
2. **No softening.** A gloss may not weaken a guarantee. "Reachable" never becomes "affected", "unknown" never becomes "safe", "candidate" never becomes "done".
3. **Derived, not invented.** Every plain sentence restates something the transition table, an invariant or the code already says.

Objects

work item, status $\in$ {open, failed, accepted} — *UI shows:* **Line** — centre pane title, atlas rows · *Plain reading:* One unit of work taking form through its required gates. It ends as an accepted artifact, or it stays closed.

stage, Required(c) — *UI shows:* **Gate** — a card on the load path · *Plain reading:* One required stage of a line. Each gate binds an agent, one exact model and a context policy.

write stage — *UI shows:* WRITE tag · *Plain reading:* Produces the work. Whoever passes it is recorded as an author of the line.

check stage, π_chk (A6, I6) — *UI shows:* CHECK tag · *Plain reading:* Reviews the work. It cannot admit anyone who authored this line.

c (I4) — *UI shows:* class in the line header · *Plain reading:* What kind of work this is. It decides the allow set and the required gates, and it is fixed the moment the line opens.

candidate — *UI shows:* CANDIDATE seal, artifact pane · *Plain reading:* Produced but not accepted. It sits outside the store and can still be retried or discarded.

id $\in$ **S (I8)** — *UI shows:* ACCEPTED seal · *Plain reading:* Accepted and immutable. A fix is a new line, never an edit of this one. Accepted is the identity layer's word; sealed is the scrutiny layer's, and it is stronger.

Lifecycle

Open (I4) — *UI shows:* gate state open · *Plain reading:* The line starts. Its class and body are fixed here and can never be rewritten.

Admit(a), $a \in \pi^*(c) \setminus \delta(c) \setminus \text{tried}$ — *UI shows:* "Admit and run this gate"; the envelope reads "Editable before Admit", "Retryable", "No specialists left to try", or "Locked at Admit" · *Plain reading:* The machine takes one specialist from the allow set. Admitting freezes the envelope: agent, exact model, context package and steering base stop being editable.

Bind, $m_decl \leftarrow \phi(a)$ — *UI shows:* declared ... in the route row · *Plain reading:* The machine writes down which exact model it intends to use. Intent only, never what ran.

Observe, $m_exec \leftarrow m$ (A1) — *UI shows:* executed ... in the route row · *Plain reading:* The provider reports which model actually ran. The only place executed identity is written.

Pass, $\text{norm}(m_exec)=\text{norm}(m_decl)$ (I1) — *UI shows:* gate state held, green load-path segment, MATCH · *Plain reading:* The gate holds. Allowed only when the model that ran matches the model that was bound.

PassRefuse / Close, $\text{pub}=1$ — *UI shows:* gate state broke, the **shear** in the load path, F... stamp · *Plain reading:* The gate broke and the break was published. Nothing downstream runs.

(no hop transition) — *UI shows:* "The line never hops out of it" under Recovery · *Plain reading:* Silently continuing on a different model after a break. This machine has no such transition.

Retry (I9, A7) — *UI shows:* "Run this gate again" in the envelope, "Retry inside allow set" in recovery · *Plain reading:* Reopen the same gate with a different allowed specialist. The contract is never rewritten, and a specialist already tried is not offered again.

Accept (A8, I5) — *UI shows:* Accepted seal, "Accept is the only writer" · *Plain reading:* The only writer of the store. Every required gate must have held first.

$pc=\text{Open}$ below a Closed stage, line finished — *UI shows:* **not reached** badge, dashed rail and card · *Plain reading:* This gate never ran. Not queued work — work that did not happen.

$pc=\text{Open}$ below a Closed stage, line still open — *UI shows:* **not started** badge · *Plain reading:* An earlier gate broke, but the line is still open. This gate runs only if that gate is retried and holds.

watchdog (A2, A9) — *UI shows:* — (server side) · *Plain reading:* Runs outside the worker and reports its death. It can only close a line, never accept one.

Route

$\pi^*(c)$, π_chk (A6) — *UI shows:* "allow set" in the contract card and recovery hint · *Plain reading:* The specialists this gate may admit. On a check gate it excludes everyone who authored the line.

$\delta(c)$ (A0) — *UI shows:* deny set (settings/policy) · *Plain reading:* Specialists this class may never admit. Deny always overrides allow.

ϕ — *UI shows:* provider / api_id in the route row · *Plain reading:* Which exact provider model each specialist resolves to.

norm(x)=norm(y) (A5) — *UI shows:* MATCH / MISMATCH verdict · *Plain reading:* Two model names match only as API identities. Vendor prefixes count; a display suffix is not an identity.

policy version — *UI shows:* policy ... chip · *Plain reading:* The identity of the rules this line runs under. A line finishes under the version it opened with.

ExecutionAdapter — *UI shows:* "Execution adapter" field · *Plain reading:* The outside worker that runs the gate. It produces candidates and receipts; it can never pass or accept.

execution-readiness.schema.json — *UI shows:* "Route ready" / "Route not ready" with the failing checks named · *Plain reading:* Explicit checks that this exact provider route can run now. Any failed check blocks Admit rather than degrading quietly.

Context envelope

X, nonce (I10) — *UI shows:* the gate envelope panel · *Plain reading:* The frozen record of one attempt: agent, exact model, context package and the steering it started from.

ContextPackage (I11) — *UI shows:* package receipt row · *Plain reading:* The exact bytes handed to the executor. A pass requires the executor to report back the same hash the machine expects.

AdapterReceipt (I17) — *UI shows:* receipt valid / receipt missing · *Plain reading:* The executor's statement of what it ran and what context it received. A stale or mismatched receipt refuses the pass.

steering continuation (I15) — *UI shows:* steering readout in the command bar · *Plain reading:* Steering is evidence the run must acknowledge; it cannot rewrite the contract.

project_shared | fresh_scoped | fresh_blind | contract_only — *UI shows:* Context select, each with its own sentence · *Plain reading:* What the executor is given. **fresh_blind** (I12) excludes the author's transcript, reasoning and prior verdicts, and forces a fresh session.

fresh | executor_continue | executor_fork — *UI shows:* Continuity select · *Plain reading:* Whether the executor starts a new session, resumes its own, or forks it. The last two need declared adapter capability.

current (P_v, K_v) — *UI shows:* project ... · memory ... in the command rail · *Plain reading:* The versions accepted **right now**. A line pins these when it opens and finishes against the pin even after the head moves on.

(P_v, K_v) write-once at Open (I10, I14) — *UI shows:* project pin on the gate receipt · *Plain reading:* The versions this line opened against. They never change underneath it.

ImpactReview (A12, I13) — *UI shows:* **drift** flag and the impact-review banner · *Plain reading:* The accepted project moved on after this line opened. The line keeps its pin; the impact must be reviewed before Accept.

telemetry outside the transition table — *UI shows:* executor cache receipt row · *Plain reading:* What the executor says it reused. Its own report, unchecked, and **no guarantee rests on it**.

FCD cache (I16) — *UI shows:* — (not yet surfaced) · *Plain reading:* The machine's own cache identity, tied to one attempt, envelope and continuation. Distinct from the row above.

Admissibility (layers R and C)

The scrutiny and standing layers (rga/core.py, rga/calibration.py; papers paper/RGA/DRAFT.md, paper/admissible/DRAFT.md). The terminal on an accepted line repeats the server's admissibility sentence verbatim, and its tone may never outrun the record: amber for accepted-without-seal, the broke tone once a line is no longer admissible.

admissible(id) := id ∈ S_R ∧ mediated ∧ ¬tainted ∧ ¬impeached — *UI shows: layer chip I / IR / IRC in the line header · Plain reading:* Which layers' guarantees govern this line. I: the identity layer alone — nothing beyond class dispatch is claimed, and the line's own state says how far it got. IR: it also sealed under scrutiny, but the calibration authority never counter-signed the seal, so its standing is not tracked. IRC: all three. A missing letter is a smaller claim, never a weaker version of the next one.

mediated(id): one cal_stamp bound to seal.sealed_at (C5) — *UI shows: terminal label Sealed — layer IR, not mediated when absent · Plain reading:* The calibration authority counter-signed this seal: exactly one track-record stamp, bound to this seal and no other. Without it the line sealed at the scrutiny layer only — it reads IR, and it is not admissible.

Seal, id ∈ S_R ⊆ S (R1, R8) — *UI shows: terminal label Sealed · Plain reading:* The scrutiny layer's acceptance. A line seals only after its samples survived every pinned refuter, and the seal carries the measured strength of that scrutiny. Write-once; nothing on it can be raised later.

accepted, not sealed — *UI shows: terminal label Accepted — layer I, with the refusal reason · Plain reading:* The identity layer accepted it, the scrutiny layer could not. The reason is stated, never a smaller green.

admissible(id) = sealed ∧ mediated ∧ ¬tainted ∧ ¬impeached — pure queries (C3, §8.2) — *UI shows: admissible / not admissible chip · Plain reading:* Counter-signed, not tainted, not impeached — right now. A live query against the escape ledger, never a stored flag: recomputed on every read, so it cannot be set. What the ledger cannot see is an entry deleted from its own record.

impeached(id) entailed by a valid escape (C3); tier A past kernel checks, tier B past adjudication (C1) — *UI shows: terminal label Impeached · Plain reading:* A valid escape stands against a sealed claim: either the seal's own pinned refuter, re-run at an input a finder chose, killed the artifact (automatic once replayed), or another checker did and an adjudicator accepted the claim-match. The seal never rewrites; the record of the find stands beside it, and it lapses only by journal facts — a discredited checker — never by discretion.

tainted(id): sealed line relied on a refuter refused after sealing (§4; refusal is R5-monotone) — *UI shows: terminal label Tainted · Plain reading:* A refuter this seal relied on was later caught giving different answers on replay. Everything that trusted it is marked, monotonically — the mark never comes off.

power = kills/|D| or 1-(1-ε)^N (R2) — *UI shows: power ... chip · Plain reading:* How hard the refuters actually tried, as a number the kernel counted: kills over a named defect set, or a bound computed from declared trial parameters. Never a scalar anyone wrote down.

witness agreement at θ (R4) — *UI shows: concordance (agreeing, k) chip · Plain reading:* Whether independent samples agreed on the claim's observable behaviour. At k=1 it is visibly unmeasured, not silently perfect.

seal.residual — check_stage requires a Passed check stage (§4, V15) — *UI shows: residual chips on the terminal · Plain reading:* What the seal does not cover: every claim that was not attacked by any

refuter, named on the seal with how it was left — reviewed at a check gate, or not reviewed at all.

Certainty bands

docs/EVIDENCE_MODEL.md requires these three to stay disjoint. The UI prints each definition next to its band, every time.

Observed — *Plain reading:* It happened, and the journal shows it. · *Rendering:* Oxide top rule

Reachable — *Plain reading:* Analysis proves it may be affected. It has not been seen. · *Rendering:* Amber top rule

Unknown — *Plain reading:* The evidence does not bound it. Not a list of safe things. · *Rendering:* Neutral top rule, hatched ground

Fault codes

Stamped on a gate that broke. Hovering the stamp shows the plain reading.

Each code is shown on screen with its complete name, because a bare F1 is as unreadable as a bare P2 · K2.

F1 — *Name shown on screen:* executed model did not match the bind · *Plain reading:* The model that ran was not the model that was bound. · *Formal:* $\text{norm}(m_{\text{exec}}) \neq \text{norm}(\phi(a))$

F2 — *Name shown on screen:* two specialists shared one runtime instance · *Plain reading:* Two specialists shared one runtime instance. · *Formal:* needs a runtime-instance field to measure

F3 — *Name shown on screen:* the bound model was unusable · *Plain reading:* The bound model was unusable, or a call continued outside the unused allow set instead of failing closed. · *Formal:* after $u = 0$, another call without fail-closed

F4 — *Name shown on screen:* the worker died with no published close · *Plain reading:* The worker exited while the gate was running and no close was published. · *Formal:* Running exit with no published close

F5 — *Name shown on screen:* the mapped model is not an API identity · *Plain reading:* The mapped model is not a real API identity. · *Formal:* $\phi(a)$ not an API identity

F6 — *Name shown on screen:* passed on a denied specialist · *Plain reading:* A gate passed on a denied specialist. · *Formal:* Pass with $a \in \delta(c)$

F7 — *Name shown on screen:* a check gate admitted an author of this line · *Plain reading:* A check gate admitted someone who authored this line. · *Formal:* check admit with $a \in \text{authors}$

F8 — *Name shown on screen:* ran without a well-formed gate · *Plain reading:* Something ran without a well-formed stage. · *Formal:* run without a well-formed stage

F9 — *Name shown on screen:* stopped while not accepted · *Plain reading:* The line was stopped in conversation while it was not accepted. · *Formal:* Stop in chat with status not accepted

F10 — *Name shown on screen*: the class changed, or a tried specialist was retried · *Plain reading*: The same id came back with a different class, or a specialist already tried was retried. · *Formal*: same id, class changes; or retry of a $\in$ tried

Gate states

The label at the right of every gate card.

pc	UI label	Meaning
Open	open	Declared, not yet admitted.
Admitted	admitted	A specialist is taken; the envelope is frozen.
Running	running	Bound and executing.
Passed	held	Executed identity matched the bind.
Closed	broke	Broke and published. Nothing downstream ran.
Stopped	stopped	Closed and deliberately ended.

Surfaces

What will this prompt actually enforce? — The contract card, compiled by the authority before anything opens

Which models can this project run, and can they run now? — **Models** in the command rail — every route, the gate that uses it, and each readiness check

What does this term mean? — Hover any dotted term, or **Legend** in the command rail

What did this attempt actually bind and execute? — The gate receipt: route configured, bound this attempt, executed

Why did it break, and what is still safe? — The failure card: what happened, what remains safe, three certainty bands, evidence, recovery

What did the authority refuse? — The refusal strip, which does not time out

Where the vocabulary is enforced

- apps/cockpit/src/domain/glossary.ts — the table, typed.
- apps/cockpit/src/components/Gloss.tsx — inline hover/focus cards.
- apps/cockpit/src/components/LegendPanel.tsx — the searchable legend, opened from **Legend** in the command rail.
- server/app.py _event_label — evidence rows read as sentences, with the exact payload one click away.

Part V — Contracts and records

What the machines emit, and what the machines measured. The event contract is language-neutral and conformance-tested against live emissions; the bench numbers are kernel queries over the journals, under simulated generators the record labels as simulated.

Telemetry contract

Events for fail-closed class dispatch. Policy fields are as-of ts.

A **cut** is $[t_0, t_1]$. W is class p95 of completed stage durations in the previous cut, or 12 minutes if $n < 30$. Exclude $ts > t_1 - W$. Policy as-of ts.

Zeros are not yet gathered. Compute with `fcd.metrics.rates / survival` on a write-ahead journal after a named cut. Do not backfill mixed historical logs.

stage

- ts
- work_item_id
- class
- stage_id
- stage_kind write | check
- assigned_specialist_id
- declared_model (Bind, pre-norm)
- authors specialist ids of prior writing stages
- body_hash
- policy_version
- well_formed true only if this event is the control-plane start
- pi_star allow set used at admit (π or π_chk)

open

- ts
- work_item_id
- class
- body_hash

- policy_version
- depends_on item ids whose accepted artifacts this body builds on (DAG gate)

bind

- ts
- work_item_id
- stage_id
- declared_model ($\phi(a)$, pre-norm)
- policy_version

Emitted on bind success. Durable: replay restores Running even if the process died before the first provider call.

call

- ts
- work_item_id
- stage_id
- declared_model (Bind, pre-norm)
- executed_model (Observe, pre-norm)
- runtime_instance optional; required to distinguish F2 from F1
- on_bind recomputed as $\text{norm}(\text{executed}) == \text{norm}(\text{declared})$
- first_attempt true on the first Observe of this stage
- signal none | 401 | 403 | 404 | 429 | exhausted | not_found

decide

- ts
- work_item_id
- stage_id
- result pass | fail_closed
- fault F1..F10 or null

- next retry | ask | stop | accept
- tried specialists already attempted this stage

accept

- ts
- work_item_id
- store_id

Context-envelope events

These events support I10–I17 audits. They do not add empirical rates by themselves.

work_pin

- ts, project_id, work_item_id
- project_version, memory_version, contract_revision

envelope_admit

- ts, work_item_id, gate_id
- attempt_counter, unpredictable_nonce, envelope_hash
- agent_id, agent_revision, specialist
- executor_id, executor_revision
- exact_model_provider, model_api_id
- context_mode, memory_scope, instruction_hash, tool_manifest_hash
- initial_steering_hash, steering_channel

context_package

- ts, attempt_id, nonce
- effective_categories (include minus exclude)
- package_hash_expected

adapter_receipt

- ts, attempt_id, nonce
- executor_id, run_id

- package_hash_observed
- latest continuation_hash
- exact executed_provider, executed_model
- optional reported_reuse, opaque executor cache/session ID; telemetry only

steering

- ts, attempt_id, sequence
- scope project | work | gate | stage | artifact | evidence | failure
- target_id, text_hash, continuation_hash

impact_review

- ts, work_item_id
- classification unaffected | reachable | direct_conflict | unknown
- decision continue_pinned | refresh | owner_override
- actor, reviewed_project_version, reviewed_memory_version, signature

memory_promote

- ts, work_item_id
- expected and resulting project/memory versions
- accepted knowledge-delta hash and artifact/evidence references
- CAS result success | refuse

Rates (empty)

Rate	num / den	Notes
Misbind	—	first Observe / stage; publish only with silent-fail
Silent fail	—	stages + orphan opens; fail-closed = published
Bleed	—	$a \notin \pi^*$ as-of ts
Time-to-stage	—	survival, not a mean; right-censor ts > t1-W

Refutation-gated admission events

Emitted by rga/core.py. They support the R1–R13 audits (fault codes V1–V15) in paper/RGA/PROOFS.md. Positions are indices into the RGA journal; fcd_position is an index into the FCD journal. None of these adds an empirical rate by itself.

rga_declare

- ts, refuter_id, refuter_version, author (a declared identity string, B11), mode ledger | bounded

rga_measure

- ts, refuter_id, refuter_version
- defect_model_hash, defect_model_author
- ledger list of {defect_id, verdict} with verdict killed | survived | inconclusive
- kills, size, power — computed by the kernel from ledger; inconclusive counts in size only

rga_bound

- ts, refuter_id, refuter_version, epsilon, n, power = $1 - (1 - \epsilon)^n$, computed by the kernel

rga_open

- ts, work_item_id, class, body_hash, generator, declared_model
- fcd_policy_version, sampling_hash, policy_version (RGA), k, theta, p_min
- claims ids pinned; refuters (id, version) pairs pinned
- fcd_position FCD journal length at Open, read by the kernel (never caller-supplied); no stage event for the item's first k stages precedes it. Replay cross-checks (k, theta, p_min, claims, refuters) against the rebuilt line

rga_sample

- ts, work_item_id, sample_index, stage_id
- artifact_hash sha256 computed by the kernel over the bytes it was handed
- nonce drawn after the hash and after every Sample guard (B9); replay reads it
- fcd_position FCD journal length at Sample, read by the kernel; no stage event for a later sample stage precedes it
- executed_model, declared_model copied from the FCD stage (I1 holds of them)
- package_categories as reported; disjoint from the class's excluded set
- sampling_hash equal to the line's

rga_trial

- ts, work_item_id, trial_index, refuter_id, refuter_version, claim_id, sample_index
- seed = H(nonce | artifact_hash | refuter | claim), equality-checked
- inputs_hash (carried for audit; not compared by the kernel), verdict refuted | survived | inconclusive, witness_hash

rga_replay

- ts, work_item_id, trial_index, refuter_id, refuter_version, verdict, witness_hash, diverged

rga_refuse

- ts, refuter_id, refuter_version, reason. Monotone: there is no un-refuse event.

rga_seal

- ts, work_item_id, class, body_hash, artifact_hash, k, theta, p_min, power_min, sampling_hash, policy_version (RGA), fcd_policy_version, generator, executed_model
- claims list of {claim_id, spec_hash, composite, composition single|union|max, agreeing, k, refuters: [{id, version, mode, power, defect_model_hash, kills, size, epsilon, n}]} — ledger refuters carry kills/size, bounded refuters epsilon/n; the null fields mark the other mode
- residual list of [intent, disposition]; check_stage only if an FCD check stage Passed

rga_close

- ts, work_item_id, result fail_closed, fault V1..V5 or null, reason, next ask
- V2 adds concordance [{claim_id, agreeing, k}] and miss_observed [[refuter_id, version]] — exactly the refuters whose own witnesses differ from their sample-0 witness
- V5 adds power_min

RGA rates (empty)

Refutation — *num / den:* — · Notes: rga_trial.verdict=refuted over trials, per class and refuter version

Discord — *num / den:* — · Notes: rga_close.fault=V2 over lines reaching Seal, at declared (k, sampling_hash) joined from rga_open

Miss observed — *num / den:* — · Notes: per refuter version: V2 closes naming it in miss_observed; a detected miss only where the witness is a function of the claim value

Replay divergence — *num / den:* — · Notes: rga_replay.diverged over replays, per refuter version; publish only with silent-fail

Escape replay — *num / den:* — · Notes: per successor refuter version; numerator needs a defect-report channel outside the journal; informative only for a version other than the sealing one

Calibration events

Emitted by rga/calibration.py (CalibrationAuthority). They support the C1–C7 audits in paper/RGA/PROOFS.md; fault codes are E1–E9. Positions are indices into the calibration journal.

cal_run

- ts, run_index, line_id, class, claim_id, checker_id, checker_version
- tier A (checker pinned to the claim in the seal; seed is the kernel's derivation over the sealed hash) | B (any other declared checker; consequences require adjudication)
- nonce finder-chosen; artifact_hash sha256 computed by the kernel over the filed bytes, equal to the seal's; seed
- verdict refuted (escape) | survived (audit), witness_hash, finder (journal-cited provenance; gates nothing)

cal_replay

- ts, run_index, verdict, witness_hash, diverged. Equal outcome establishes the run; divergence discredits the checker.

cal_discredit

- ts, checker_id, checker_version, run_index. Monotone: there is no un-discredit event; validity of every run by that checker degrades at query time.

cal_adjudicate

- ts, run_index, actor, decision accept | reject, reason. Tier B escapes only; once.

cal_exclude

- ts, class, as_of (the Admission position the corpus was read at), run_indices, actor, reason, corpus_size, excluded_total. Releases named corpus entries from successor coverage; waives nothing else.

cal_install

- ts, policy_version (the **admission** policy), calibration_policy_version, as_of (the Admission position the ratchet was read at), budgets per class {e_max, demotion_gate}, coverage per class {corpus_size, excluded, models: {claim_id: defect_model_hash}}, dropped_defect_ids (the predecessor-diff), dropped_classes (classes leaving the policy; refused while they owe coverage). Emitted only past the ratchet guards (E4) and the class-coverage guard (E9); rga/core.py install itself emits nothing. The budgets are journaled because demoted() gates CalOpen and CalSeal: a budget nobody can read from the record is a gate nobody can audit, and replay refuses a supplied policy that disagrees with the one the journal installed.

cal_stamp

• ts, line_id, sealed_at, track_records per pinned refuter: {charged_cells, seals_participated, corpus_size, corpus_excluded, as_of}, and corpus_provenance {finder_is_generator, independent} — the class corpus split by finder for this seal's generator. Primaries, never a rate; a zero charged_cells is absence of filed evidence, never measured power; the finder split is journal-cited provenance and gates nothing.

cal_close

• ts, line_id, fault E5, as_of (the Admission position the demotion was read at), refuter_id, refuter_version, primaries. The demotion gate where the class declared it; the line closes through Admission's operator close in the same step. as_of is what makes the close replayable: without it, rebuild re-reads the demotion against final state and refuses honest journals.

Calibration rates (empty)

Escape rate — *num / den:* — · *Notes:* valid escapes over seals, per class and tier; doubly selected, one-sided

Adjudication acceptance — *num / den:* — · *Notes:* tier-B accepts over adjudications

Exclusion load — *num / den:* — · *Notes:* excluded over corpus size per install; primaries already on the event

Charge accrual — *num / den:* — · *Notes:* charged cells per refuter version; never read as quality without seals_participated

Audit coverage — *num / den:* — · *Notes:* lines with a valid audit over lines sealed; reads only with the audit events

Bench results: the admissibility kernel, end to end

Generated by python3 eval/bench/bench.py --write. Do not edit; re-run.

What these numbers are

A deterministic evaluation of the three-layer kernel (identity, scrutiny, standing) on reference implementations of four well-established stdlib specifications — median, wrap, normpath, quantiles (wrap on the domain where words fit the width) — with the stdlib itself as the strong refuter's differential oracle. Defect models are built by a seeded AST mutator; every mutant admitted to a model carries a demonstrated killing witness (B5), and mutants no search input separates are dropped and counted. Every number below is computed from the kernels' write-ahead journals and run records on the named cut — line outcomes from rga_seal/ rga_close events, standing from the calibration ledger's own runs and queries; nothing is declared.

What they are not. The generators are simulated — honest (correct code under cosmetic variation), sloppy (a latent subtle mutant in every sample), unstable (behavior flips between samples). No claim about any LLM's defect rate follows from this table; the bench evaluates the *machinery* — that seals require survival, that discord and refutation close loudly, that escapes impeach, charge, demote and ratchet.

Setup

master seed — 20260821

k (samples per line) — 3

e_max (charge tolerance) — 1

lines per scenario — {'honest': 8, 'sloppy': 24, 'unstable': 6}

trial budgets — weak: 2 inputs/sample (properties), strong: 4 inputs/sample (properties + stdlib oracle)

escape search — pinned strong refuter, 40 finder nonces per sealed line, kernel-derived seeds

witness (B8) — outputs on a fixed 16-probe set — seed-independent, so cosmetic twins agree and behavioral twins do not

Target: median

Defect model D-median-be3c84fb075c16ee: **11** mutants with a killing witness; **0** dropped as possibly equivalent.

Carried power (kernel-counted, kills/|D|)

refuter	kills	D	power
spec-weak@v1	7	11	0.6364
spec-strong@v1	9	11	0.8182

Union composite carried on seals: **0.9091**.

Line outcomes

scenario	sealed	V1 refuted	V2 discord	other
honest	8	0	0	0
sloppy	2	22	0	0
unstable	0	6	0	0

Standing

- Escapes: **2 filed, 2 established**, 2 lines impeached — by scenario: honest: 0, sloppy: 2, unstable: 0.
- Charges: {'spec-strong@v1': 2, 'spec-weak@v1': 2} — demoted: {'spec-strong@v1': True, 'spec-weak@v1': True} (e_max=1).
- Ratchet: corpus 2; successor policy installed; successor power **0.9231** (was 0.8182); 1/2 escaped artifacts killed by the successor at its calibration seed. The successor is the SAME checker code re-versioned and re-measured at fresh calibration seeds over $D \cup \text{corpus}$ — the power movement is seed-roll and corpus growth, not a better checker — and demotion charges do not carry across versions: what the ratchet proves is coverage (C4), not improvement.
- Journals on cut [0, 896]: fcd 372, rga 357, cal 15 events.

Target: wrap

Defect model D-wrap-9ac6cbd200246341: **3** mutants with a killing witness; **0** dropped as possibly equivalent.

Carried power (kernel-counted, kills/|D|)

refuter	kills	D	power
spec-weak@v1	2	3	0.6667
spec-strong@v1	3	3	1.0

Union composite carried on seals: **1.0**.

Line outcomes

scenario	sealed	V1 refuted	V2 discord	other
honest	8	0	0	0
sloppy	0	24	0	0
unstable	0	4	2	0

Standing

- Escapes: **0 filed, 0 established**, 0 lines impeached — by scenario: honest: 0, sloppy: 0, unstable: 0.
- Charges: {'spec-strong@v1': 0, 'spec-weak@v1': 0} — demoted: {'spec-strong@v1': False, 'spec-weak@v1': False} (e_max=1).
- Ratchet: corpus 0; no install — no escapes, nothing owed.
- Journals on cut [0, 838]: fcd 352, rga 326, cal 8 events.

Target: normpath

Defect model D-normpath-060baf315f8240ae: **9** mutants with a killing witness; **0** dropped as possibly equivalent.

Carried power (kernel-counted, kills/|D|)

refuter	kills	D	power
spec-weak@v1	2	9	0.2222
spec-strong@v1	6	9	0.6667

Union composite carried on seals: **0.6667**.

Line outcomes

scenario	sealed	V1 refuted	V2 discord	other
honest	8	0	0	0
sloppy	2	22	0	0
unstable	1	5	0	0

Standing

- Escapes: **2 filed, 2 established**, 2 lines impeached — by scenario: honest: 0, sloppy: 2, unstable: 0.
- Charges: {'spec-strong@v1': 2, 'spec-weak@v1': 2} — demoted: {'spec-strong@v1': True, 'spec-weak@v1': True} (e_max=1).
- Ratchet: corpus 2; successor policy installed; successor power **0.9091** (was 0.6667); 2/2 escaped artifacts killed by the successor at its calibration seed. The successor is the SAME checker code re-versioned and re-measured at fresh calibration seeds over $D \cup \text{corpus}$ — the power movement is seed-roll and corpus growth, not a better checker — and demotion charges do not carry across versions: what the ratchet proves is coverage (C4), not improvement.
- Journals on cut [0, 895]: fcd 369, rga 358, cal 16 events.

Target: quantiles

Defect model D-quantiles-1bd64ed62158ea5d: **21** mutants with a killing witness; **0** dropped as possibly equivalent.

Carried power (kernel-counted, kills/|D|)

refuter	kills	D	power
spec-weak@v1	15	21	0.7143
spec-strong@v1	20	21	0.9524

Union composite carried on seals: **0.9524**.

Line outcomes

scenario	sealed	V1 refuted	V2 discord	other
honest	8	0	0	0
sloppy	3	21	0	0
unstable	2	4	0	0

Standing

- Escapes: **3 filed, 3 established**, 3 lines impeached — by scenario: honest: 0, sloppy: 3, unstable: 0.
- Charges: {'spec-strong@v1': 3, 'spec-weak@v1': 3} — demoted: {'spec-strong@v1': True, 'spec-weak@v1': True} (e_max=1).
- Ratchet: corpus 3; successor policy installed; successor power **0.9167** (was 0.9524); 3/3 escaped artifacts killed by the successor at its calibration seed. The successor is the SAME checker code re-versioned and re-measured at fresh calibration seeds over $D \cup \text{corpus}$ — the power movement is seed-roll and corpus growth, not a better checker — and demotion charges do not carry across versions: what the ratchet proves is coverage (C4), not improvement.
- Journals on cut [0, 969]: fcd 403, rga 394, cal 20 events.

Reading

- median — honest lines sealed 8/8; subtle tier by estimated per-input detectability (m0: 0.094, m7: 0.375, m5: 0.422); sloppy lines sealed with a latent defect 2/24, refuted at trial 22; unstable lines closed V1 6 / V2 discord 0. Every latent-defect seal was impeached (2/2) and no honest or unstable seal was.
- wrap — honest lines sealed 8/8; subtle tier by estimated per-input detectability (m2: 0.25, m0: 0.859, m1: 0.891); sloppy lines sealed with a latent defect 0/24, refuted at trial 24; unstable lines closed V1 4 / V2 discord 2. Every latent-defect seal was impeached (0/0) and no honest or unstable seal was.
- normpath — honest lines sealed 8/8; subtle tier by estimated per-input detectability (m3: 0.141, m5: 0.188, m6: 0.203); sloppy lines sealed with a latent defect 2/24, refuted at trial 22; unstable lines closed V1 5 / V2 discord 0. Every latent-defect seal was impeached (2/2) and no honest or unstable seal was.

- quantiles — honest lines sealed 8/8; subtle tier by estimated per-input detectability (m0: 0.078, m1: 0.078, m7: 0.578); sloppy lines sealed with a latent defect 3/24, refuted at trial 21; unstable lines closed V1 4 / V2 discord 0. Every latent-defect seal was impeached (3/3) and no honest or unstable seal was.
- Obvious defects die at trial (V1); only the subtle tier can seal, and the pinned refuter at fresh nonces is what impeaches it. Where no subtle mutant exists, the escape search finds nothing — the pipeline does not manufacture escapes to have something to show.
- Detectability estimates are bench selection metadata; they never enter a kernel record. The kernel carries only what it counted.

Section status

Admissible (paper/admissible/)

Name — Admissible — the admissibility kernel. Argued from the code's top predicate `admissible(id) = sealed ∧ mediated ∧ ¬tainted ∧ ¬impeached` and the evidence-law/Daubert mapping (testability→refuters, known error rate→carried power, controlling standards→pinned policy, impeachment→escape ledger). Certifies procedure, never truth

Unified paper — `paper/admissible/DRAFT.md`: model, threat model, three layer summaries, Theorem 1 (soundness of the record) + Theorem 2 (loudness of deviation) as compositions by citation, methodology (delete-the-guard, citation binding, premise-first rounds), consolidated related work and limits. Adds framing and composition, not new mathematics, and says so

Counts (measured) — 595 checks green: 487 tests/ + 37 atlas + 71 cockpit; 43 per-guard deletion proofs + 2 joint; 2 citation binders

Empirical (§9) — Three studies, none admissible evidence. Kernel bench (`eval/bench/`, Fig. 8) — machinery only; honest is the oracle, sloppy is drawn from D, and the figure says so; 7/7 latent-defect seals impeached, 3 unstable seals stand as misses. Real generator (`eval/generators/`) — 48 samples, 16/16 lines sealed, **zero defective**, ceiling structural. Real defects (`eval/realdefects/`, Fig. 9) — **8 hand-verified**, no defensible rate. Full account and every voided run: `eval/LOG.md`

Canonical machine: `fcd/`. Round 4 conditions applied. Metrics empty until a named cut.

Name — Fail-closed class dispatch. Work item → accepted artifact

Observe vs Bind — `fcd/core.py`. Bind writes `m_decl`. Observe writes `m_exec`

Proofs I1–I17 — `paper/PROOFS.md`

Tests — 595 repository checks total: 487 Python kernel/server/context/RGA/calibration/paper/custody + 37 atlas/protocol + 71 cockpit

Context authority — `fcd/context.py`; attempt/nonce, package receipts, steering, CAS promotion

Execution boundary — `server/execution.py`; mature executor internals stay external

Watchdog — `fcd/watchdog.py` (injected `alive_fn`)

Stage cache — `fcd/cache.py` — same specialist + ϕ + prefix only

Store — Accept only

Rates — Defined in `fcd/metrics.py` + `metrics/SCHEMA.md`. No numbers

PDF — `paper/fail-closed-class-dispatch.pdf`

Hop — Illustration

Site collector — `eval/private/` only

Refutation-gated admission (paper/RGA/)

Premise — paper/RGA/PREMISE.md. 38 findings, five lenses; none of six "kills-premise" claims survived as such; two pre-registered positions fell; §9 records round 1

Round 1 — Five-lens review of the finished kernel: five REVISE verdicts; 5 RGA kernel defects (3 of which invalidated published proofs) + 2 FCD kernel defects (no_admit pc guard, open fresh-id guard) repaired with tests; eval/reviews/rga-r1-SYNTHESIS.md

Name — Refutation-gated admission. Line → sealed artifact. Title kept; abstract scopes it to the refutation record

Kernel — rga/core.py, composed over fcd/core.py; writes no FCD field (R11)

Proofs R1–R13 — paper/RGA/PROOFS.md; cites path:line:symbol, move-detected by tests/test_rga_citations.py

Tests — 97: 65 invariant traces, 30 guard-deletion and coverage checks (26 per guard, 2 joint, 2 coverage — every guard method individually load-bearing), 2 citation checks

Faults — V1–V5 publish a close; V6–V15 raise. Own namespace so codes never collide with theorem numbers

Seal — $S_R \subseteq S$; carries power per claim with its defect model or (ϵ, N) , (agreeing, k), sampling config, residual; tainted/admissible/check_dependencies are the consumer surface

Rates — metrics/SCHEMA.md RGA section. No numbers

Not proved — PROOFS.md "What is not proved", written first in PREMISE.md §7, extended by round 1

Calibration — rga/calibration.py (C1–C7, faults E1–E9): escapes as counterfactual trials, charges per wrong-verdict cell, install ratchet (incl. class-drop refusal), demotion as query, provenance-stamped seals, re-guarded replay. Rounds: rga-calibration-premise-SYNTHESIS.md, rga-calibration-r1-SYNTHESIS.md (11 confirmed findings, all applied); 71 tests incl. 17 guard deletions and 7 repair traces